Discrete vs. Continuous Visual Representations

How images become tokens,
and how they are represented

Qihang Yu

Amazon FAR

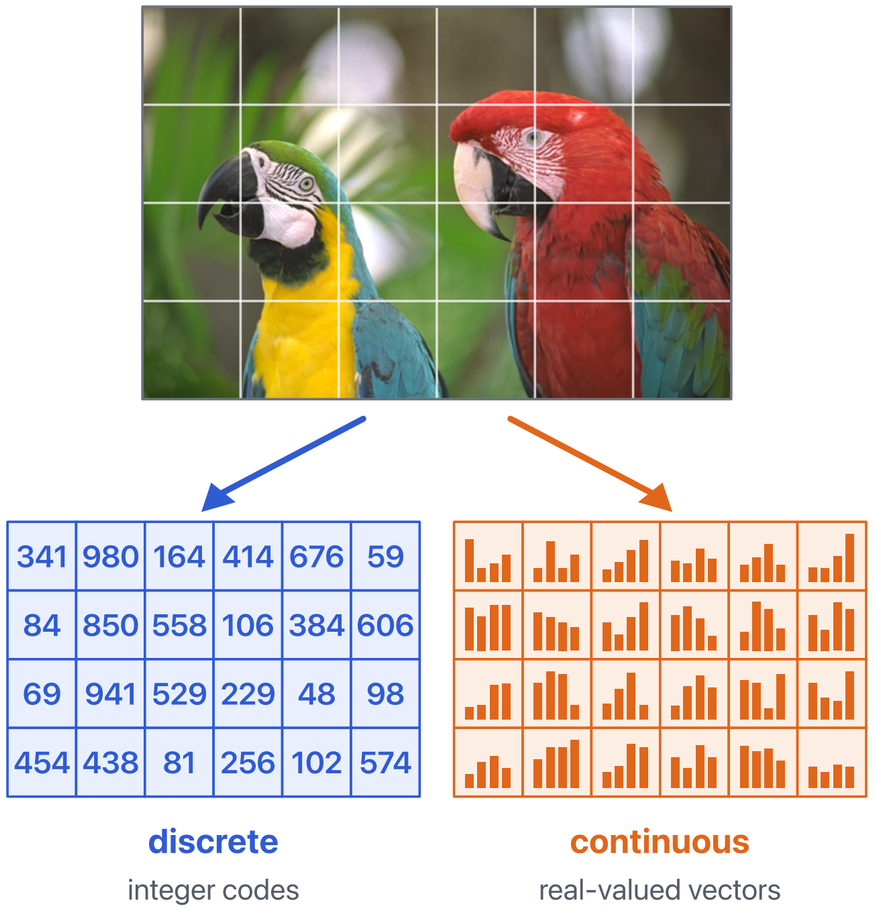


Image: Kodak kodim23

GOAL

Goal of this talk

Provide a **unified view** of discrete and continuous visual representations

GOAL

Goal of this talk

Provide a **unified view** of discrete and continuous visual representations

Spoiler: they may not be that different.

GOAL

Goal of this talk

Provide a **unified view** of discrete and continuous visual representations

Spoiler: they may not be that different.

bits
how much information?

shape
how is it arranged?

generator
can it use it?

ROADMAP

Four questions for today

1 · **Images are not text**

Why can't we tokenize images like text?

Image: Kodak kodim23

ROADMAP

Four questions for today

1 · **Images are not text**

Why can't we tokenize images like text?

2 · **Discrete vs. continuous**

Should a latent be a symbol or a vector?

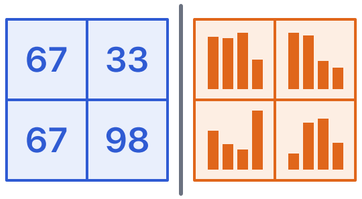
Image: Kodak kodim23

ROADMAP

Four questions for today

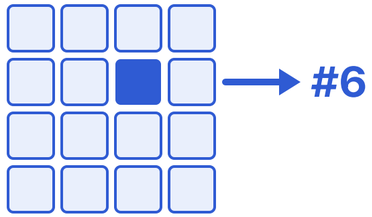
1 · Images are not text

Why can't we tokenize images like text?



2 · Discrete vs. continuous

Should a latent be a symbol or a vector?



3 · The discrete path

How did discrete tokenizers get better?

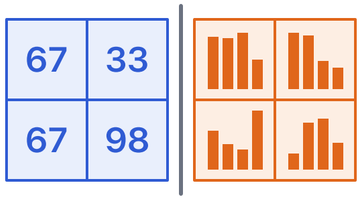
Image: Kodak kodim23

ROADMAP

Four questions for today

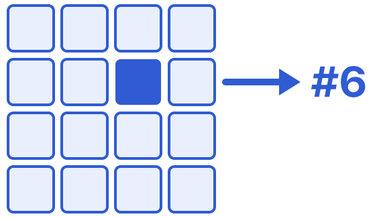
1 · Images are not text

Why can't we tokenize images like text?



2 · Discrete vs. continuous

Should a latent be a symbol or a vector?



3 · The discrete path

How did discrete tokenizers get better?

4 · Bits bridge the gap

What if discrete tokens get enough bits?

Image: Kodak kodim23

01

Images are not text

Why can't we tokenize images the way we tokenize text?

1 Images are not text

2 Discrete vs. continuous

3 The discrete path

4 Bits bridge the gap

Text is a code people made; an image is a measurement of light

TEXT

People designed it. It's already discrete and compact, and each piece means something.

A	␣photo	␣of	␣a	␣cor	gi
32	4590	286	257	1162	12397

␣wearing	␣sunglasses	␣on	␣the
5762	36251	319	262

␣beach	.
10481	13

GPT-2 tokenizer: Radford et al., GPT-2, OpenAI report 2019 · Sec. 2.2 · Ids and round trip: ours · Image: Kodak kodim23

Text is a code people made; an image is a measurement of light

TEXT

People designed it. It's already discrete and compact, and each piece means something.

A	␣photo	␣of	␣a	␣cor	gi
32	4590	286	257	1162	12397

␣wearing	␣sunglasses	␣on	␣the
5762	36251	319	262

␣beach	.
10481	13

The tokenizer is **lossless**: ids → the exact same sentence.

GPT-2 tokenizer: Radford et al., GPT-2, OpenAI report 2019 · Sec. 2.2 · Ids and round trip: ours · Image: Kodak kodim23

Text is a code people made; an image is a measurement of light

TEXT

People designed it. It's already discrete and compact, and each piece means something.

A	␣photo	␣of	␣a	␣cor	gi
32	4590	286	257	1162	12397

␣wearing	␣sunglasses	␣on	␣the
5762	36251	319	262

␣beach	.
10481	13

The tokenizer is **lossless**: ids → the exact same sentence.

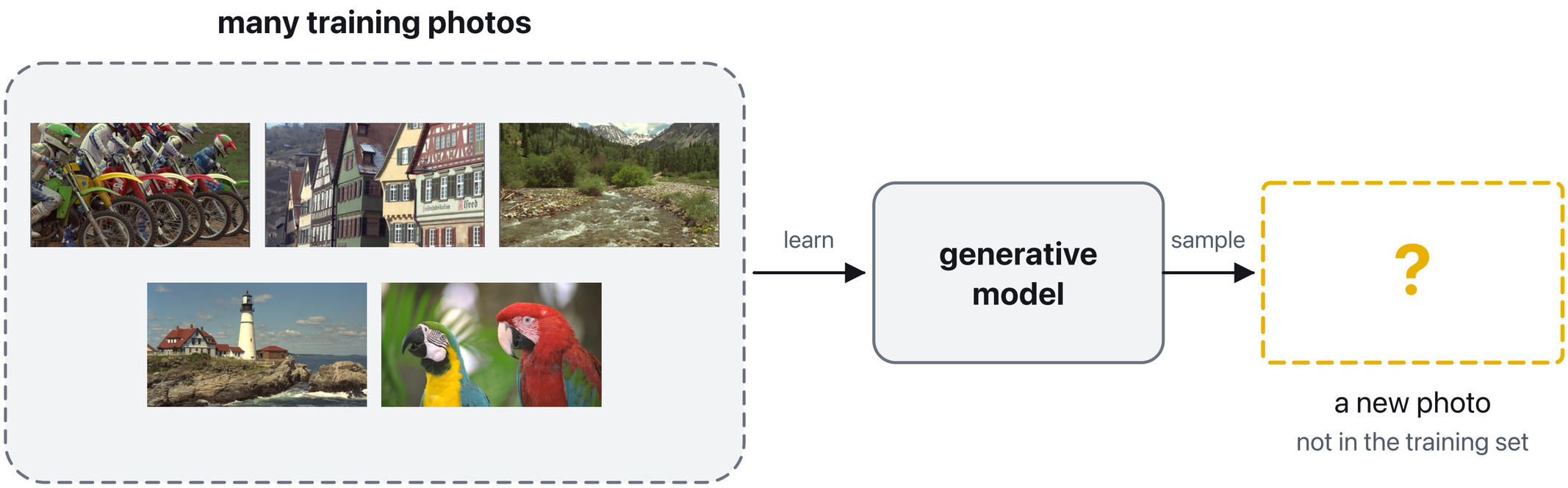
IMAGE

A natural signal. Nobody designed it; it's a measurement of light. Lots of numbers, lots of repetition, and the meaning is in how they're arranged.

GPT-2 tokenizer: Radford et al., GPT-2, OpenAI report 2019 · Sec. 2.2 · Ids and round trip: ours · Image: Kodak kodim23

PRIMER

A generative model makes new examples



Learn from many photos, then sample new ones that look like them.

Teaching illustration: ours · Photos: Kodak kodim05, kodim08, kodim13, kodim21, kodim23

Visual generation is everywhere

Parti-20B

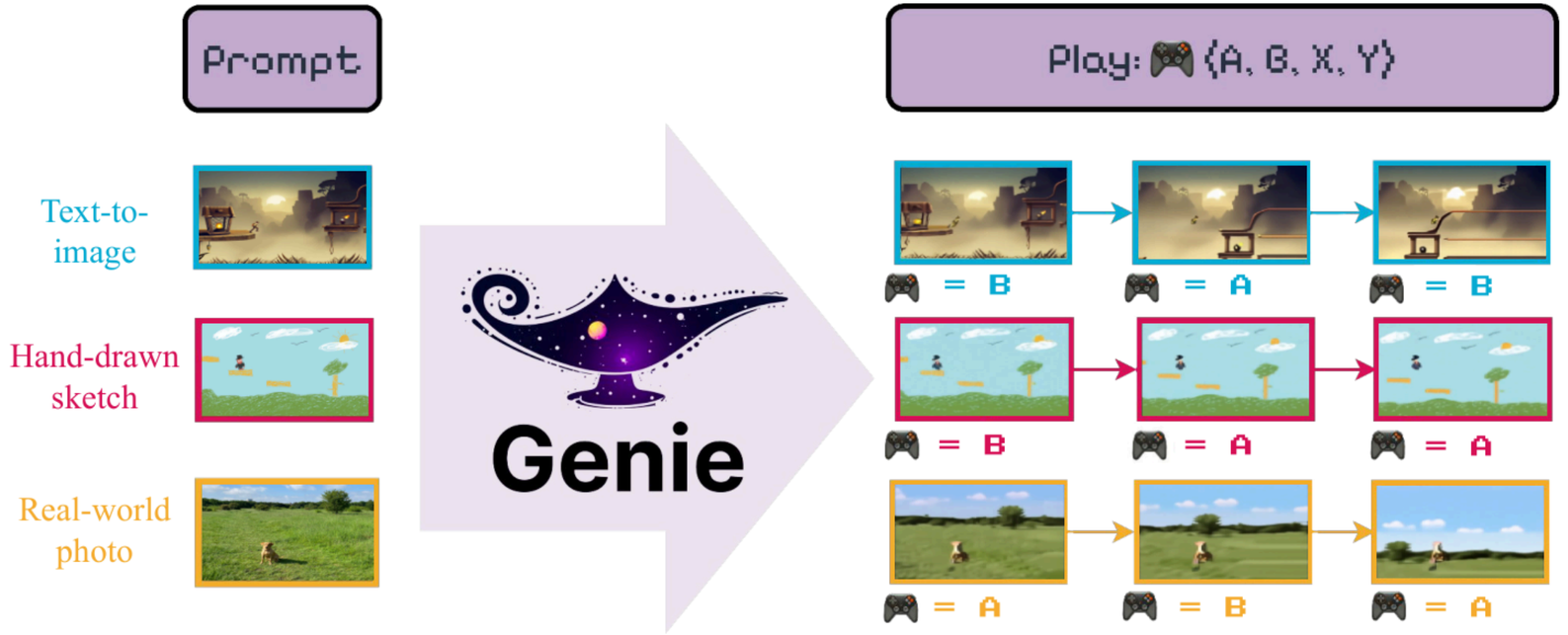
Text to image: a readable sign

Yu et al., Parti, TMLR 2022 · Fig. 10, 20B column · Bruce et al., Genie, ICML 2024 · Fig. 1

Visual generation is everywhere

Parti-20B

Text to image: a readable sign



World model: a playable game from one picture

Also video, robotics, and assistants that both read and draw.

A photo is 196,608 numbers

12 pieces

the caption (51 characters)

196,608 numbers

a 256 × 256 photo: 256 × 256 × 3 (192 KB raw)

≈ 3,900×

more bytes than the caption

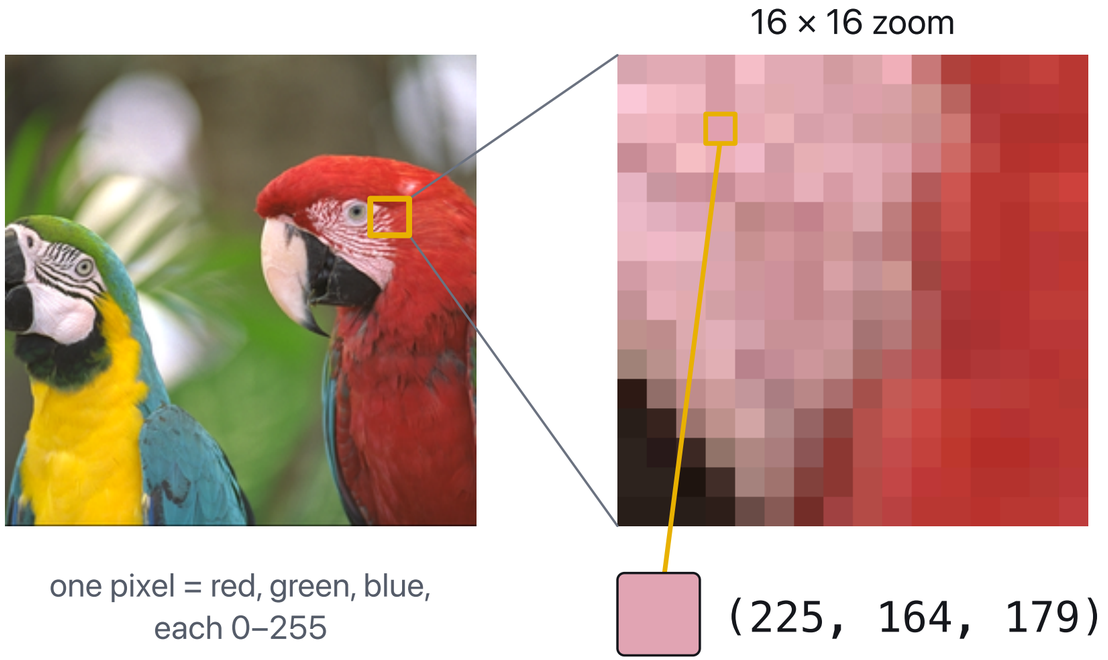
Arithmetic: ours · caption tokenized with GPT-2 (Radford et al., GPT-2, OpenAI report 2019) · Image: Kodak kodim23

A photo is 196,608 numbers

12 pieces
the caption (51 characters)

196,608 numbers
a 256 × 256 photo: 256 × 256 × 3 (192 KB raw)

≈ 3,900×
more bytes than the caption



one pixel = red, green, blue, each 0–255

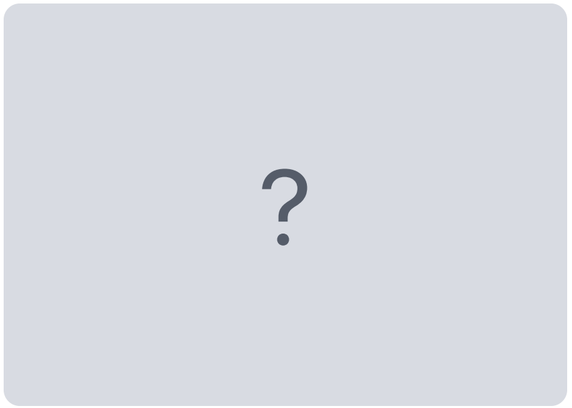
TAKEAWAY

A word piece means something. One pixel value means almost nothing.

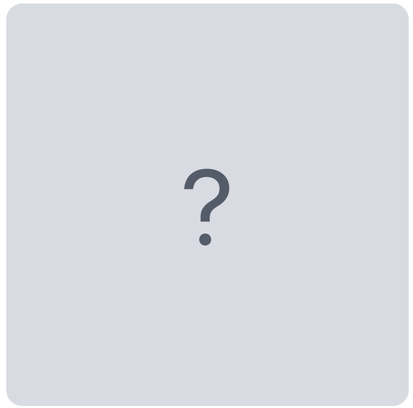
Arithmetic: ours · caption tokenized with GPT-2 (Radford et al., GPT-2, OpenAI report 2019) · Image: Kodak kodim23

Shuffle it, or change it

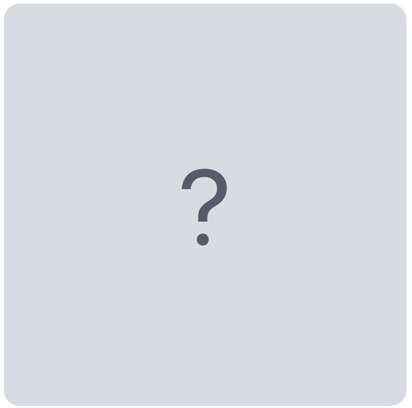
caption words shuffled



16 × 16 patches shuffled



pixels shuffled



Your guess? Which ones can you still read?

Measured: ours · Image: Kodak kodim23 (256 × 256) · noise test: PSNR 42.2 dB

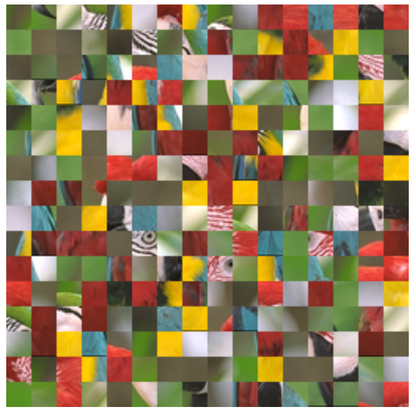
Shuffle it, or change it

caption words shuffled

```
beach the photo
corgi a on
sunglasses of
wearing A
```

still roughly readable

16 × 16 patches shuffled



every patch is still an eye, a beak, feathers

pixels shuffled

pure noise

The meaning of an image lives in the arrangement.

Measured: ours · Image: Kodak kodim23 (256 × 256) · noise test: PSNR 42.2 dB

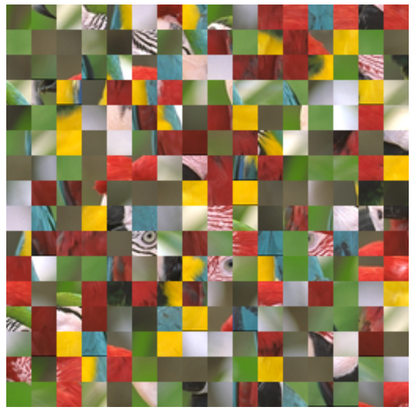
Shuffle it, or change it

caption words shuffled

```
beach the photo
corgi a on
sunglasses of
wearing A
```

still roughly readable

16 × 16 patches shuffled



every patch is still an eye, a beak, feathers

pixels shuffled

pure noise

The meaning of an image lives in the arrangement.

Change one word: "very **red**" → "very **blue**". New meaning.

Change every value by up to ±3: looks the same.

Measured: ours · Image: Kodak kodim23 (256 × 256) · noise test: PSNR 42.2 dB

Neighbouring pixels are nearly identical

a 5 × 5 patch of background

132	133	131	131	129
130	131	130	129	128
131	132	130	127	127
132	131	130	130	127
131	132	131	130	127

its brightness values (0–255)

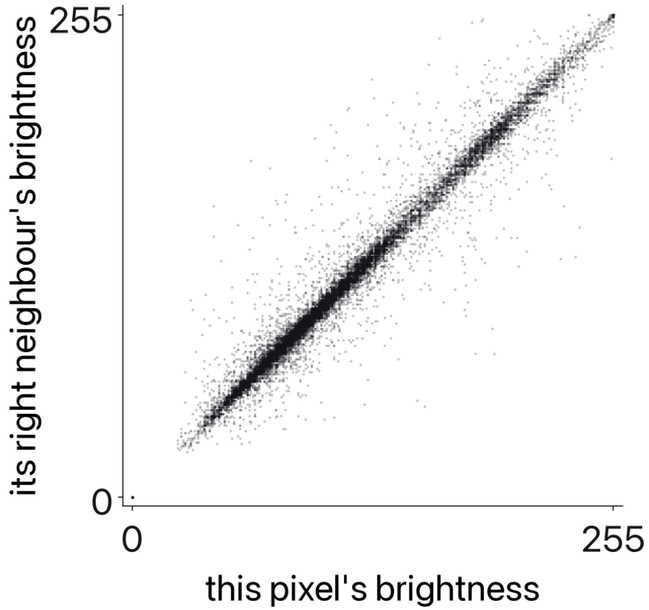
Measured: ours · brightness of Kodak kodim23

Neighbouring pixels are nearly identical

a 5 × 5 patch of background

132	133	131	131	129
130	131	130	129	128
131	132	130	127	127
132	131	130	130	127
131	132	131	130	127

its brightness values (0–255)



each dot: one pixel and the one to its right

TAKEAWAY

Most pixels can be predicted from their neighbours: images are highly redundant.

Measured: ours · brightness of Kodak kodim23

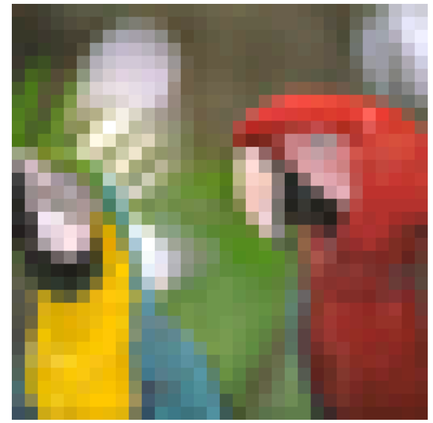
Same parrot, five sizes

512 × 512
786,432 values
PNG 383 KB · JPEG 31 KB

256 × 256
196,608 values
PNG 104 KB · JPEG 10.5 KB

128 × 128
49,152 values
PNG 29 KB · JPEG 3.9 KB

64 × 64
12,288 values
PNG 8.3 KB · JPEG 1.7 KB



32 × 32
3,072 values
PNG 2.4 KB · JPEG 1.0 KB

All shown at the same size; the small ones are enlarged for display. File sizes measured at each native size (JPEG quality 75).

TAKEAWAY

You can still see the parrot at 64 × 64. The other 98.4% of the numbers are low-level detail.

Image: Kodak kodim23, centre 512 crop · Measured: ours (Pillow, LANCZOS downsampling) · KB = 1,024 bytes

Most of an image is detail the eye ignores

Original: 768 KB raw

Lossless (PNG): only **2×** smaller

JPEG quality 75: 31 KB

25× smaller, looks the same

JPEG quality 5: 6.5 KB

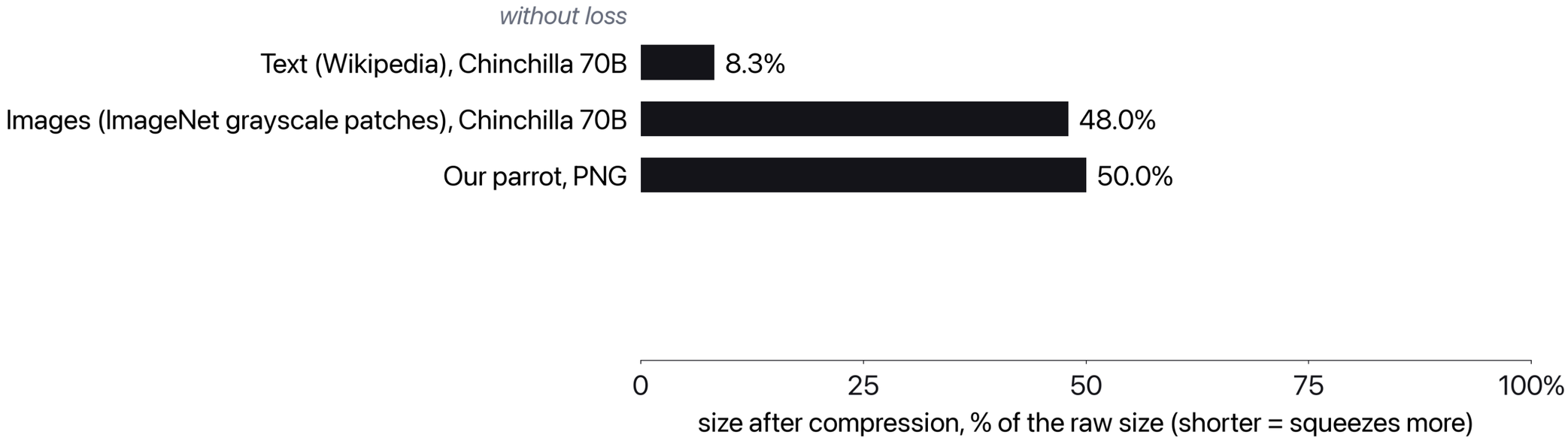
119× smaller, visible blocks

TAKEAWAY

Lossless must keep everything. Lossy (JPEG) drops detail the eye won't miss.

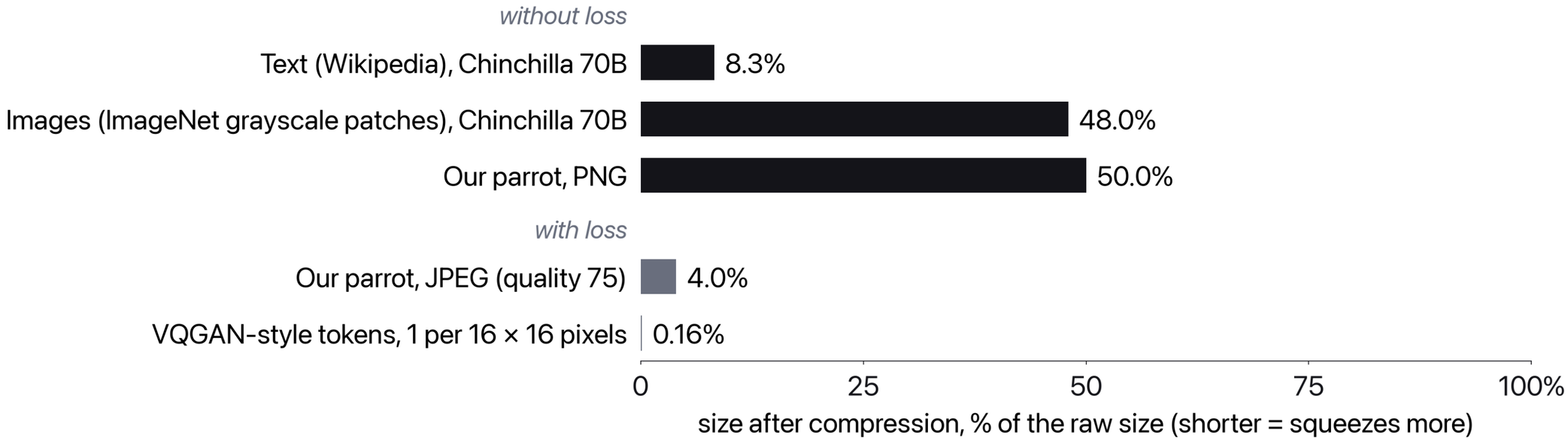
Measured: ours (Pillow PNG, JPEG q75 / q5) on the 512 × 512 Kodak kodim23 crop; panels show a zoomed part

Text compresses without loss. Images barely do.



Delétang et al., ICLR 2024, Tab. 1 · PNG, JPEG, tokens: ours · Kodak kodim23

Text compresses without loss. Images barely do.



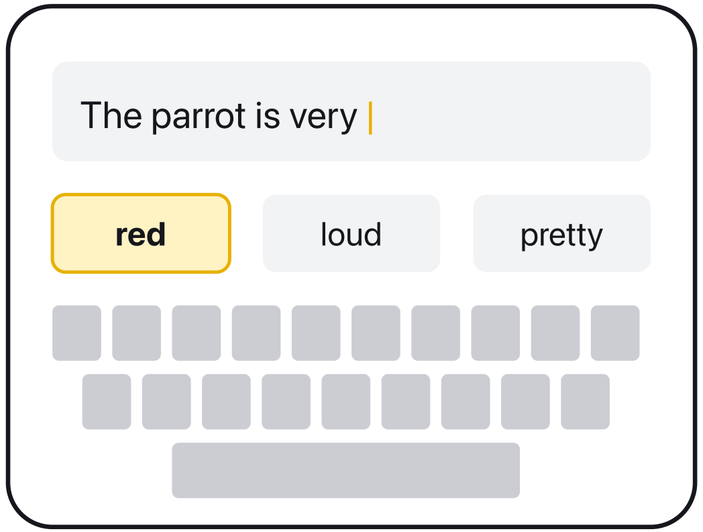
TAKEAWAY

Text: every character matters, so we tokenize it without loss. Images: lossless barely shrinks them; the big savings come from dropping detail.

Delétang et al., ICLR 2024, Tab. 1 · PNG, JPEG, tokens: ours · Kodak kodim23

PRIMER

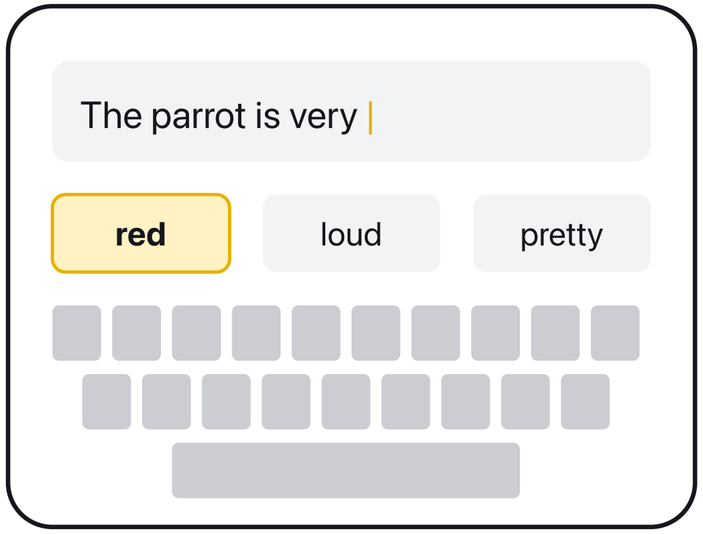
Autoregressive = autocomplete, one piece at a time



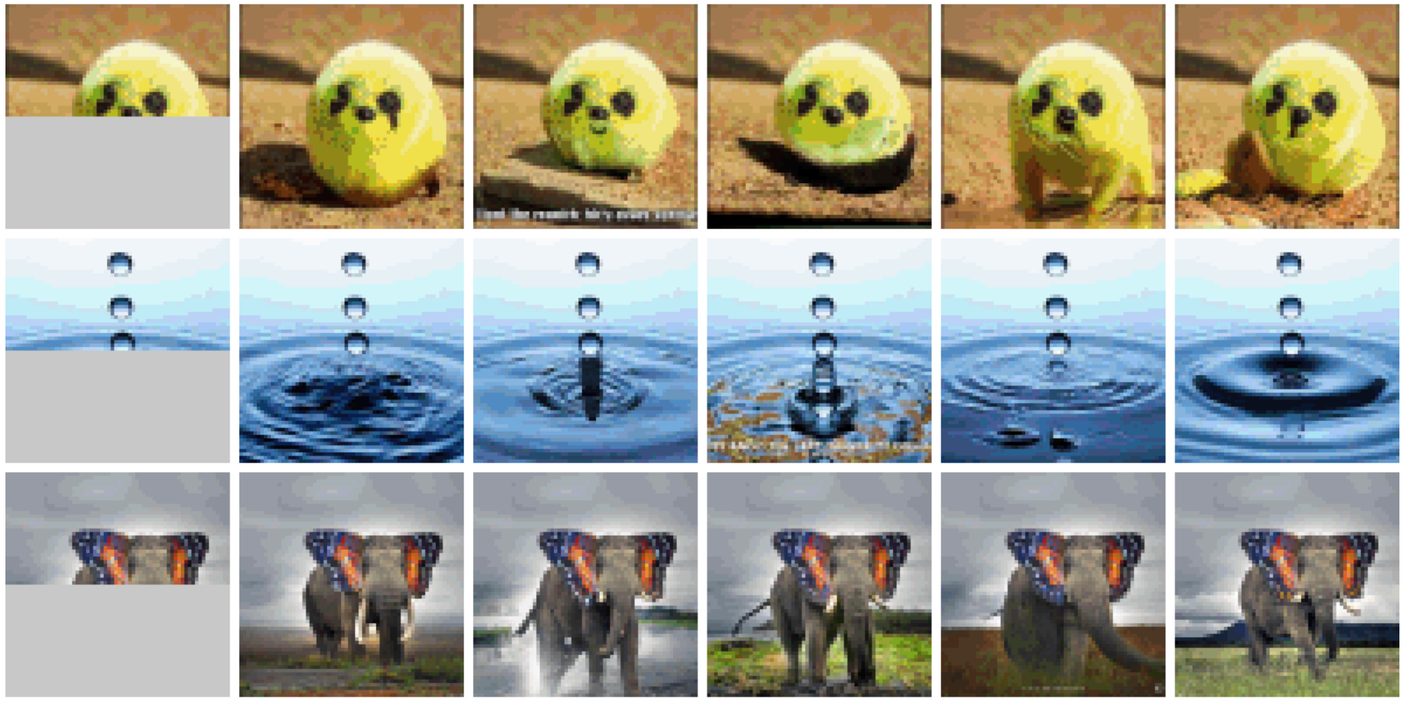
Text: the next word

van den Oord et al., PixelRNN, ICML 2016 · Chen et al., iGPT, ICML 2020 · word counts: ours

Autoregressive = autocomplete, one piece at a time



Text: the next word

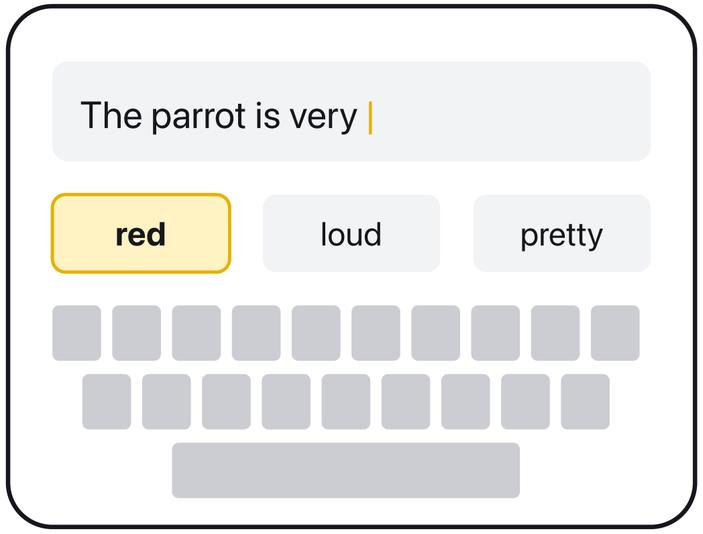


Images: the next pixel, row by row. iGPT gets the top half (left) and draws the rest.

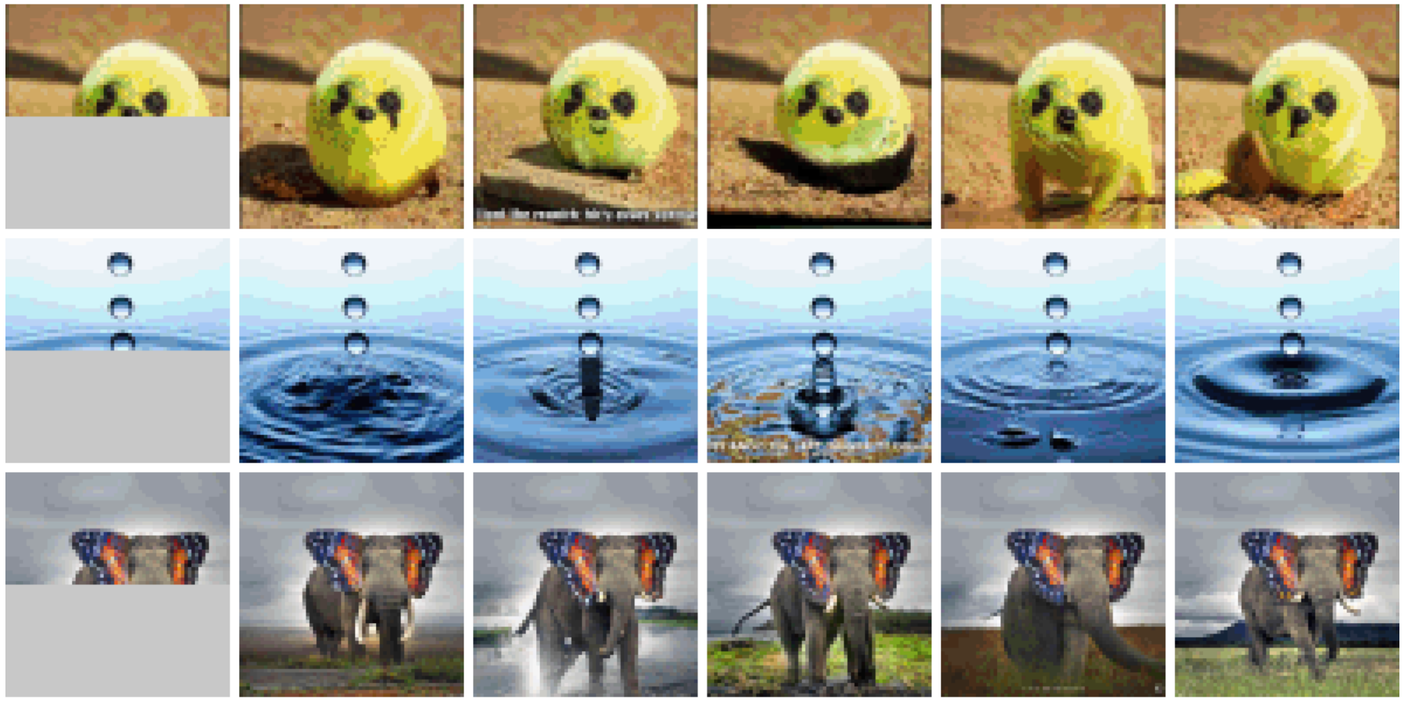
right: original · Chen et al., Image GPT blog 2020 · Input column: ours

van den Oord et al., PixelRNN, ICML 2016 · Chen et al., iGPT, ICML 2020 · word counts: ours

Autoregressive = autocomplete, one piece at a time



Text: the next word



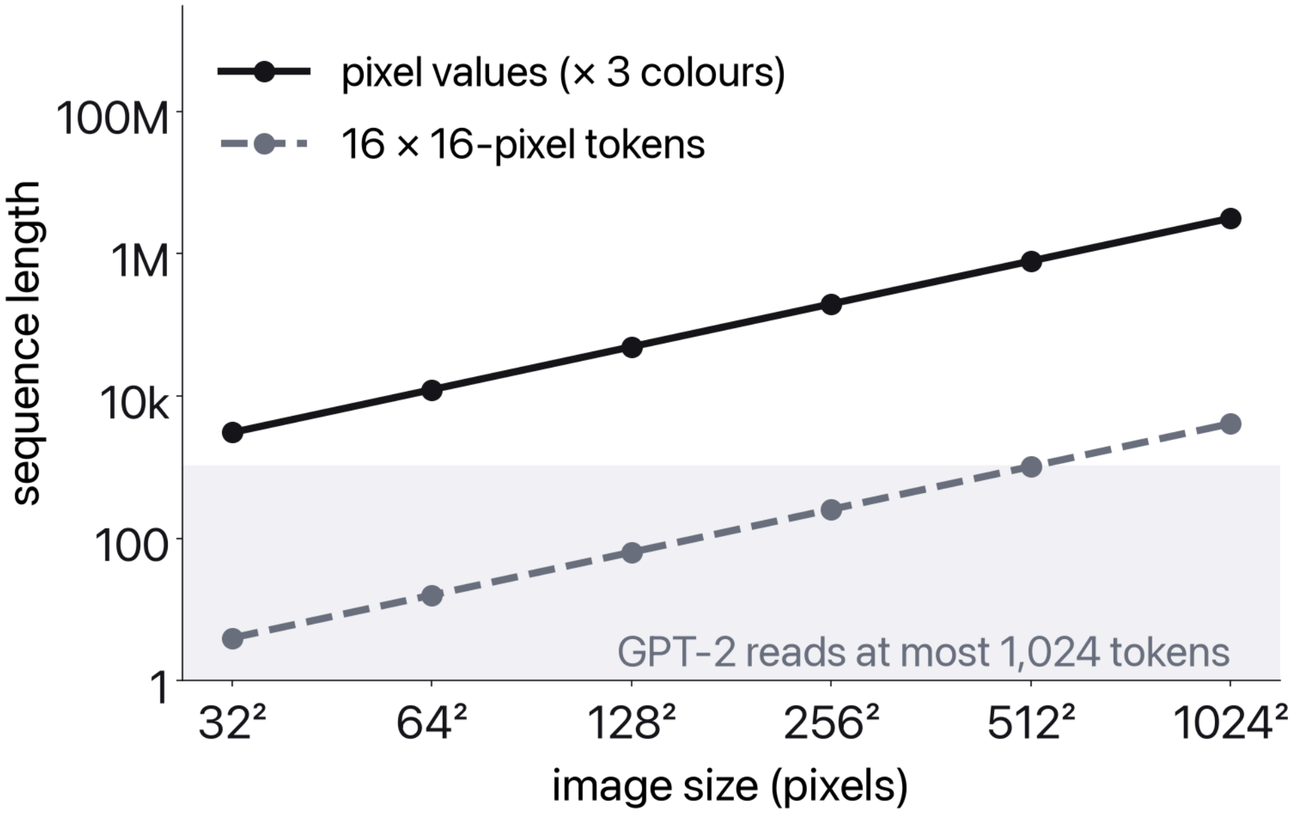
Images: the next pixel, row by row. iGPT gets the top half (left) and draws the rest.

right: original · Chen et al., Image GPT blog 2020 · Input column: ours

12,288 steps → one 64 × 64 thumbnail. **12,288 text tokens** → a ~30-page short story.

van den Oord et al., PixelRNN, ICML 2016 · Chen et al., iGPT, ICML 2020 · word counts: ours

Why doesn't it scale?

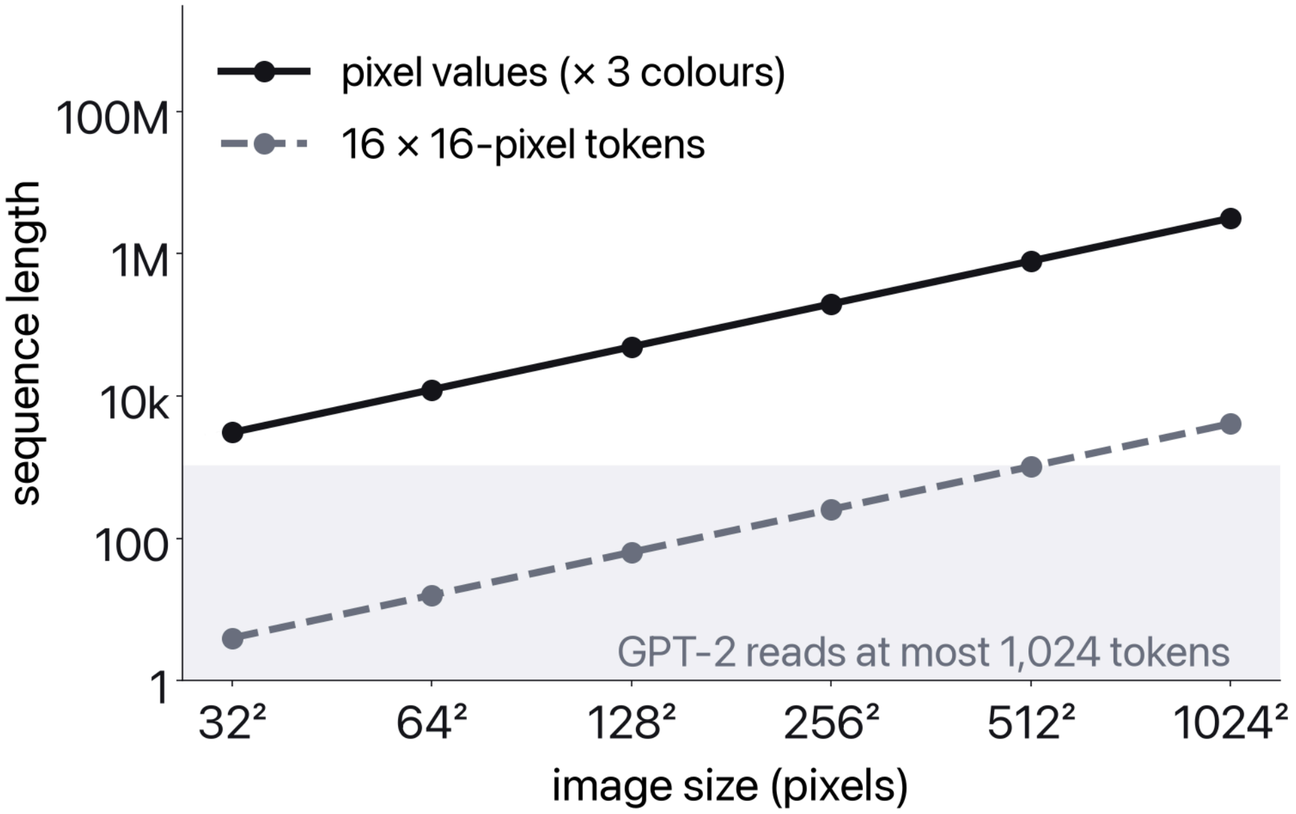


Sequence length vs. image size

A 256 × 256 image is **196,608** pixel values. One small photo; as text, a whole novel.

Arithmetic: ours · GPT-2 context 1,024 (Radford et al., GPT-2, OpenAI report 2019 · Sec. 2.3)

Why doesn't it scale?



Sequence length vs. image size

A 256 × 256 image is **196,608** pixel values. One small photo; as text, a whole novel.

Transformers (the network behind GPT) use attention: every token looks at every other token, so the cost grows with the **square** of the length (2× longer → 4× the compute).

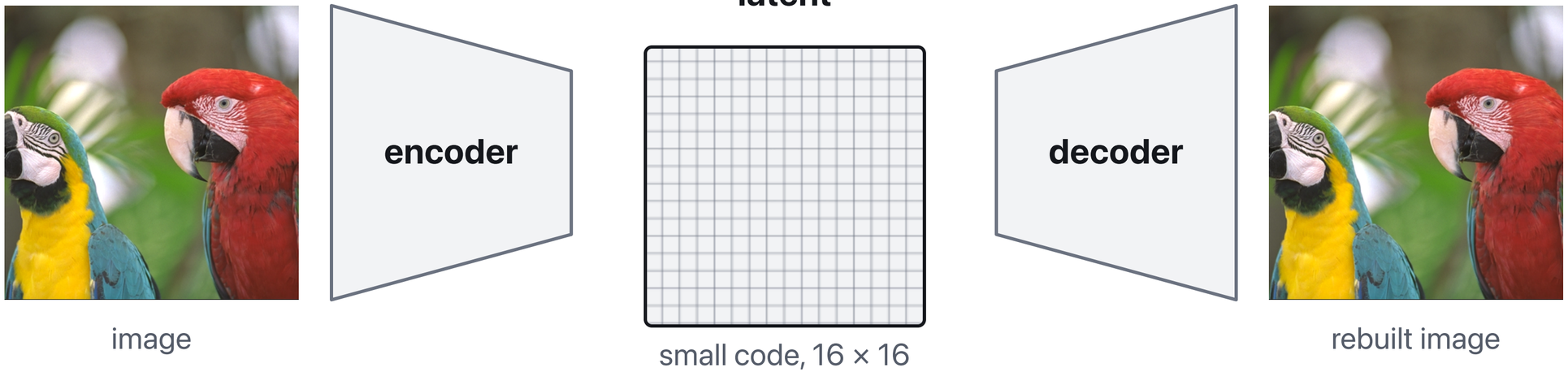
TAKEAWAY

Compress first, then model.

Arithmetic: ours · GPT-2 context 1,024 (Radford et al., GPT-2, OpenAI report 2019 · Sec. 2.3)

PRIMER

An autoencoder squeezes, then rebuilds



image

small code, 16 × 16

rebuilt image

No labels needed. Smaller code, blurrier rebuild:

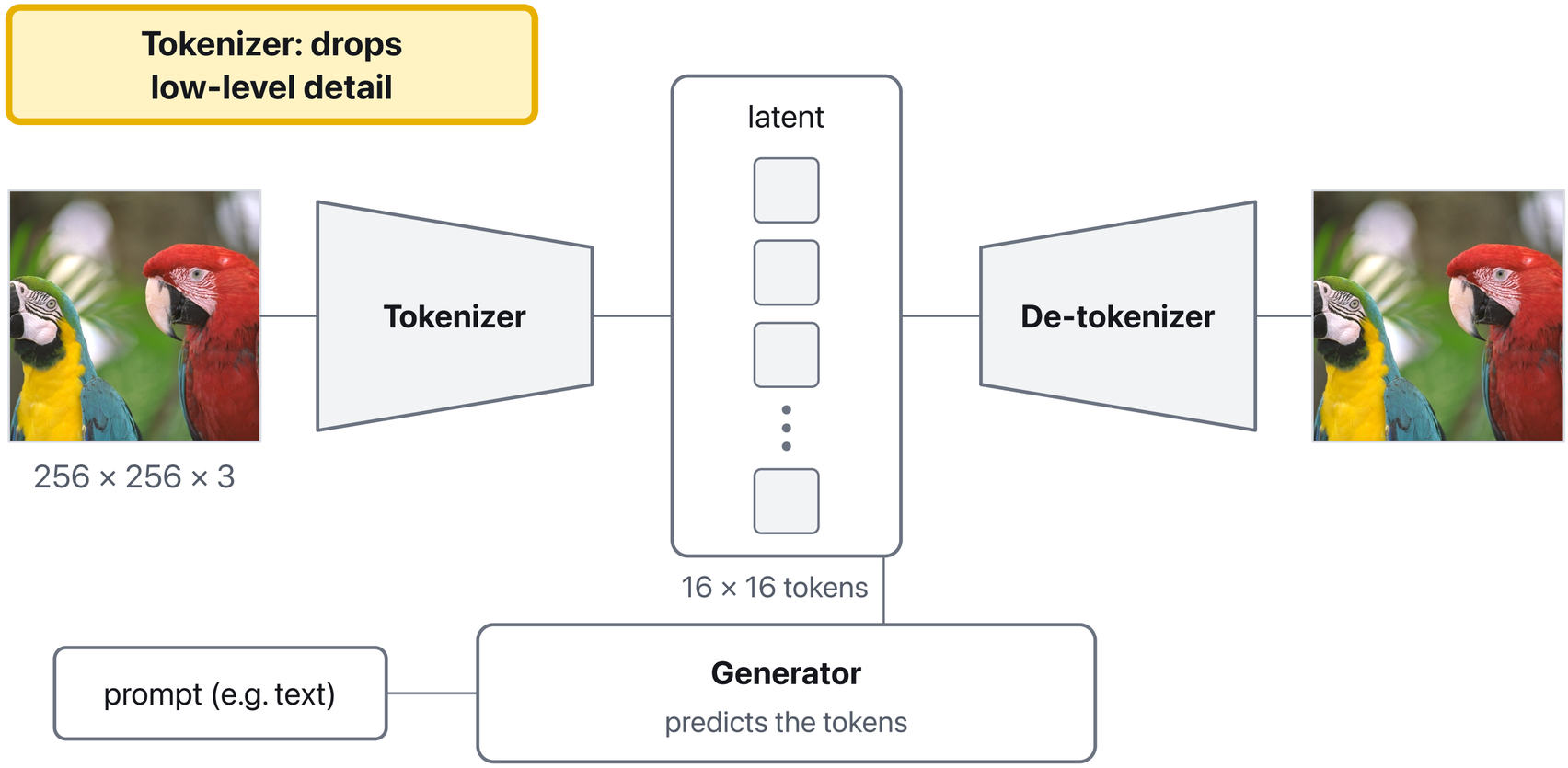
64 × 64

32 × 32

16 × 16

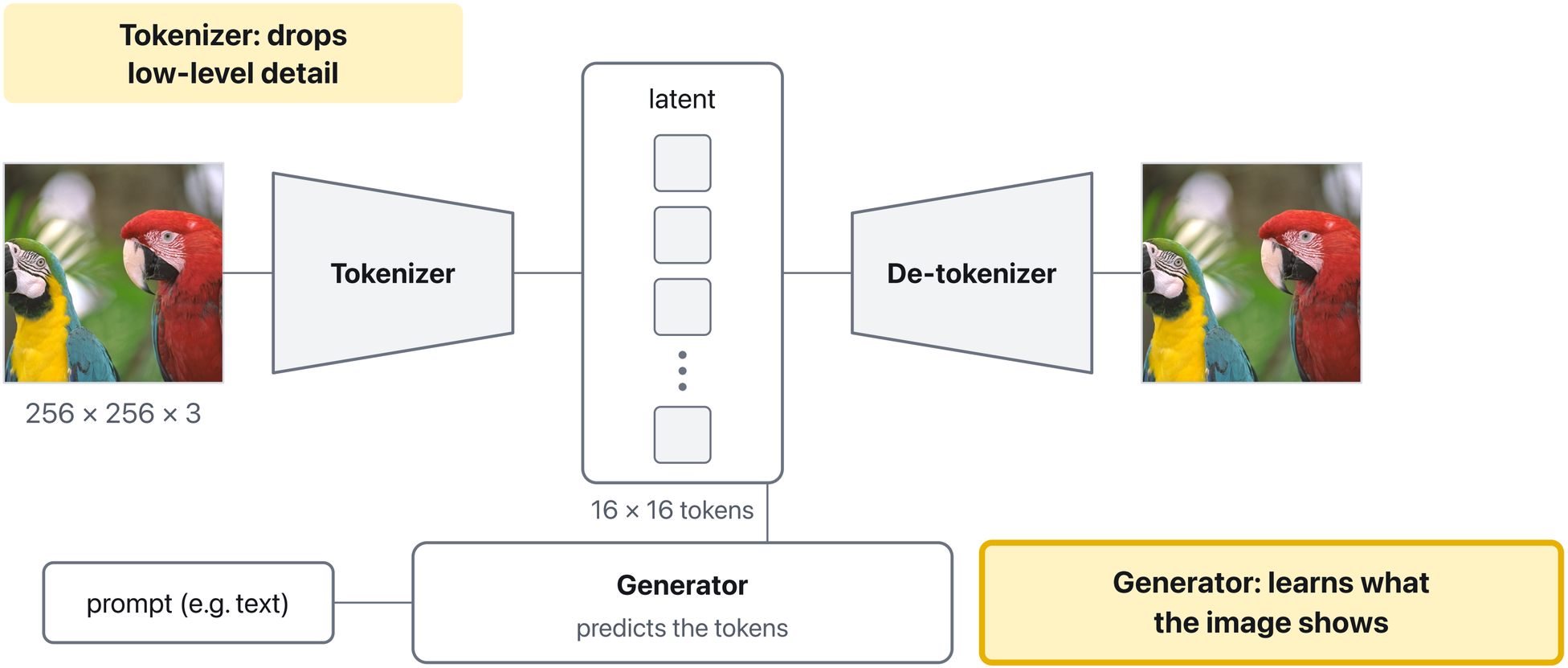
Teaching illustration: ours · rebuilds made by plain resizing, not a trained network · Image: Kodak kodim23

Compress first, then generate



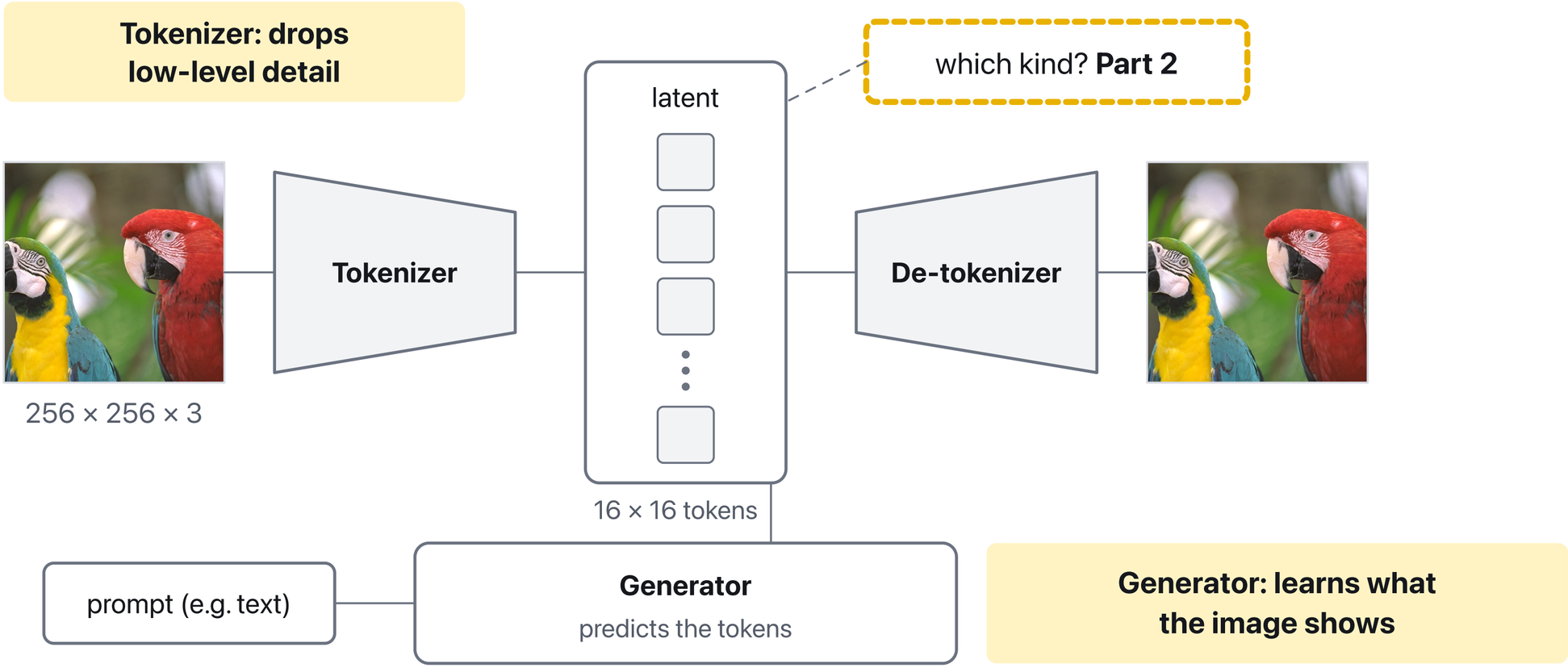
The standard route today, and by far the cheapest one.

Compress first, then generate



The standard route today, and by far the cheapest one.

Compress first, then generate



The standard route today, and by far the cheapest one.

Part 1 in three lines

`A` `␣photo` `␣of`

Text: a human-made code, so a **lossless** tokenizer is enough.

Part 1 in three lines

`A` `␣photo` `␣of`

Text: a human-made code, so a **lossless** tokenizer is enough.

Same parrot, 512 down to 32 pixels

Images: natural signals. Huge, redundant, mostly low-level detail, so a lossless “BPE for pixels” doesn't work.

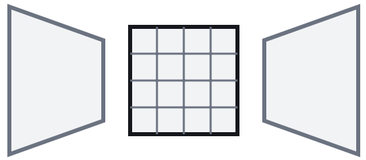
Part 1 in three lines

`A` `␣photo` `␣of`

Text: a human-made code, so a **lossless** tokenizer is enough.

Same parrot, 512 down to 32 pixels

Images: natural signals. Huge, redundant, mostly low-level detail, so a lossless “BPE for pixels” doesn't work.



So we compress: tokenize **lossily** into a latent, then generate there.

02

Discrete vs. continuous

Should the latent be a symbol or a vector?

1 Images are not text ✓

2 Discrete vs. continuous

3 The discrete path

4 Bits bridge the gap

Two families of latents

Discrete tokens

487	267	757	970
134	26	960	531
479	392	963	975
813	308	474	310

One whole number per position: an index into a list.

Generator: next slides

Teaching illustration: ours, after NVIDIA et al., Cosmos World Foundation Model Platform for Physical AI, arXiv 2025 · Fig. 7

Two families of latents

Discrete tokens

487	267	757	970
134	26	960	531
479	392	963	975
813	308	474	310

One whole number per position: an index into a list.

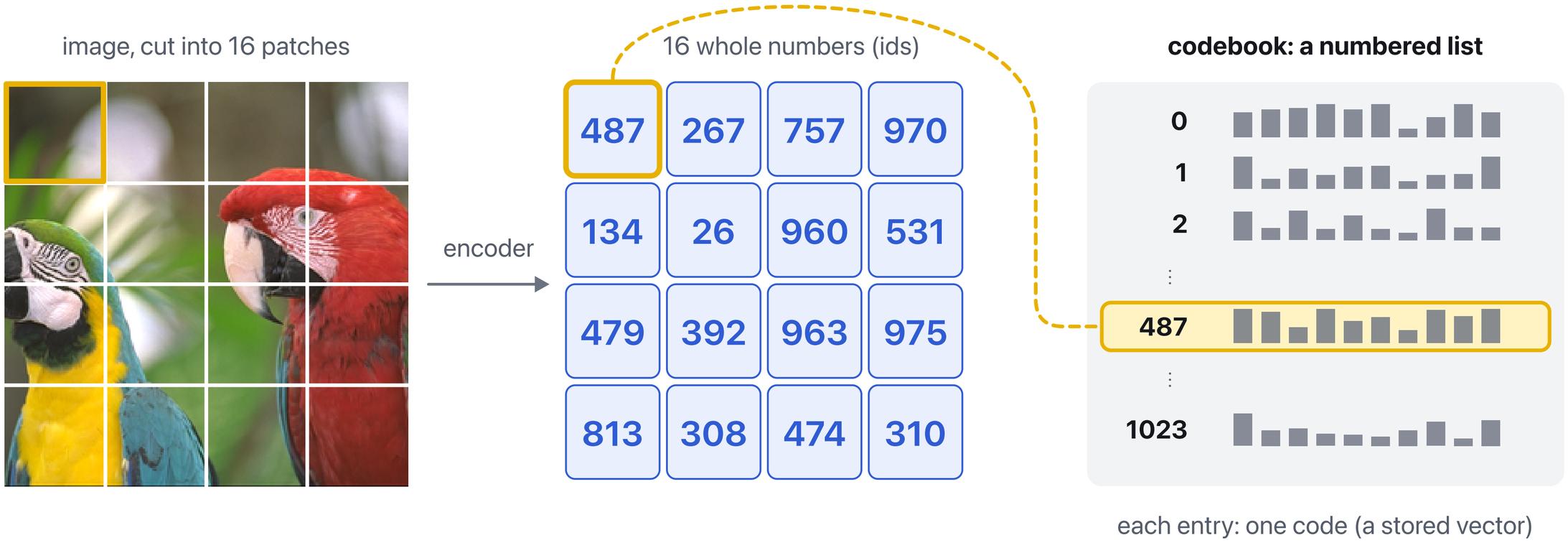
Generator: next slides

Continuous latents

A short list of real numbers (a vector) per position. Each number is a **channel**.

Generator: next slides

What does a discrete tokenizer look like?

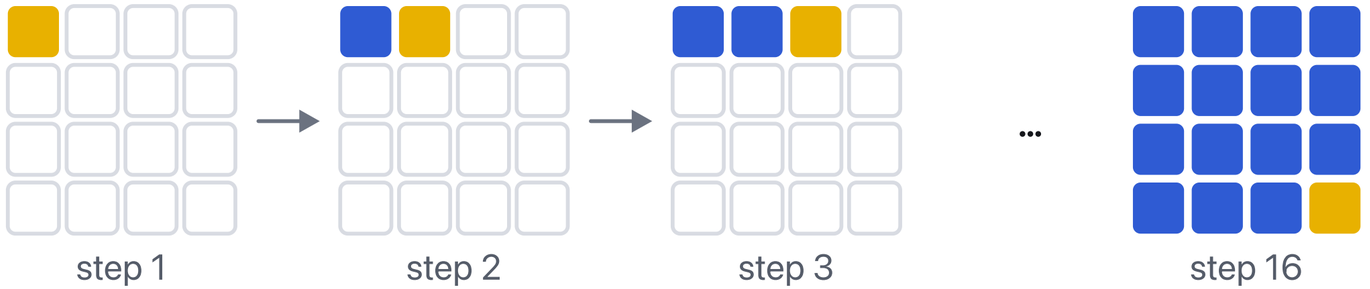


Each patch is rounded to the nearest code in a fixed list, the codebook, and we keep only its number. How the codebook is learned: Part 3.

Teaching illustration: ours, after Yu et al., ViT-VQGAN, ICLR 2022 · Fig. 1 · Image: Kodak kodim23

Two ways to generate discrete tokens

In order
autoregressive



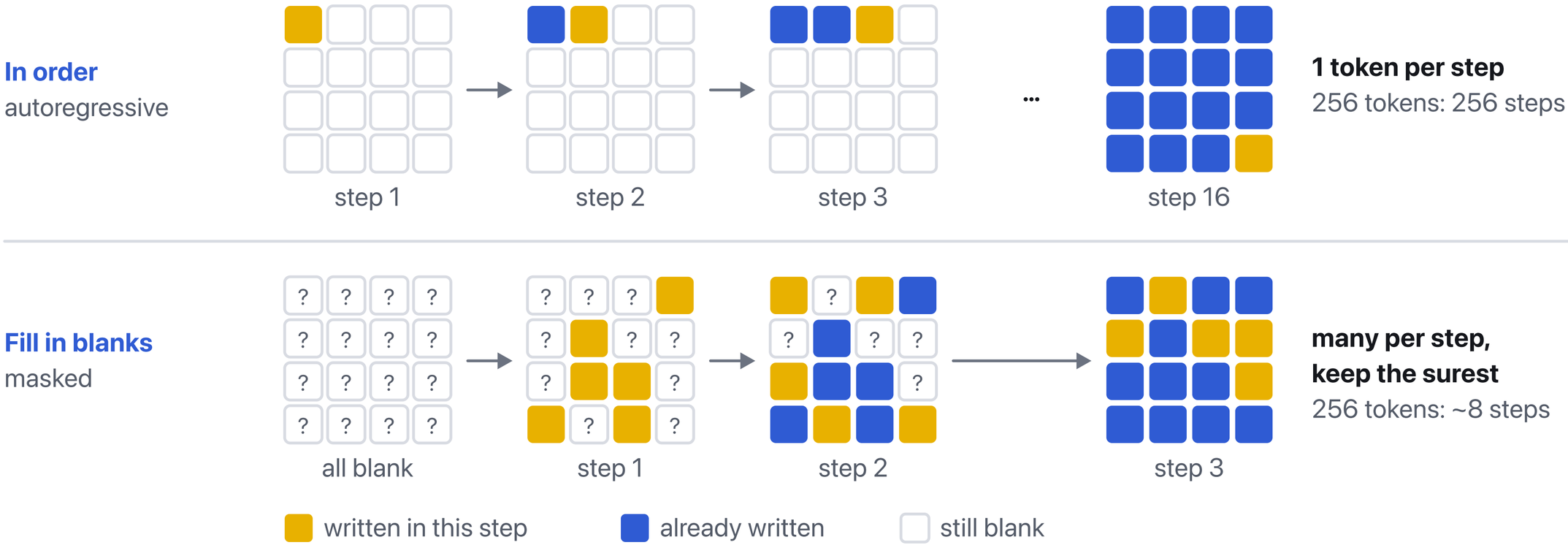
1 token per step
256 tokens: 256 steps

written in this step already written still blank

Output at each position: a softmax over all codes (one score per code, turned into probabilities), like a classifier

Teaching illustration: ours, after Chang et al., MaskGIT, CVPR 2022 · Fig. 2

Two ways to generate discrete tokens

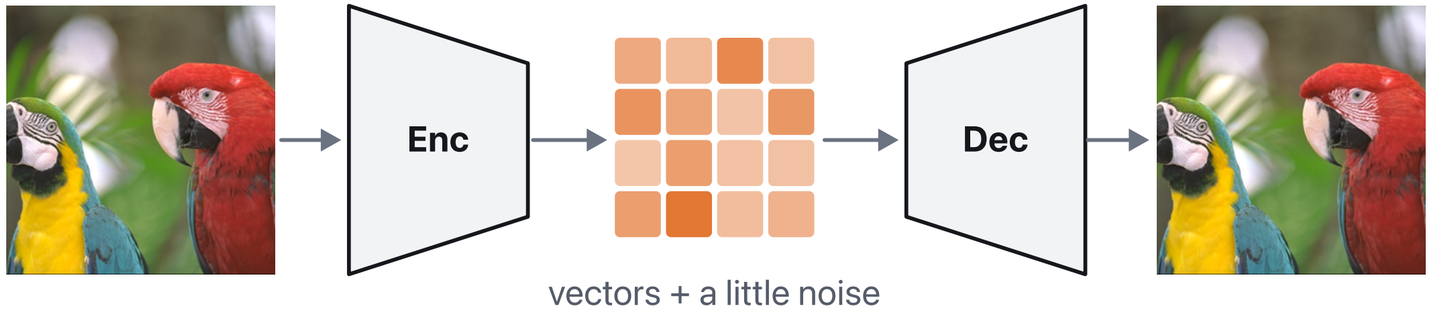


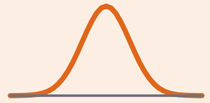
In text: “A human is [MASK] a dog.” (BERT) · Output at each position: a softmax over all codes (one score per code, turned into probabilities), like a classifier

Teaching illustration: ours, after Chang et al., MaskGIT, CVPR 2022 · Fig. 2

And a continuous tokenizer?

Tokenizer: an autoencoder with a slightly fuzzy code





a very weak pull toward a bell curve (“KL”)

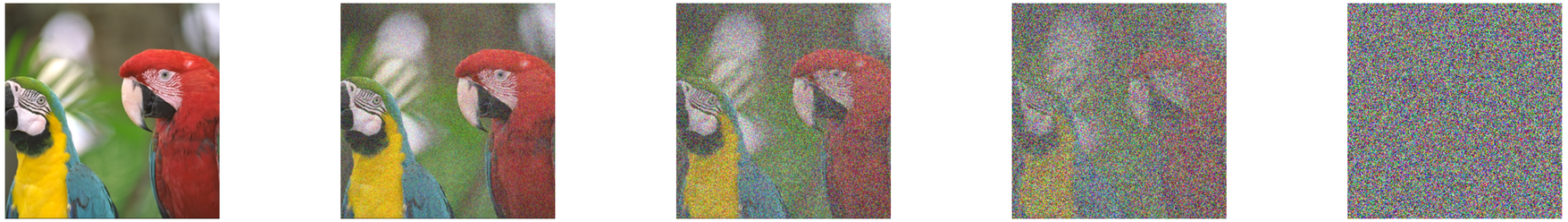
a VAE
(variational autoencoder)

No codebook and no rounding: each position keeps its vector.

Teaching illustration: ours, after Rombach et al., LDM, CVPR 2022 · Sec. 3.1, App. G · Image: Kodak kodim23

Diffusion cleans up continuous latents step by step

add a little noise, again and again (nothing to learn)



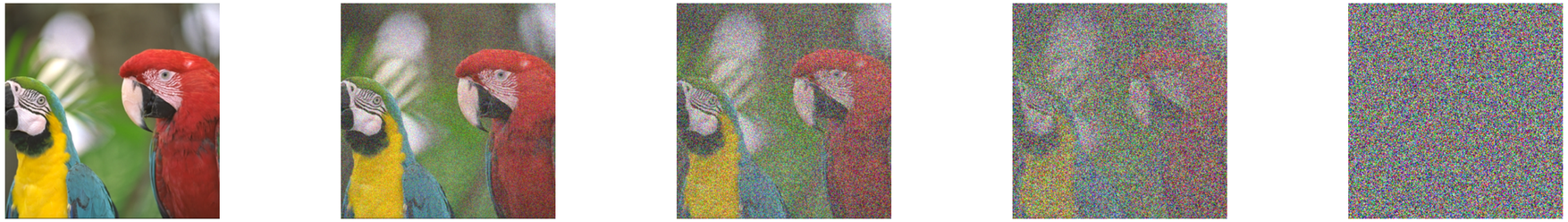
100% image | 75% image | 50% image | 25% image | 0%: pure noise

generate: remove a little noise per step

Teaching illustration: ours, after Ho et al., DDPM, NeurIPS 2020 · Rombach et al., LDM, CVPR 2022 · Image: Kodak kodim23

Diffusion cleans up continuous latents step by step

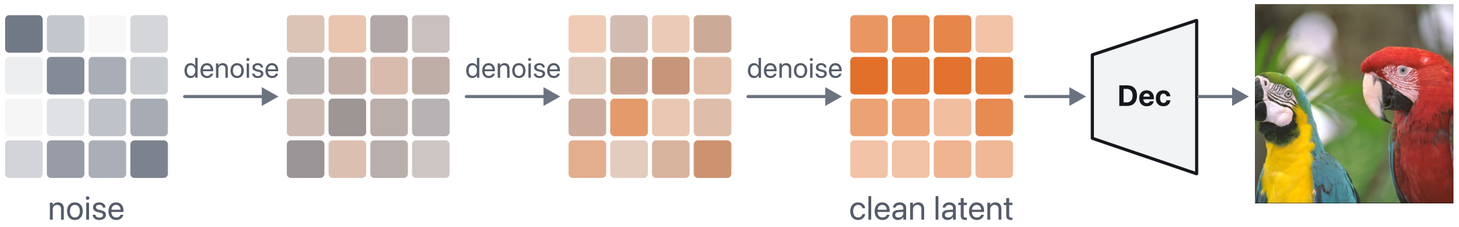
add a little noise, again and again (nothing to learn)



generate: remove a little noise per step

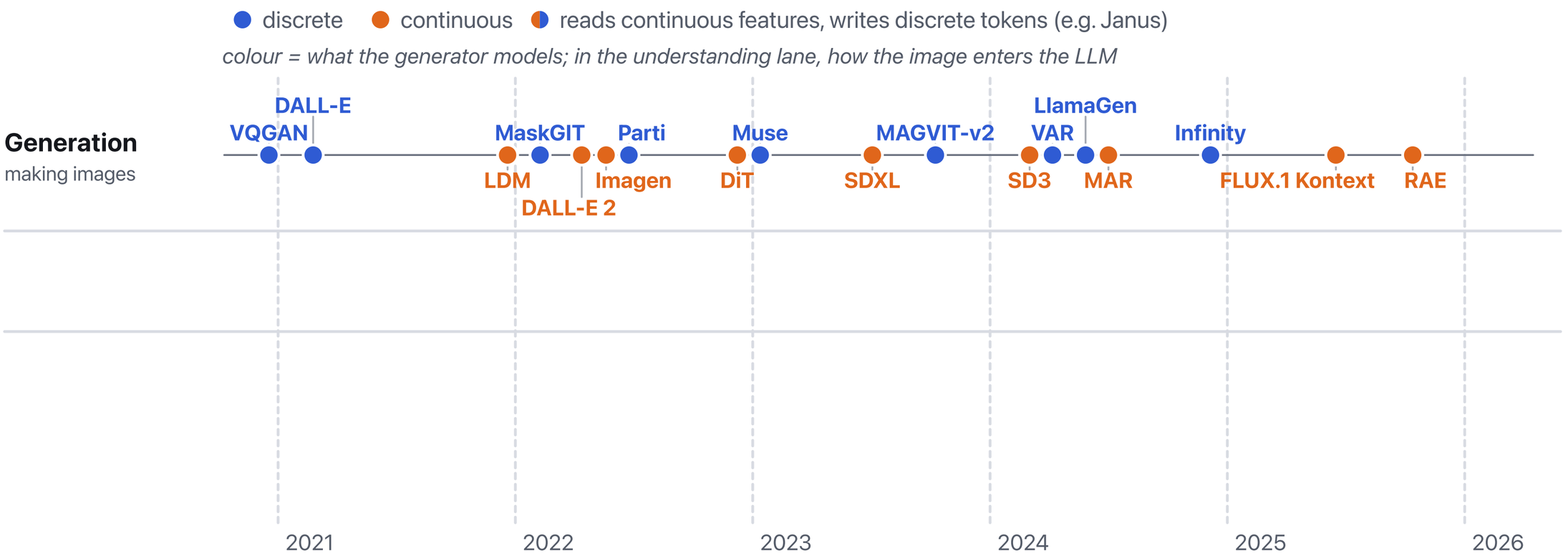
Generator: diffusion on the latent grid

each pass outputs a vector (the predicted noise)



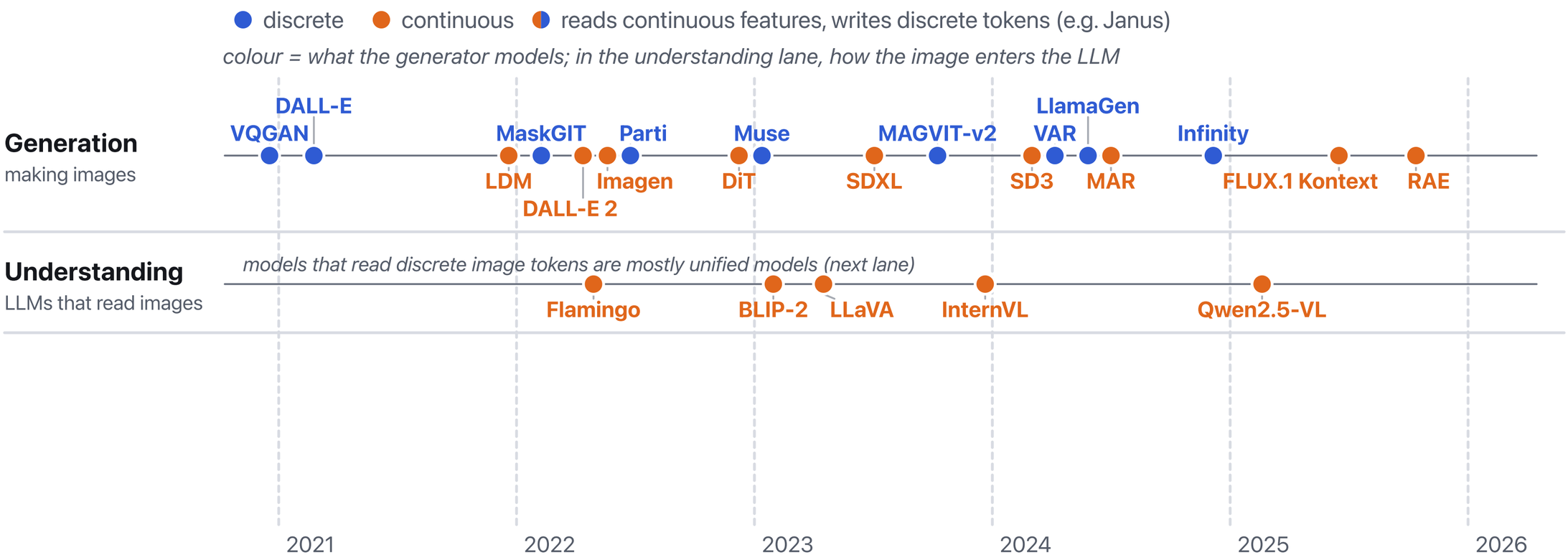
denoise several times; then the decoder runs once

Both families keep moving



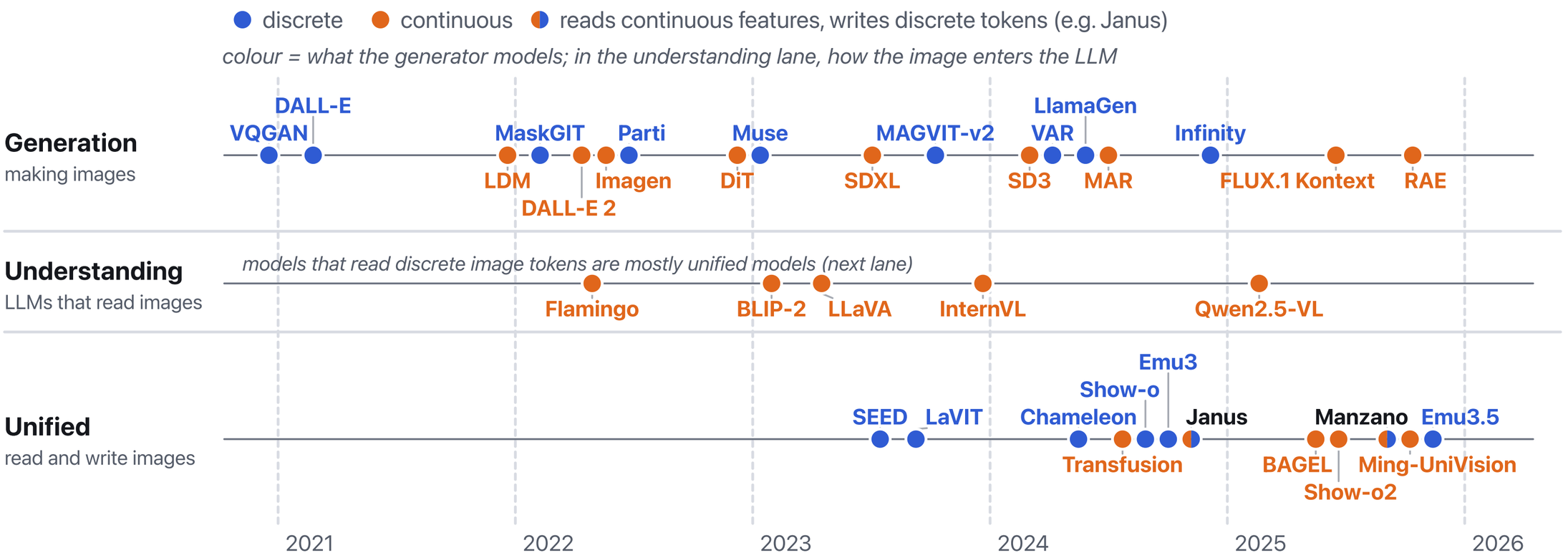
Dates: arXiv v1 of each paper (FLUX: the FLUX.1 Kontext report, 2025) · a selection, not a complete list

Both families keep moving



Dates: arXiv v1 of each paper (FLUX: the FLUX.1 Kontext report, 2025) · a selection, not a complete list

Both families keep moving



Continuous leads in volume and in most benchmarks, but both are still active, and unified models use both.

Dates: arXiv v1 of each paper (FLUX: the FLUX.1 Kontext report, 2025) · a selection, not a complete list

Why discrete?

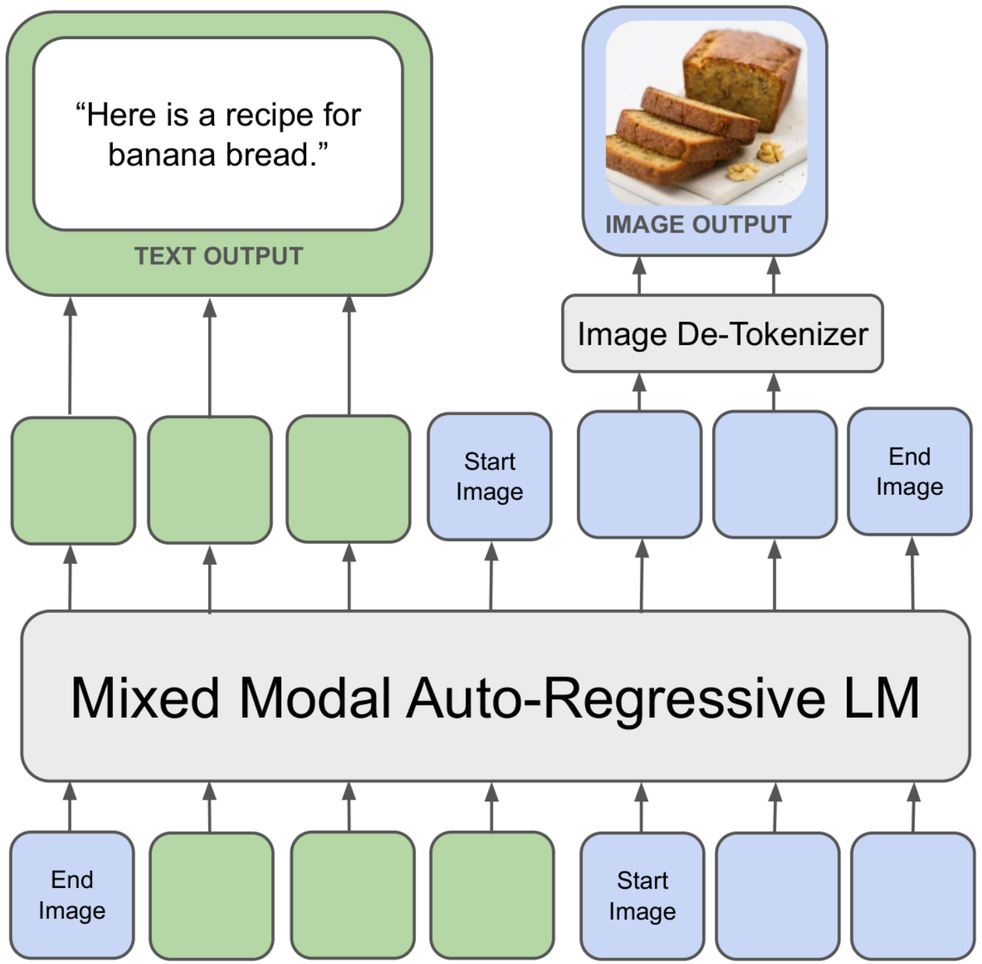
Easy for the generator
Pick the next token from a list, like a chatbot (one softmax)

Chameleon Team, Chameleon: Mixed-Modal Early-Fusion Foundation Models, arXiv 2024 · Fig. 1b

Why discrete?

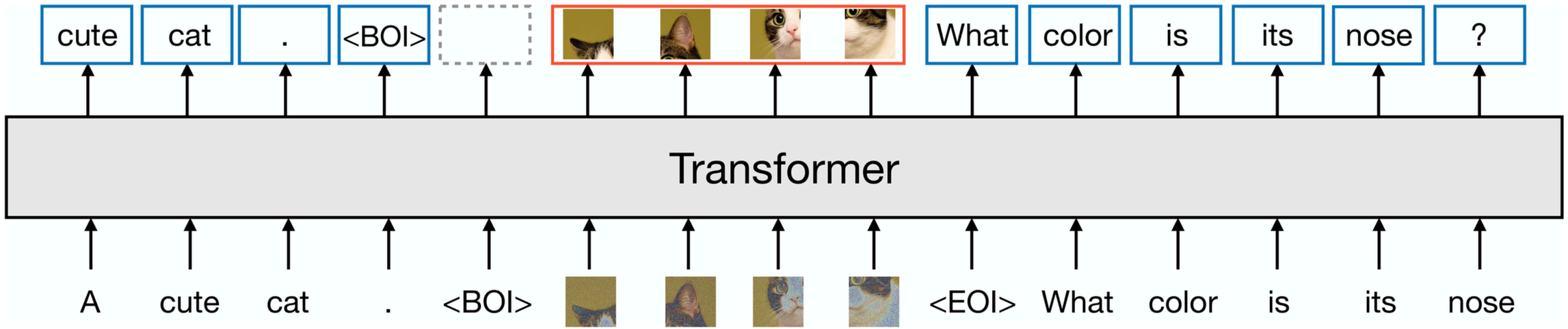
Easy for the generator
Pick the next token from a list, like a chatbot (one softmax)

Lines up with text
One stream of tokens for words and images



Chameleon: one model writes text tokens (green) and image tokens (blue)

Why continuous?



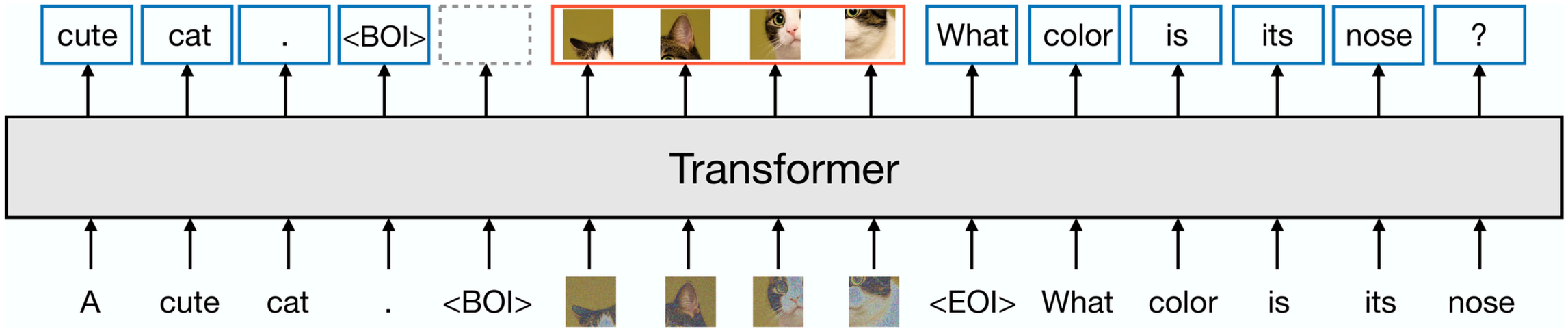
Transfusion: one model, text by next token, images by diffusion

<BOI> / <EOI> = begin / end of image

No rounding

Less information lost

Why continuous?



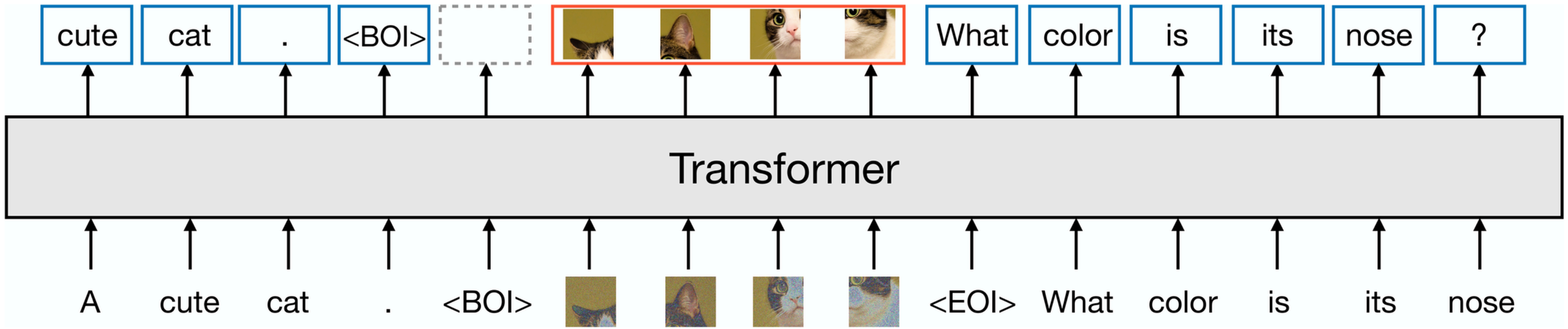
Transfusion: one model, text by next token, images by diffusion

<BOI> / <EOI> = begin / end of image

No rounding
Less information lost

Natural for raw signals
Pixels and sound are continuous

Why continuous?



Transfusion: one model, text by next token, images by diffusion

<BOI> / <EOI> = begin / end of image

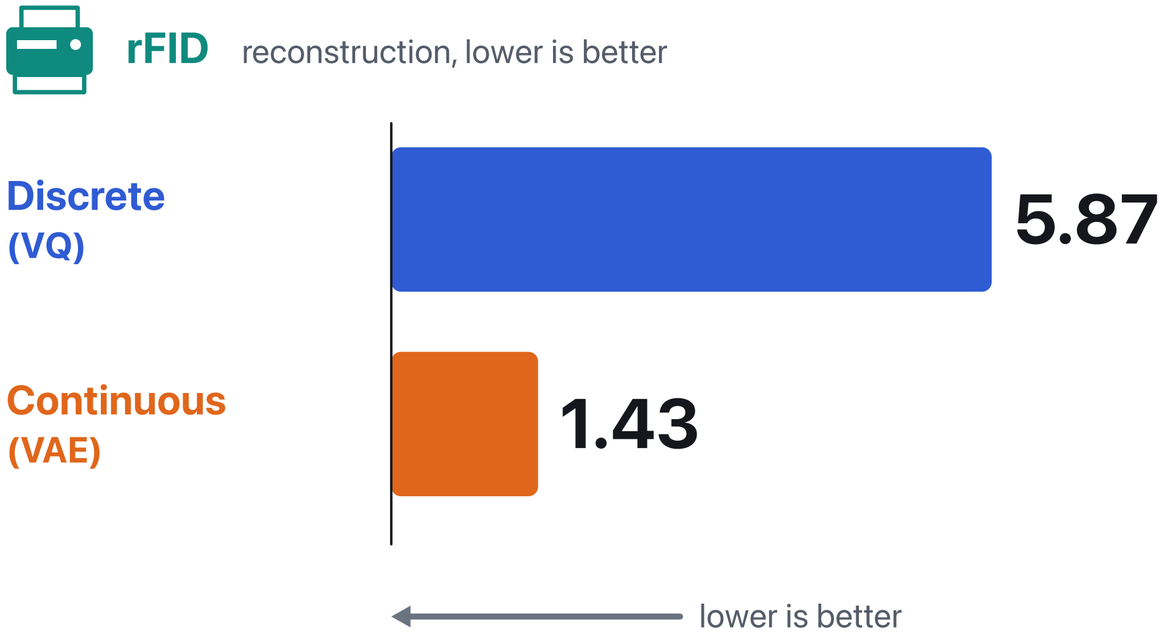
No rounding
Less information lost

Natural for raw signals
Pixels and sound are continuous

Better results so far
Best scores 2021–2025

Zhou et al., Transfusion, ICLR 2025 · Fig. 1

Is continuous simply better?



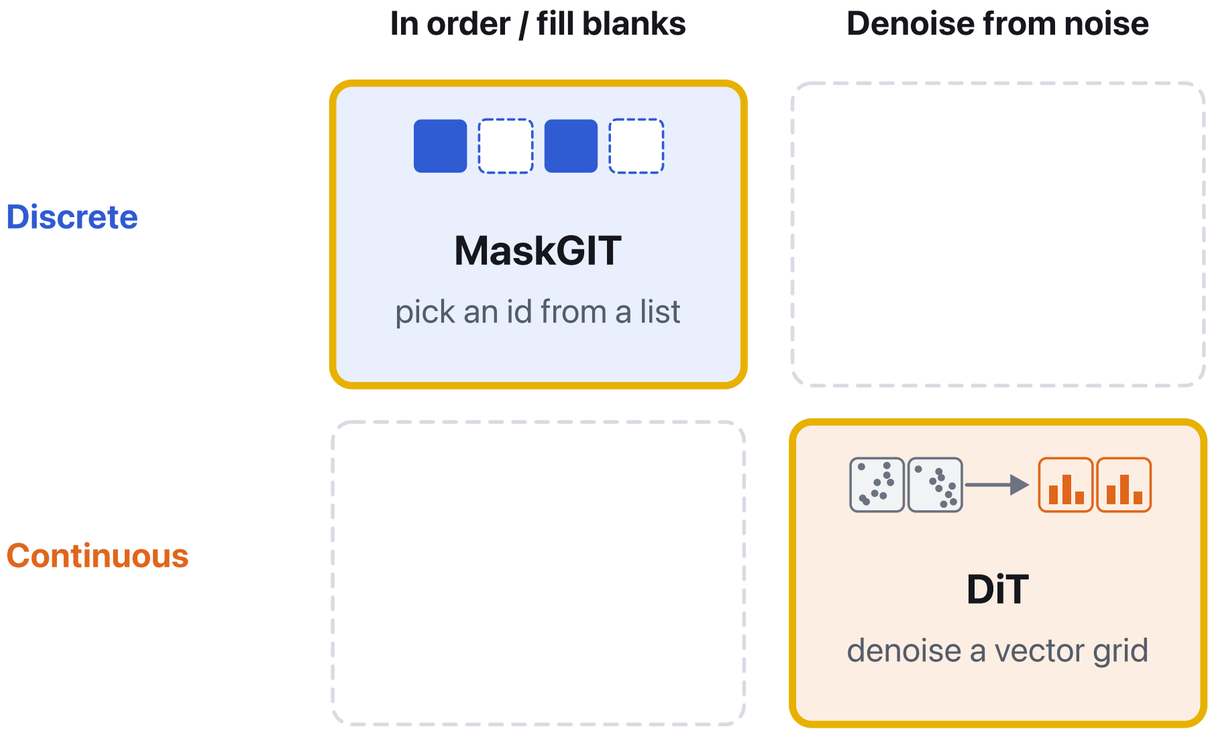
> **rFID** measures reconstruction quality: how well the tokenizer copies a real image. Lower is better.

One paper (MAR), same main network, two tokenizers (and two output heads)
head = the small last part that turns the network's output into an answer

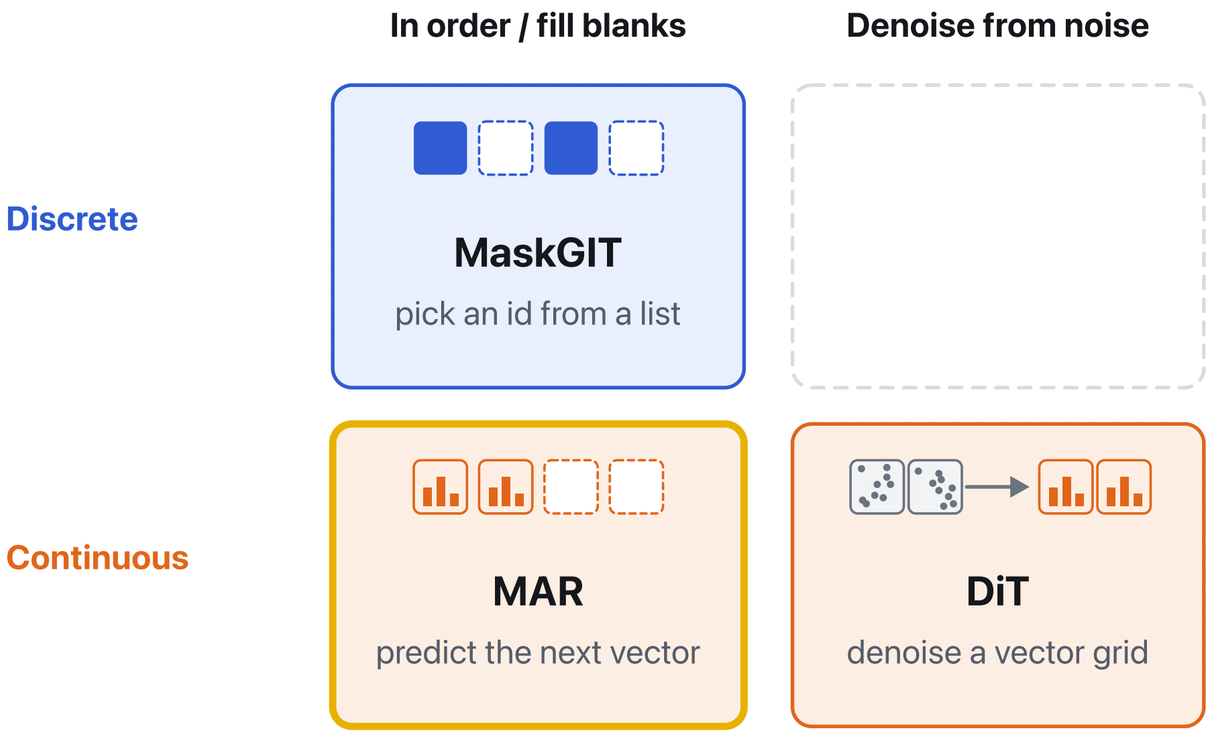
Its discrete tokenizer copies far worse.

Was this a fair comparison?

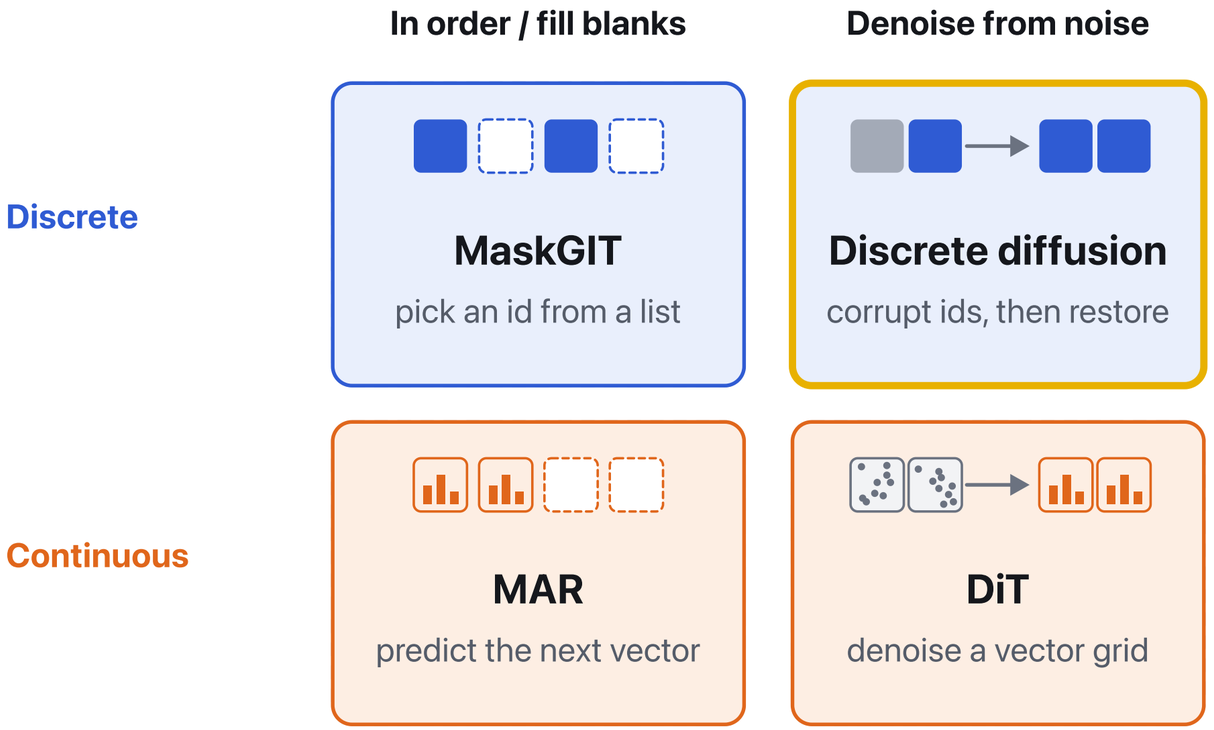
Does the representation decide the generator?



Does the representation decide the generator?



Does the representation decide the generator?



TAKEAWAY

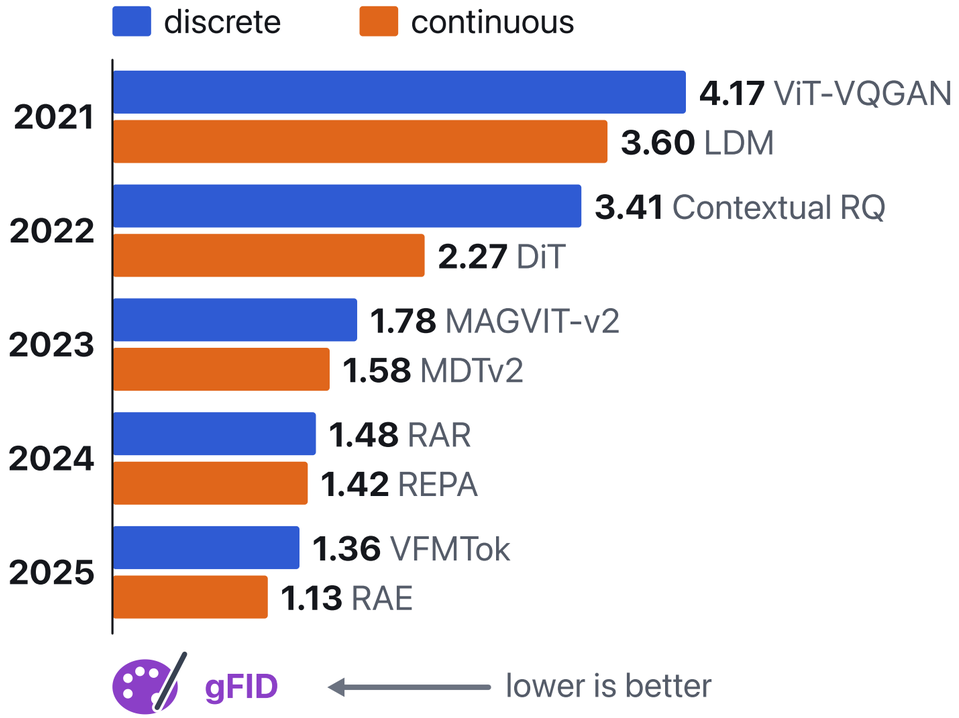
You can mix and match. The diagonal is just the popular choice.

Continuous is ahead, but the gap is shrinking

The best **continuous** model beat the best **discrete** one every year from 2021 to 2025.

gFID measures generation quality: how close brand-new images are to real ones. Lower is better.

ImageNet 256: the standard benchmark, photos of 1,000 kinds of things at 256 × 256 pixels.



Best ImageNet 256 gFID per year · label = that year's best model

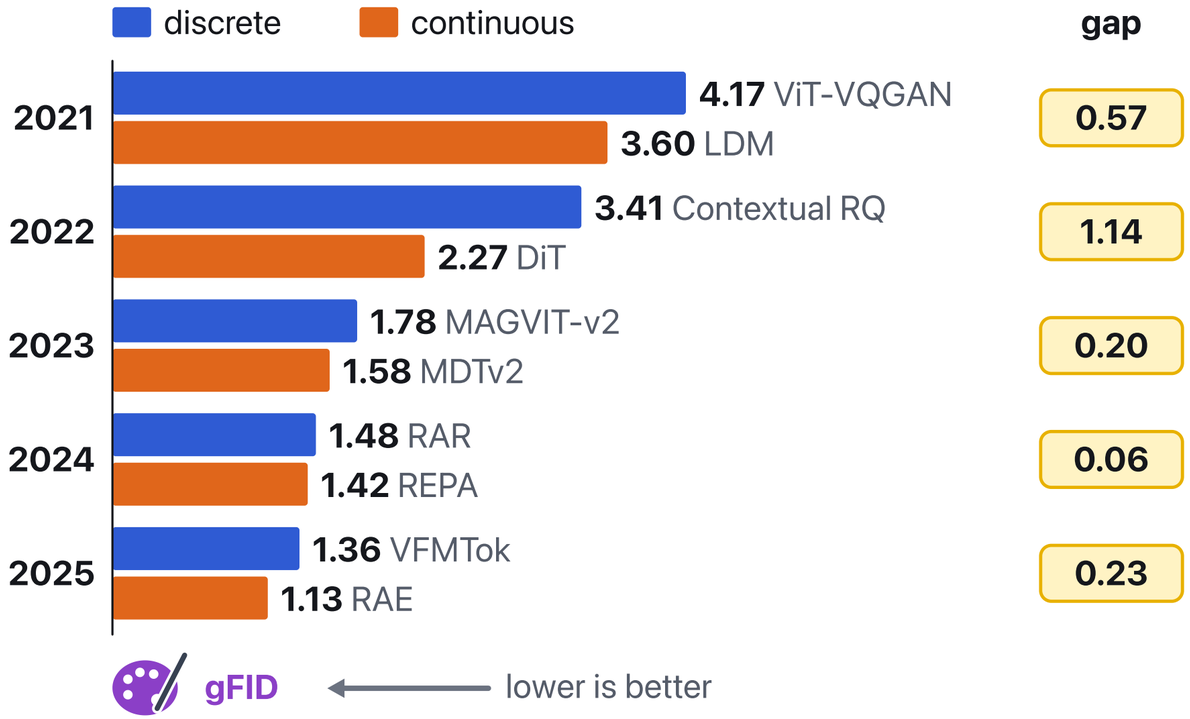
Continuous is ahead, but the gap is shrinking

The best **continuous** model beat the best **discrete** one every year from 2021 to 2025.

gFID measures generation quality: how close brand-new images are to real ones. Lower is better.

ImageNet 256: the standard benchmark, photos of 1,000 kinds of things at 256 × 256 pixels.

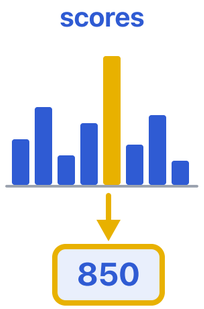
> From over 1 point in 2022 to about 0.2 or less since 2023. Both lines are heading for the same place.



Best ImageNet 256 gFID per year · label = that year's best model

Pros and cons

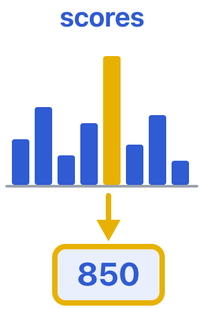
Discrete tokens



+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

Pros and cons

Discrete tokens

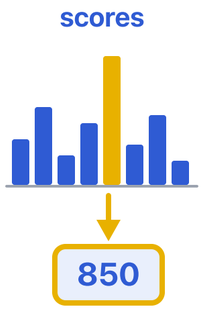


+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

+ The amount of information is capped and countable, and modern quantizers use the codes evenly, so the generator gets a smaller, clear target

Pros and cons

Discrete tokens



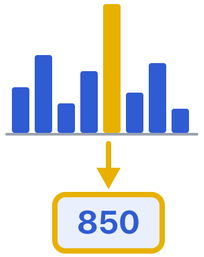
+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

+ The amount of information is capped and countable, and modern quantizers use the codes evenly, so the generator gets a smaller, clear target

– Rounding loses detail and is hard to train (Part 3 fixes this)

Pros and cons

Discrete tokens

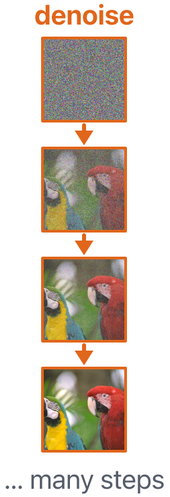


+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

+ The amount of information is capped and countable, and modern quantizers use the codes evenly, so the generator gets a smaller, clear target

– Rounding loses detail and is hard to train (Part 3 fixes this)

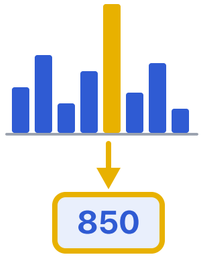
Continuous latents



+ No rounding: fine detail, easy to train

Pros and cons

Discrete tokens

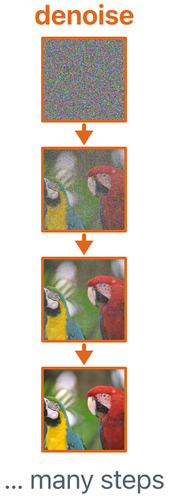


+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

+ The amount of information is capped and countable, and modern quantizers use the codes evenly, so the generator gets a smaller, clear target

– Rounding loses detail and is hard to train (Part 3 fixes this)

Continuous latents

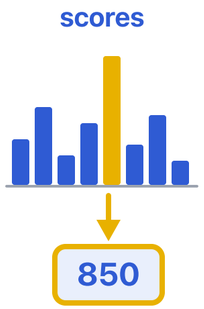


+ No rounding: fine detail, easy to train

+ Best scores every year so far (2021–2025)

Pros and cons

Discrete tokens

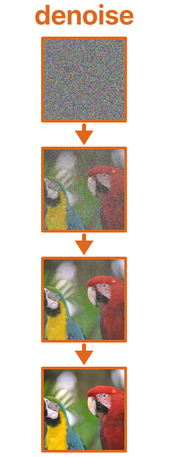


+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

+ The amount of information is capped and countable, and modern quantizers use the codes evenly, so the generator gets a smaller, clear target

– Rounding loses detail and is hard to train (Part 3 fixes this)

Continuous latents



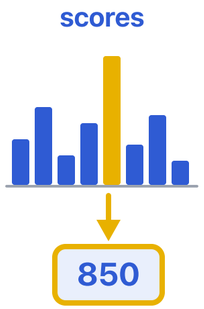
+ No rounding: fine detail, easy to train

+ Best scores every year so far (2021–2025)

– Precision isn't capped, so how much the generator must model isn't under your control; standard samplers run the network tens to hundreds of times per image

Pros and cons

Discrete tokens

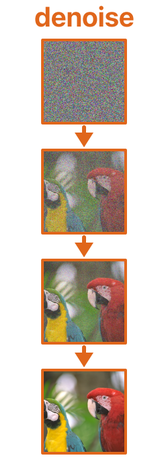


+ Easy for the generator: it only picks from a fixed list (one softmax; LLM tools like exact likelihood and sampling tricks)

+ The amount of information is capped and countable, and modern quantizers use the codes evenly, so the generator gets a smaller, clear target

– Rounding loses detail and is hard to train (Part 3 fixes this)

Continuous latents



+ No rounding: fine detail, easy to train

+ Best scores every year so far (2021–2025)

– Precision isn't capped, so how much the generator must model isn't under your control; standard samplers run the network tens to hundreds of times per image

Is this divide fundamental?

03

The discrete path

How did discrete tokens improve, and what do they share with continuous ones?

1 Images are not text ✓ | 2 Discrete vs. continuous ✓ | **3 The discrete path** | 4 Bits bridge the gap

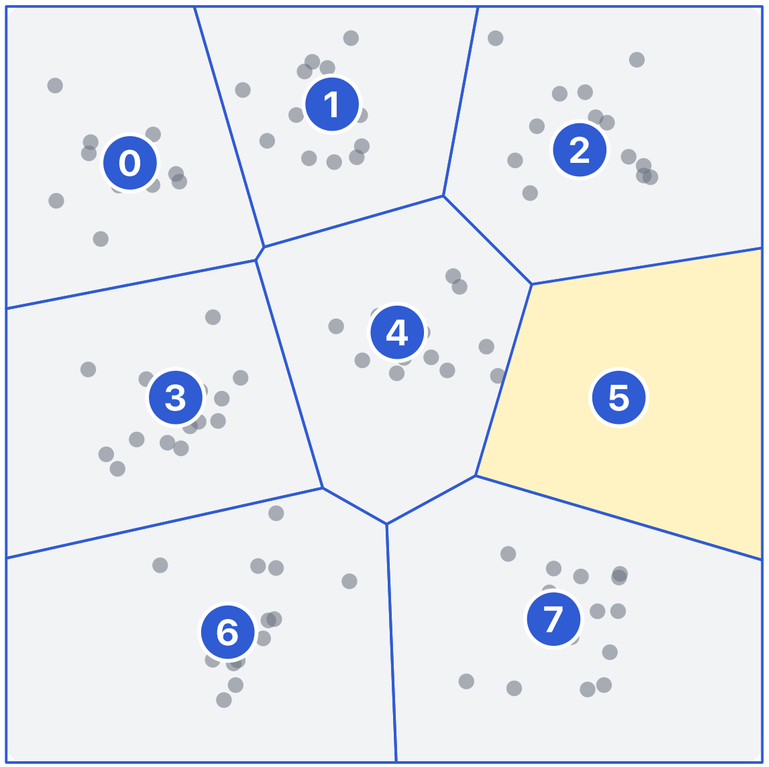
PRIMER

A codebook is a learned list of codes

Teaching illustration: ours (synthetic points) · Chen et al., iGPT, ICML 2020 · Sec. 3.2

PRIMER

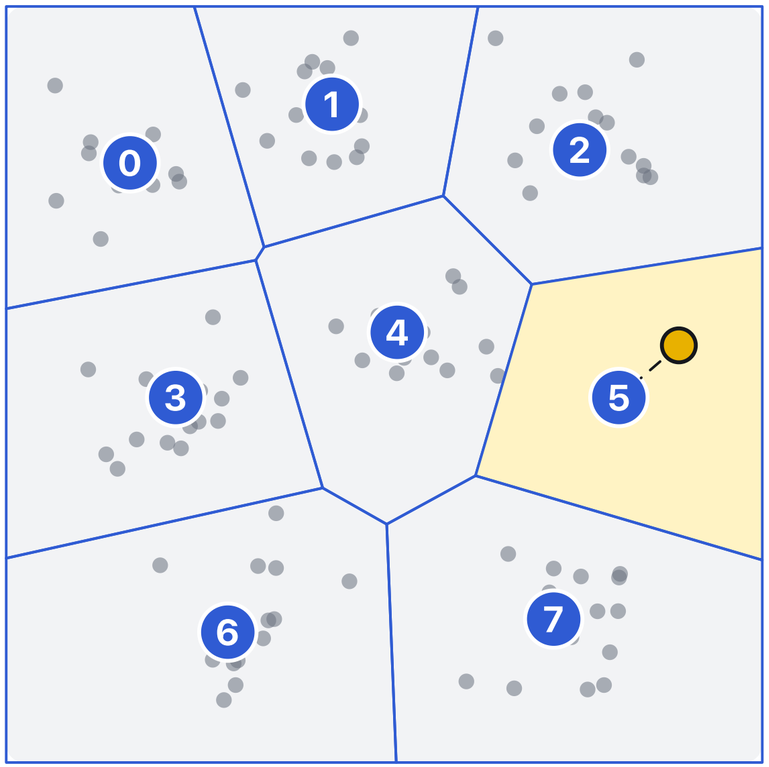
A codebook is a learned list of codes



8 centres = the codebook

Teaching illustration: ours (synthetic points) · Chen et al., iGPT, ICML 2020 · Sec. 3.2

A codebook is a learned list of codes



8 centres = the codebook

one image patch, as a vector

nearest centre

5 token = the number of that code

iGPT already did this for colours: k-means (a standard clustering method) picks a 512-colour codebook (iGPT calls it a palette), and each pixel becomes the number of its nearest colour.

Teaching illustration: ours (synthetic points) · Chen et al., iGPT, ICML 2020 · Sec. 3.2

Can a network learn its own codebook?

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

An autoencoder gives a continuous code. Can it learn a discrete one, a grid of code numbers, by itself?

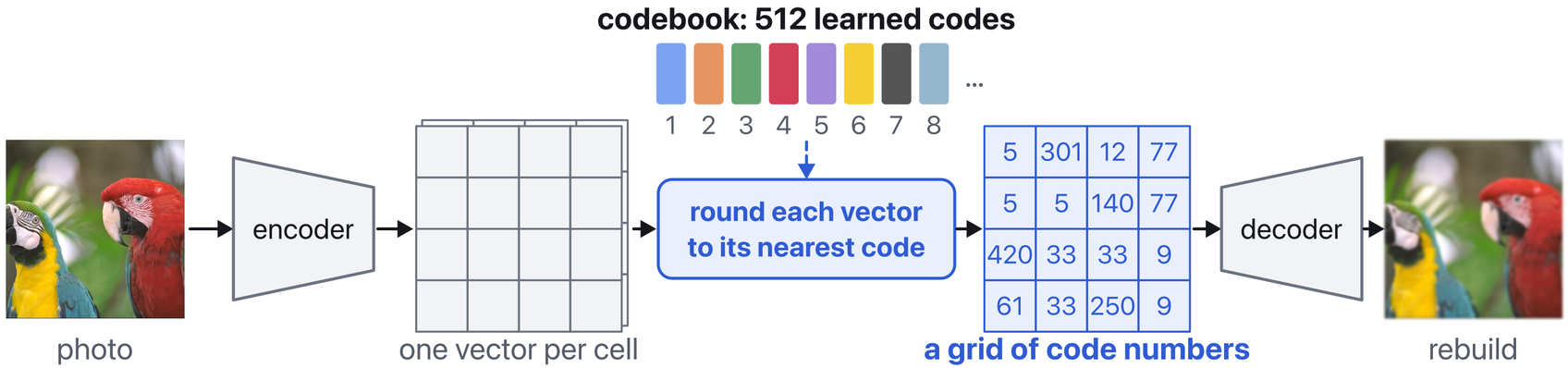
Diagram: ours, after van den Oord et al., VQ-VAE, NeurIPS 2017 · Fig. 1, Sec. 3.1, 4.2 · photo: Kodak kodim23

Can a network learn its own codebook?

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → FSQ 2024 → LFQ (MAGVIT-v2) 2024 → Masked bit modeling

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

An autoencoder gives a continuous code. Can it learn a discrete one, a grid of code numbers, by itself?



In the paper: a 128 × 128 ImageNet photo → a 32 × 32 grid of numbers, each one of **512 codes**

ImageNet = the standard benchmark of photos in 1,000 classes

Diagram: ours, after van den Oord et al., VQ-VAE, NeurIPS 2017 · Fig. 1, Sec. 3.1, 4.2 · photo: Kodak kodim23

How VQ-VAE is trained

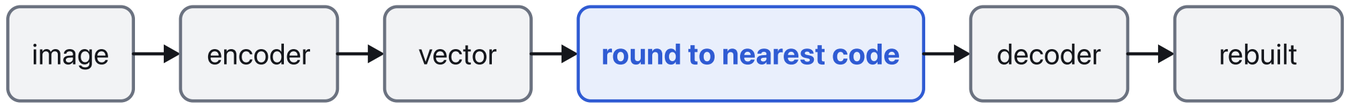
fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

$$\mathcal{L} = \text{reconstruction} + \|\text{sg}[z_e] - e\|_2^2 + \beta \|z_e - \text{sg}[e]\|_2^2$$

z_e = the encoder's vector · e = its nearest code · $\|a - b\|_2^2$ = squared distance · sg = stop-gradient: treat as a constant

1 · Reconstruction: the rebuild should match the input.

forward: quantize (round to the nearest code)



How VQ-VAE is trained

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

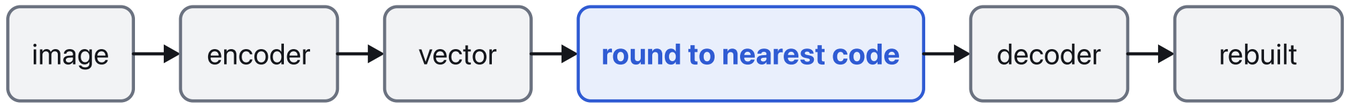
$$\mathcal{L} = \text{reconstruction} + \|\text{sg}[z_e] - e\|_2^2 + \beta \|z_e - \text{sg}[e]\|_2^2$$

z_e = the encoder's vector · e = its nearest code · $\|a - b\|_2^2$ = squared distance · sg = stop-gradient: treat as a constant

1 · Reconstruction: the rebuild should match the input.

2 · Codebook loss: move the codes toward the encoder outputs.

forward: quantize (round to the nearest code)



How VQ-VAE is trained

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

$$\mathcal{L} = \text{reconstruction} + \|\text{sg}[z_e] - e\|_2^2 + \beta \|z_e - \text{sg}[e]\|_2^2$$

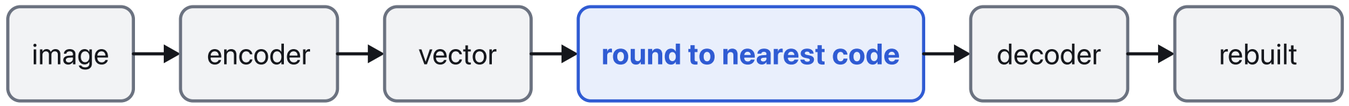
z_e = the encoder's vector · e = its nearest code · $\|a - b\|_2^2$ = squared distance · sg = stop-gradient: treat as a constant

1 · Reconstruction: the rebuild should match the input.

2 · Codebook loss: move the codes toward the encoder outputs.

3 · Commitment loss: keep the encoder outputs close to their codes (β = 0.25).

forward: quantize (round to the nearest code)



How VQ-VAE is trained

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

$$\mathcal{L} = \text{reconstruction} + \|\text{sg}[z_e] - e\|_2^2 + \beta \|z_e - \text{sg}[e]\|_2^2$$

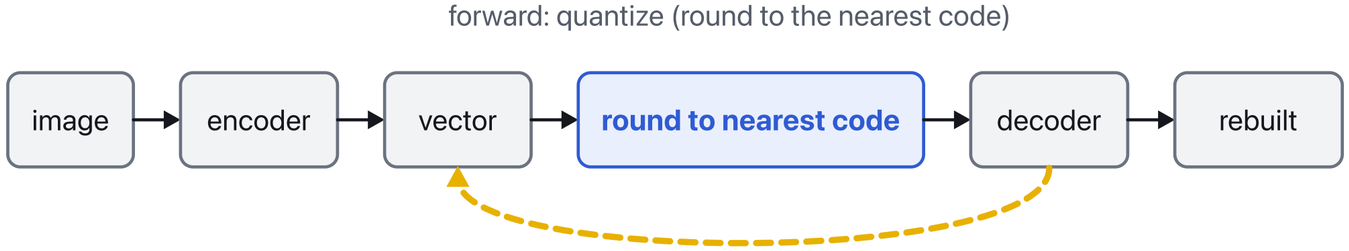
z_e = the encoder's vector · e = its nearest code · $\|a - b\|_2^2$ = squared distance · sg = stop-gradient: treat as a constant

1 · Reconstruction: the rebuild should match the input.

2 · Codebook loss: move the codes toward the encoder outputs.

3 · Commitment loss: keep the encoder outputs close to their codes (β = 0.25).

4 · Straight-through estimator: rounding has no slope, so copy the gradient through the quantization step.



It works, but the rebuilds are soft

VQ-VAE 2017 · VQGAN 2021 · ViT-VQGAN 2022 · FSQ 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

Left: originals · right: rebuilds from 32 × 32 codes. The paper calls them “only slightly blurrier”

Source: VQ-VAE, Fig. 2

TAKE-HOME

A discrete code can be learned end to end. A second model (PixelCNN) then learns which code numbers go together: the two-stage recipe.

It works, but the rebuilds are soft

VQ-VAE 2017 · VQGAN 2021 · ViT-VQGAN 2022 · FSQ 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: a learned discrete code, trained end to end · **still missing:** blurry at high compression

Left: originals · right: rebuilds from 32 × 32 codes. The paper calls them “only slightly blurrier”

Source: VQ-VAE, Fig. 2

TAKE-HOME

A discrete code can be learned end to end. A second model (PixelCNN) then learns which code numbers go together: the two-stage recipe.

WHAT IT CHANGED

Tokenize, then generate. Its codebook and commitment losses are still used. Left open: blurry rebuilds at high compression.

Pixel transformers can't reach high resolution

VQ-VAE (2017) → **VQGAN** (2021) → ViT-VQGAN (2022) → FSQ (2024) → LFQ (MAGVIT-v2) (2024) → Masked bit modeling

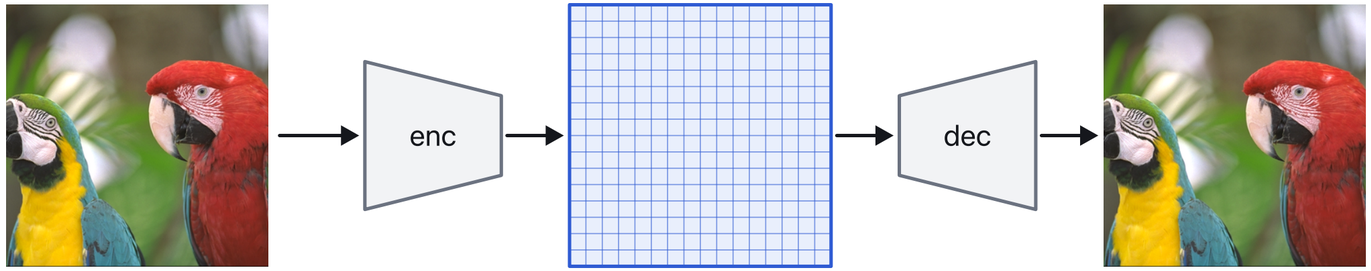
fixed: short, sharp grids: the decoder paints detail · **still missing:** most codes never used

The goal: a 1280 × 460 scene, generated token by token (the paper aims at megapixel images)

Attention cost grows with the square of the length (Part 1), so pixel sequences are far too long. And VQ-VAE tokens blur when the grid is short.

Keep VQ-VAE's quantizer, add a critic

fixed: short, sharp grids: the decoder paints detail · **still missing:** most codes never used

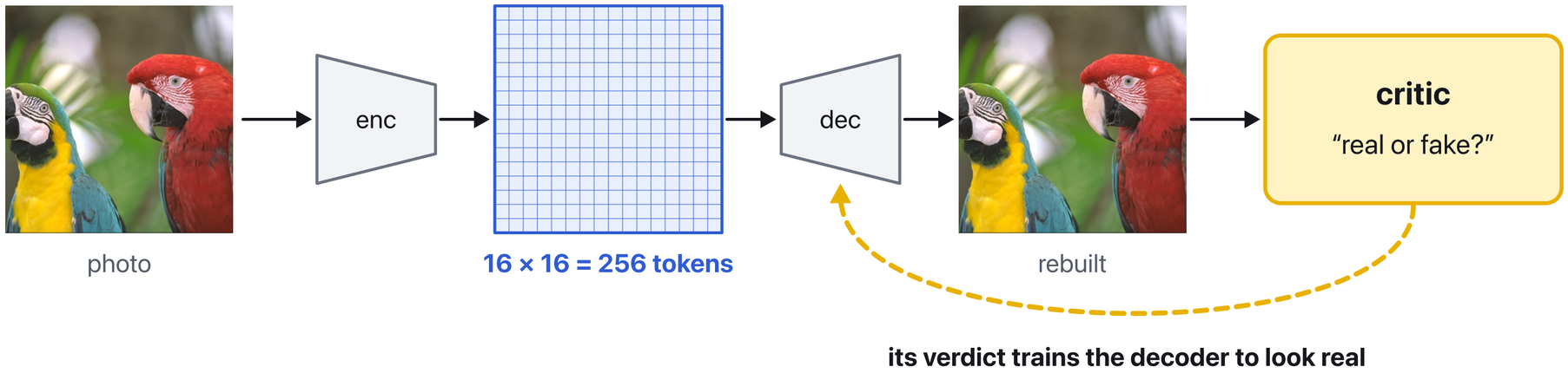


Keeps VQ-VAE's quantizer and losses. The decoder paints the image back.

Teaching illustration: ours, after Esser et al., VQGAN, CVPR 2021 · Sec. 3.1, Fig. 2 · photo: Kodak kodim23

Keep VQ-VAE's quantizer, add a critic

fixed: short, sharp grids: the decoder paints detail · **still missing:** most codes never used



Keeps VQ-VAE's quantizer and losses. The decoder paints the image back.

Adds a perceptual loss and a GAN (adversarial) loss. The critic says “real or fake?”

The decoder paints the texture, the tokens keep the structure

VQ-VAE (2017) → **VQGAN** (2021) → ViT-VQGAN (2022) → FSQ (2024) → LFQ (MAGVIT-v2) (2024) → Masked bit modeling

fixed: short, sharp grids: the decoder paints detail · **still missing:** most codes never used

Without a critic the fur blurs; with it, it stays sharp

TAKE-HOME

GAN loss: the decoder paints the fine texture, so the tokens only need the structure. That gives a short, highly compressed grid a transformer can model (16 × 16 tokens for 256 × 256, 1,024 codes).

Esser et al., VQGAN, CVPR 2021 · Fig. 12 (labels re-set), Sec. 4 (codebook of 1,024 codes)

The decoder paints the texture, the tokens keep the structure

VQ-VAE 2017 · **VQGAN** 2021 · ViT-VQGAN 2022 · FSQ 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: short, sharp grids: the decoder paints detail · **still missing:** most codes never used

Without a critic the fur blurs; with it, it stays sharp

TAKE-HOME

GAN loss: the decoder paints the fine texture, so the tokens only need the structure. That gives a short, highly compressed grid a transformer can model (16 × 16 tokens for 256 × 256, 1,024 codes).

WHAT IT CHANGED

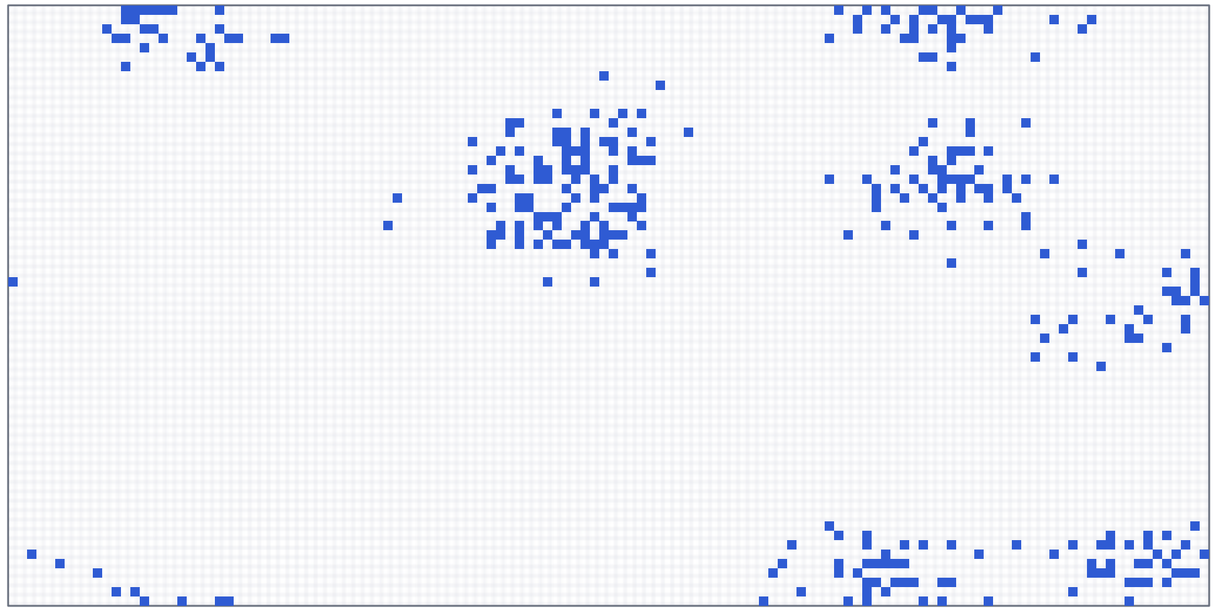
Tokenize-then-generate became practical at high resolution (MaskGIT and Parti use VQGAN-style tokenizers).

Esser et al., VQGAN, CVPR 2021 · Fig. 12 (labels re-set), Sec. 4 (codebook of 1,024 codes)

Most codes go unused

VQ-VAE 2017 → VQGAN 2021 → **ViT-VQGAN** 2022 → FSQ 2024 → LFQ (MAGVIT-v2) 2024 → Masked bit modeling

fixed: tidy lookup space: almost every code used · **still missing:** still a learned codebook, extra losses



Schematic: 8,192 codes; blue = in use

4%

of 8,192 codes in use (each code is a list of 256 numbers)

Like a big box of crayons
where you only ever use a few.

Make the lookup space small and tidy

fixed: tidy lookup space: almost every code used · **still missing:** still a learned codebook, extra losses

codes in use (higher is better)

256 numbers

numbers per vector in the lookup → fewer

Idea: squeeze each vector to a few numbers before the lookup, and scale it to length one (ℓ2-normalised).

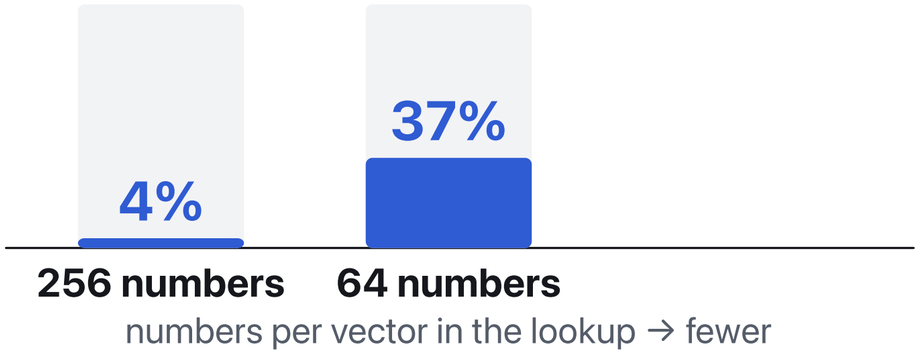
All three bars are ℓ2-normalised. At 32 numbers without normalising: 2%.

Yu et al., ViT-VQGAN, ICLR 2022 · Sec. 3.2, Tab. 4 (8,192 codes; usage over a batch of 256 test images)

Make the lookup space small and tidy

fixed: tidy lookup space: almost every code used · **still missing:** still a learned codebook, extra losses

codes in use (higher is better)



numbers per vector in the lookup → fewer

Idea: squeeze each vector to a few numbers before the lookup, and scale it to length one (ℓ2-normalised).

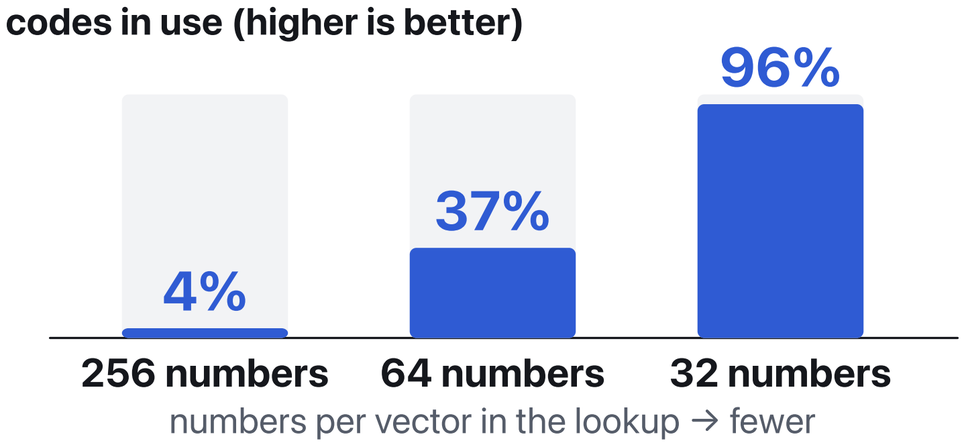
All three bars are ℓ2-normalised. At 32 numbers without normalising: 2%.

Yu et al., ViT-VQGAN, ICLR 2022 · Sec. 3.2, Tab. 4 (8,192 codes; usage over a batch of 256 test images)

Make the lookup space small and tidy

VQ-VAE 2017 · VQGAN 2021 · **ViT-VQGAN 2022** · FSQ 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: tidy lookup space: almost every code used · **still missing:** still a learned codebook, extra losses



Idea: squeeze each vector to a few numbers before the lookup, and scale it to length one (ℓ2-normalised).

All three bars are ℓ2-normalised. At 32 numbers without normalising: 2%.

32 numbers, normalised: 96% of the 8,192 codes in use

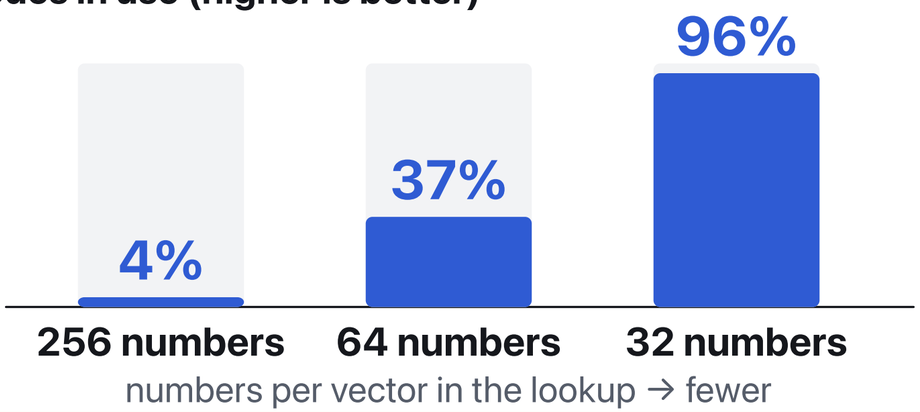
Yu et al., ViT-VQGAN, ICLR 2022 · Sec. 3.2, Tab. 4 (8,192 codes; usage over a batch of 256 test images)

Make the lookup space small and tidy

VQ-VAE 2017 · VQGAN 2021 · **ViT-VQGAN 2022** · FSQ 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: tidy lookup space: almost every code used · **still missing:** still a learned codebook, extra losses

codes in use (higher is better)



numbers per vector in the lookup → fewer

Idea: squeeze each vector to a few numbers before the lookup, and scale it to length one (ℓ2-normalised).

All three bars are ℓ2-normalised. At 32 numbers without normalising: 2%.

32 numbers, normalised: 96% of the 8,192 codes in use

Bonus: on a sphere the rounding error stays bounded, so the straight-through gradient is a better approximation.

TAKE-HOME

Regularising (constraining) the latent space leads to better codebook usage. Squeezed small and scaled to length one, encoder outputs and codes actually meet.

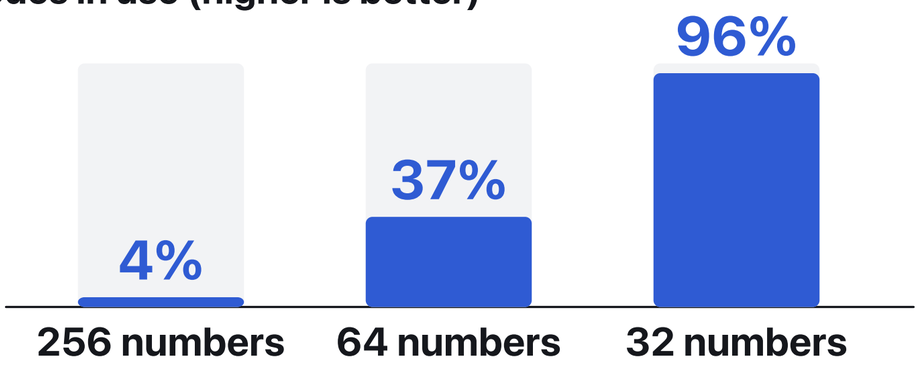
Yu et al., ViT-VQGAN, ICLR 2022 · Sec. 3.2, Tab. 4 (8,192 codes; usage over a batch of 256 test images)

Make the lookup space small and tidy

VQ-VAE 2017 · VQGAN 2021 · **ViT-VQGAN 2022** · FSQ 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: tidy lookup space: almost every code used · **still missing:** still a learned codebook, extra losses

codes in use (higher is better)



numbers per vector in the lookup → fewer

Idea: squeeze each vector to a few numbers before the lookup, and scale it to length one (ℓ2-normalised).

All three bars are ℓ2-normalised. At 32 numbers without normalising: 2%.

32 numbers, normalised: 96% of the 8,192 codes in use

Bonus: on a sphere the rounding error stays bounded, so the straight-through gradient is a better approximation.

TAKE-HOME

Regularising (constraining) the latent space leads to better codebook usage. Squeezed small and scaled to length one, encoder outputs and codes actually meet.

WHAT IT CHANGED

Codebook usage became a number people report (my reading).

Yu et al., ViT-VQGAN, ICLR 2022 · Sec. 3.2, Tab. 4 (8,192 codes; usage over a batch of 256 test images)

Training a good codebook is tricky

fixed: no codebook to learn: bound and round · **still missing:** small vocabulary; does bigger help?

EXTRA LOSSES

A learned codebook needs extra losses: the codebook loss and the commitment loss.

Training a good codebook is tricky

VQ-VAE 2017 · VQGAN 2021 · ViT-VQGAN 2022 · **FSQ** 2024 · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: no codebook to learn: bound and round · **still missing:** small vocabulary; does bigger help?

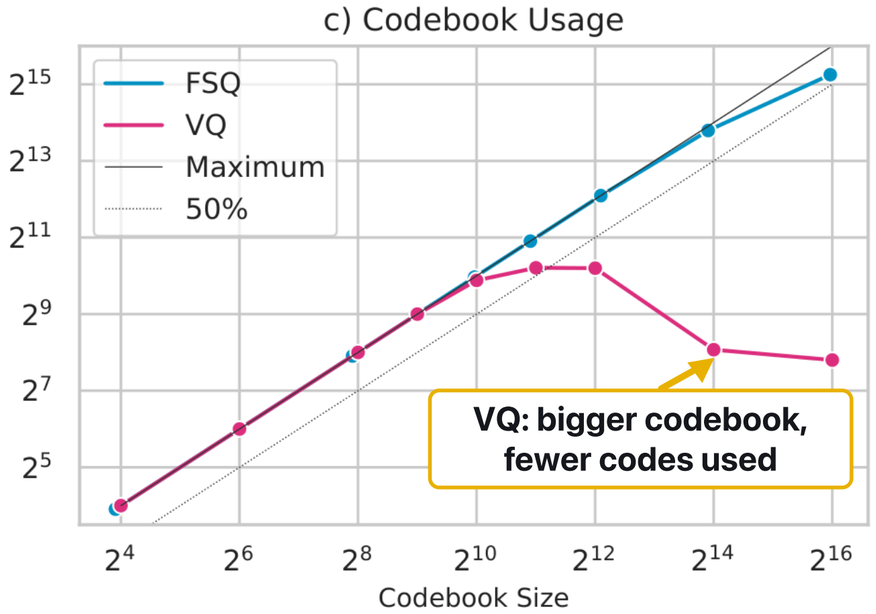
EXTRA LOSSES

A learned codebook needs extra losses: the codebook loss and the commitment loss.

A BAG OF TRICKS

FSQ's own list: commitment losses, codebook reseeding, code splitting, entropy penalties; EMA updates on the codebook, projections. And it can still collapse.

You don't need to know these terms. The point is how long the list is.



Codes actually used, as the codebook grows

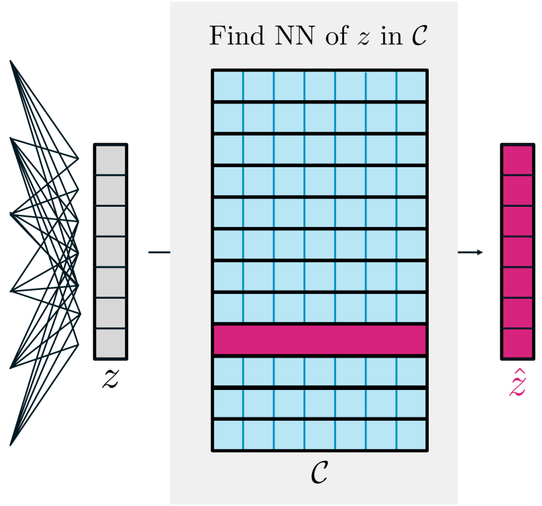
Source: FSQ, Fig. 3c (ImageNet 128)

Mentzer et al., Finite Scalar Quantization: VQ-VAE Made Simple (FSQ), ICLR 2024 · Abstract, Sec. 1, Fig. 2 (table), Fig. 3c

What if there is no codebook to learn?

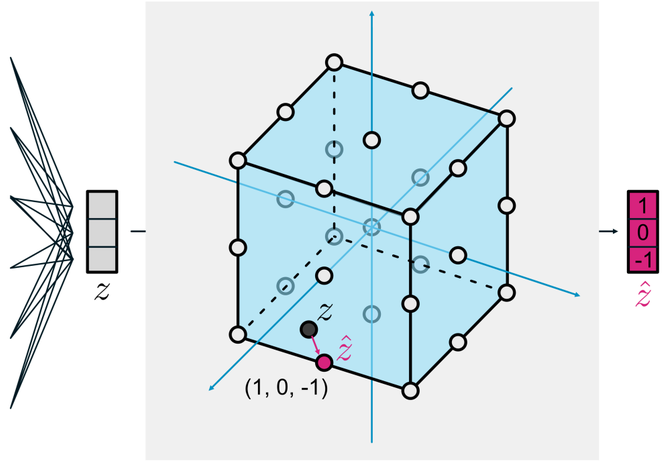
VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → **FSQ** 2024 → LFQ (MAGVIT-v2) 2024 → Masked bit modeling

fixed: no codebook to learn: bound and round · **still missing:** small vocabulary; does bigger help?



VQ: nearest code in a learned table

Source: FSQ, Fig. 1



FSQ: round each number to the nearest level (drawn: 3 numbers, 3 levels each = 27 codes)

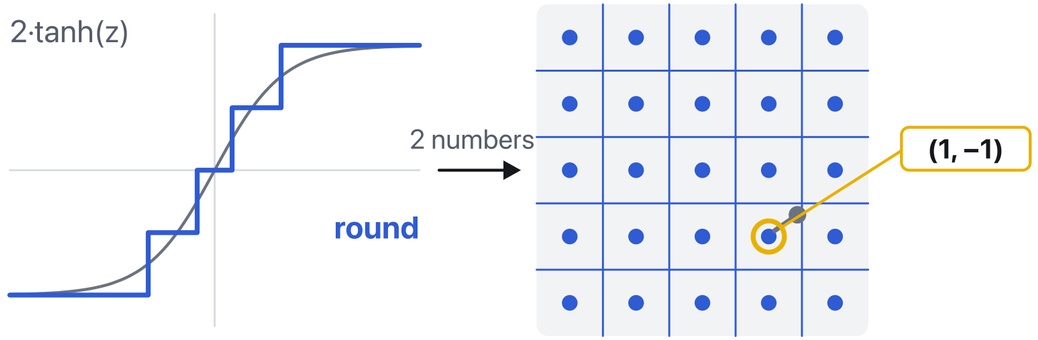
Source: FSQ, Fig. 1

A **pre-defined** codebook instead of a learned one: 4 numbers with 8 × 5 × 5 × 5 levels = **1,000 codes**, none stored

Bound with tanh, round to the nearest level

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → **FSQ** 2024 → LFQ (MAGVIT-v2) 2024 → Masked bit modeling

fixed: no codebook to learn: bound and round · **still missing:** small vocabulary; does bigger help?



one channel: 5 levels {−2,...,2}

25 codes, none stored

2 numbers, 5 levels each = 25 codes

Teaching illustration: ours, after FSQ, Figs. 1–2

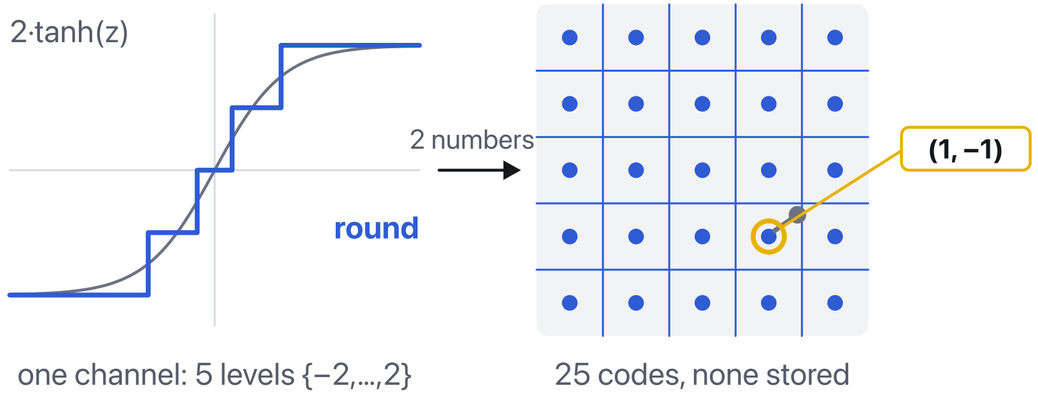
$$\hat{z} = \mathrm{round}\big(\lfloor L/2 \rfloor \cdot \tanh(z)\big)$$

tanh bounds z, one number from the encoder; then round it to one of L levels (here 5, so ⌊L/2⌋ = 2).

Bound with tanh, round to the nearest level

VQ-VAE 2017 · VQGAN 2021 · ViT-VQGAN 2022 · **FSQ 2024** · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: no codebook to learn: bound and round · **still missing:** small vocabulary; does bigger help?



one channel: 5 levels {−2,...,2} 25 codes, none stored

2 numbers, 5 levels each = 25 codes

Teaching illustration: ours, after FSQ, Figs. 1–2

$$\hat{z} = \mathrm{round}\big(\lfloor L/2 \rfloor \cdot \tanh(z)\big)$$

tanh bounds z, one number from the encoder; then round it to one of L levels (here 5, so ⌊L/2⌋ = 2).

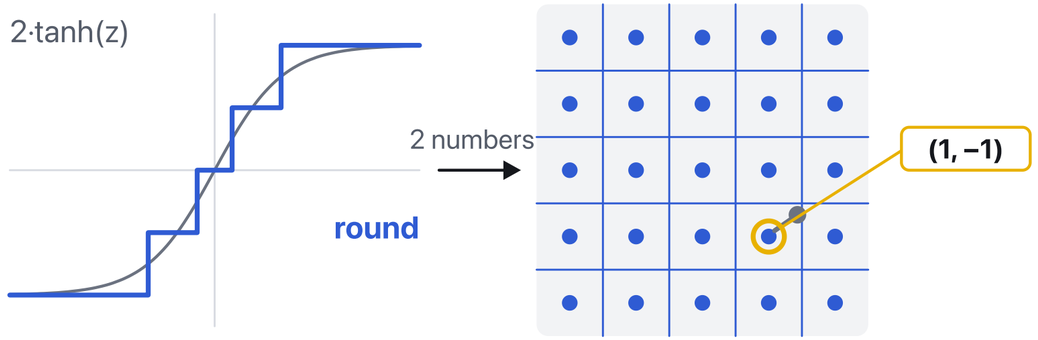
TAKE-HOME

Pre-define the codebook instead of learning it. Nothing to train, nearly every code gets used, and the codebook can grow without getting harder to learn.

Bound with tanh, round to the nearest level

VQ-VAE 2017 · VQGAN 2021 · ViT-VQGAN 2022 · **FSQ 2024** · LFQ (MAGVIT-v2) 2024 · Masked bit modeling

fixed: no codebook to learn: bound and round · **still missing:** small vocabulary; does bigger help?



one channel: 5 levels {−2,...,2} 25 codes, none stored

2 numbers, 5 levels each = 25 codes

Teaching illustration: ours, after FSQ, Figs. 1–2

$$\hat{z} = \mathrm{round}\big(\lfloor L/2 \rfloor \cdot \tanh(z)\big)$$

tanh bounds z, one number from the encoder; then round it to one of L levels (here 5, so ⌊L/2⌋ = 2).

TAKE-HOME

Pre-define the codebook instead of learning it. Nothing to train, nearly every code gets used, and the codebook can grow without getting harder to learn.

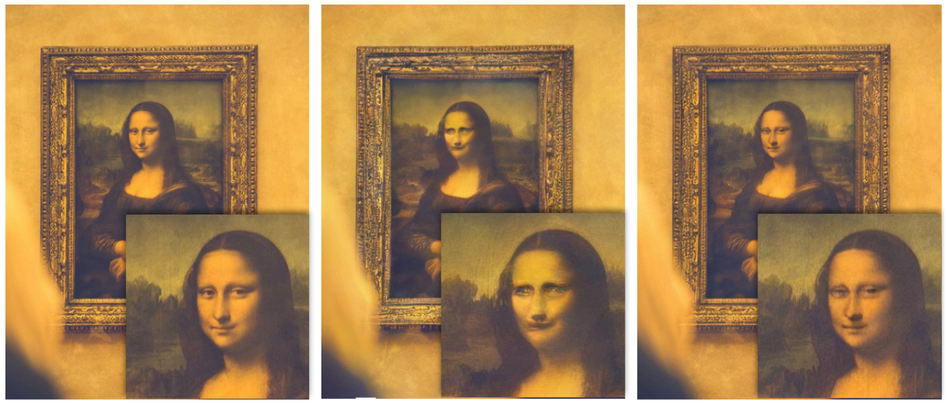
WHAT IT CHANGED

A simple drop-in replacement for VQ that later tokenizers adopted (e.g. the discrete tokenizer of NVIDIA Cosmos).

Why did token generators lag diffusion?

VQ-VAE 2017 · VQGAN 2021 · ViT-VQGAN 2022 · FSQ 2024 · **LFQ (MAGVIT-v2) 2024** · Masked bit modeling

fixed: sign per channel: vocabulary grows cheaply · **still missing:** a softmax over codes can't grow further



original VQGAN copy MAGVIT-v2 copy

The old tokenizer distorts the face; the new one keeps it

Source: MAGVIT-v2, Fig. 3 (top row, first three panels; labels re-set)

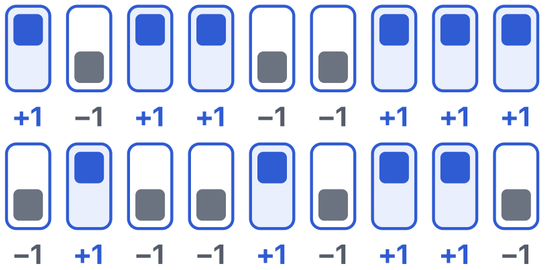
Their claim: the tokenizer, not the generator, held generation back.

Keep only the sign: a token becomes a string of bits

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → FSQ 2024 → **LFQ (MAGVIT-v2)** 2024 → Masked bit modeling

fixed: sign per channel: vocabulary grows cheaply · **still missing:** a softmax over codes can't grow further

$$q(z_i) = \text{sign}(z_i)$$



18 switches, each +1 or −1

→ 2^{18} = 262,144 codes, no table

$$\text{token id} = \sum_i 2^{i-1}\,\mathbb{1}[z_i > 0]$$

Each channel rounds to ±1, one bit; the id is the bits read as a binary number: a **bit token**. Bits per token = channels.

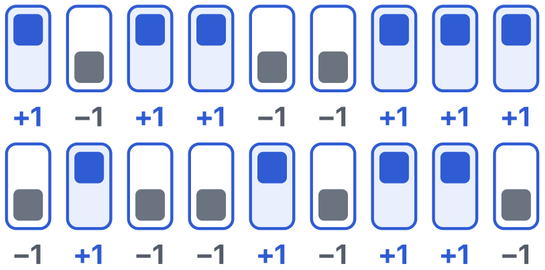
Yu et al., MAGVIT-v2, ICLR 2024 · Sec. 3.1, Fig. 1 · 2^{18} codes: Sec. 4.1

Keep only the sign: a token becomes a string of bits

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → FSQ 2024 → **LFQ (MAGVIT-v2) 2024** → Masked bit modeling

fixed: sign per channel: vocabulary grows cheaply · **still missing:** a softmax over codes can't grow further

$$q(z_i) = \text{sign}(z_i)$$

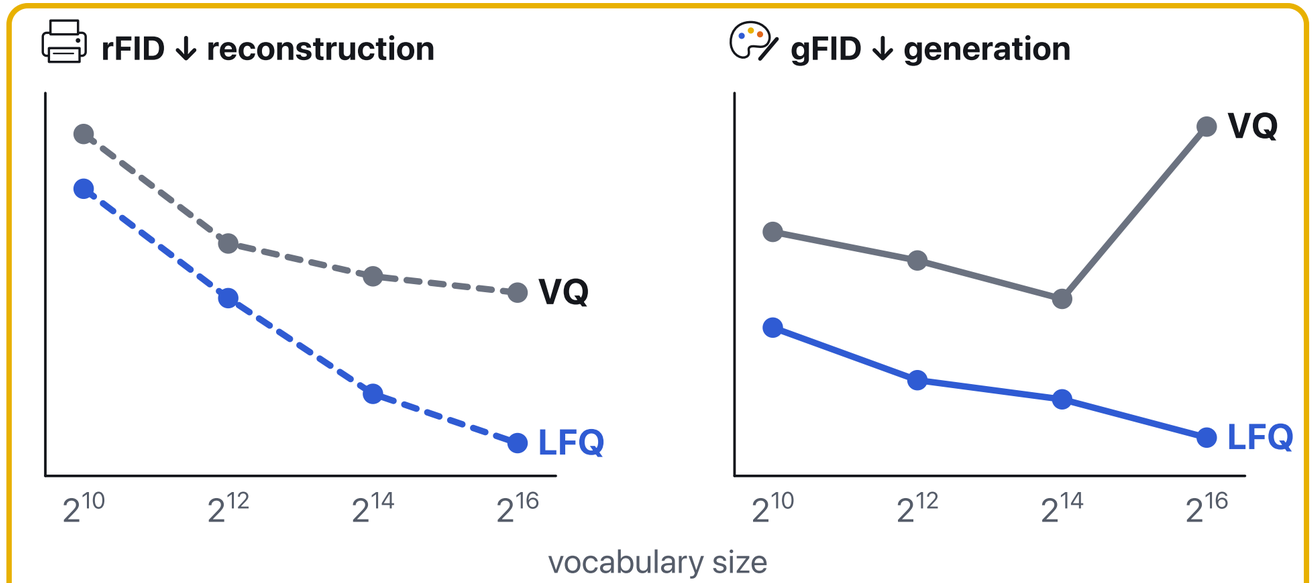


18 switches, each +1 or −1

→ 2^{18} = 262,144 codes, no table

$$\text{token id} = \sum_i 2^{i-1}\,1[z_i > 0]$$

Each channel rounds to ±1, one bit; the id is the bits read as a binary number: a **bit token**. Bits per token = channels.



Bigger vocabulary: VQ copies a bit better but generates worse; LFQ improves both

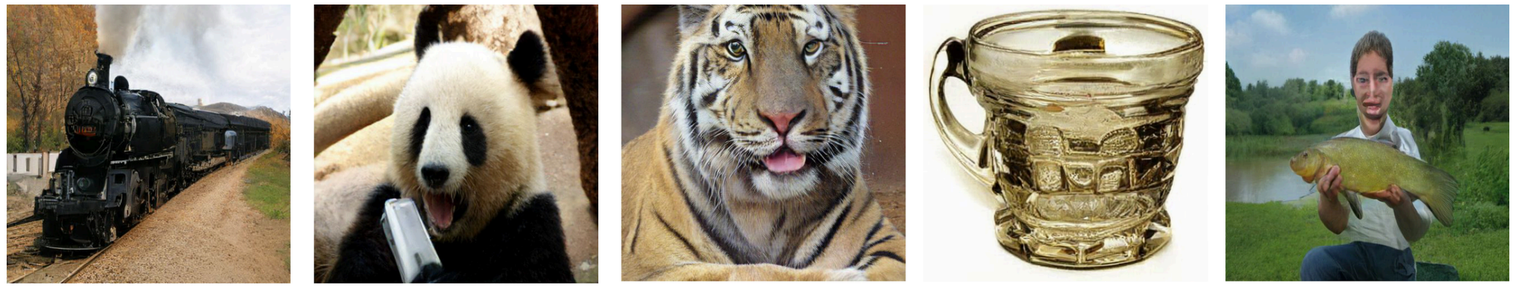
Teaching illustration: ours, after MAGVIT-v2, Fig. 1 (ImageNet 128); shapes only

Yu et al., MAGVIT-v2, ICLR 2024 · Sec. 3.1, Fig. 1 · 2^{18} codes: Sec. 4.1

Fix the tokenizer, and tokens beat diffusion (in their table)

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → FSQ 2024 → **LFQ (MAGVIT-v2)** 2024 → Masked bit modeling

fixed: sign per channel: vocabulary grows cheaply · **still missing:** a softmax over codes can't grow further



New images from a masked token model on LFQ tokens (ImageNet 512 × 512)

Source: MAGVIT-v2, Fig. 5 (bottom row)

1.78

gFID ↓ on ImageNet 256, lower is better
vs. 2.12 for the best diffusion model
in the paper's table without extra data (VDM++)

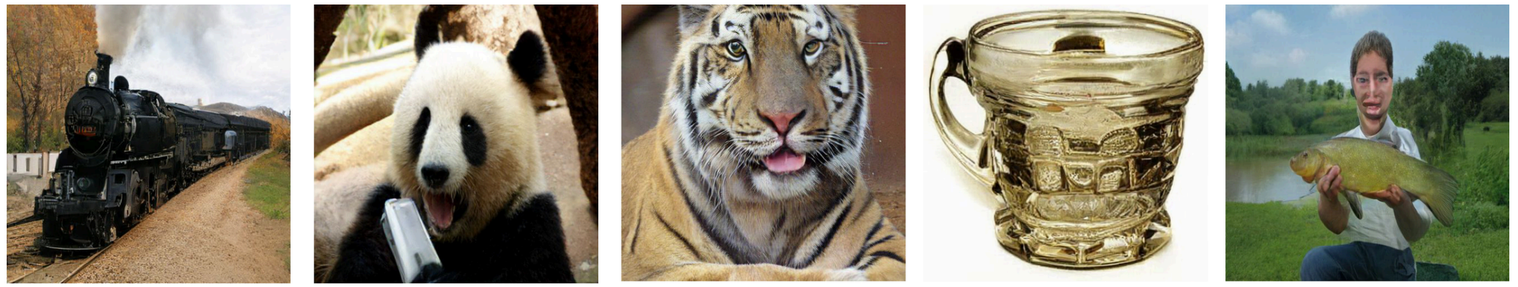
TAKE-HOME

Fix the tokenizer, and a token model can compete with diffusion.

Fix the tokenizer, and tokens beat diffusion (in their table)

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → FSQ 2024 → **LFQ (MAGVIT-v2) 2024** → Masked bit modeling

fixed: sign per channel: vocabulary grows cheaply · **still missing:** a softmax over codes can't grow further



New images from a masked token model on LFQ tokens (ImageNet 512 × 512)

Source: MAGVIT-v2, Fig. 5 (bottom row)

1.78

gFID ↓ on ImageNet 256, lower is better
vs. 2.12 for the best diffusion model
in the paper's table without extra data (VDM++)

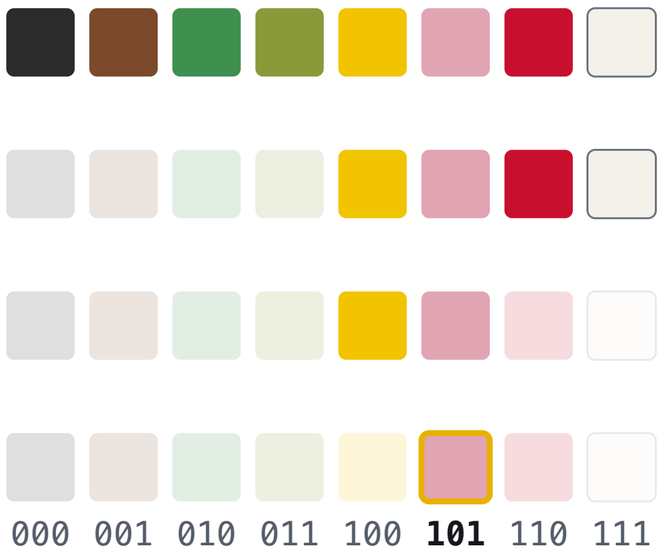
TAKE-HOME

Fix the tokenizer, and a token model can compete with diffusion.

WHAT IT CHANGED

The tokenizer became the thing to fix, and big vocabularies the goal.

A bit answers one yes/no question



8 options

Right half? **yes → 1**

Right half again? **no → 0**

Right one? **yes → 1**

2 options → **1 bit**

8 options → **3 bits**

1,024 options → **10 bits**

65,536 options → **16 bits**

$$\text{bits} = \log_2 K$$

Choosing 1 of K options takes $\log_2$ K yes/no answers.

LFQ's sign is exactly one yes/no answer per number.

Teaching illustration: ours

Five quantizers, one unit: bits per token

Different machinery, same unit: a codebook of K codes costs $\log_2 K$ bits per token. Along the whole path the budget barely moved.

$$\text{bits per image} = \text{tokens} \times \text{bits per token}$$

Codebook sizes: VQ-VAE Sec. 4.2; VQGAN Sec. 4; ViT-VQGAN Sec. 3.2; FSQ Tab. 1; MAGVIT-v2 Sec. 4.1 · Arithmetic: ours

PART 3 · UNIFIED VIEW

Five quantizers, one unit: bits per token

Different machinery, same unit: a codebook of K codes costs $\log_2 K$ bits per token. Along the whole path the budget barely moved.

$$\text{bits per image} = \text{tokens} \times \text{bits per token}$$

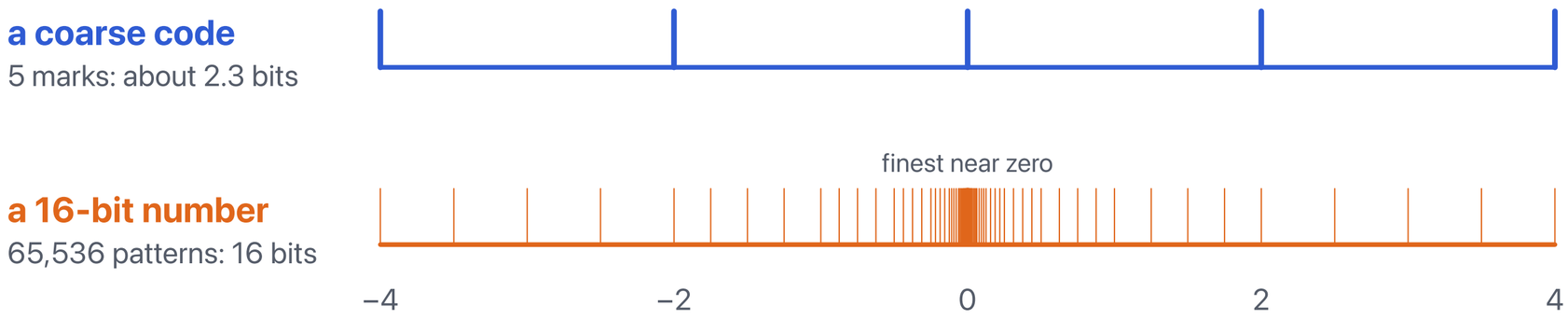
1,572,864 bits
the raw parrot: 196,608 numbers × 8 bits

2,560 bits
256 tokens × 10 bits

Codebook sizes: VQ-VAE Sec. 4.2; VQGAN Sec. 4; ViT-VQGAN Sec. 3.2; FSQ Tab. 1; MAGVIT-v2 Sec. 4.1 · Arithmetic: ours

A “continuous” number is also a code

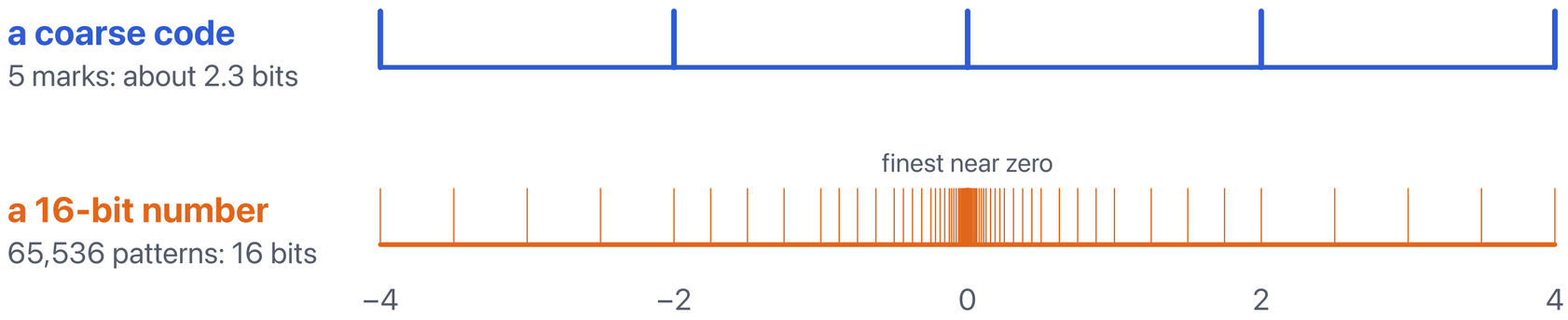
Generators usually read latents in bf16 (training often runs in fp32): 16 bits, one of 65,536 patterns.



Arithmetic: ours (ruler = bf16, every 32nd value; 5 marks ≈ 2.3 bits; bell-curve entropy: bf16 ≈ 10.5, fp16 ≈ 13.5 bits)

A “continuous” number is also a code

Generators usually read latents in bf16 (training often runs in fp32): 16 bits, one of 65,536 patterns.



Same axis for both: how many bits are really used, how they are shaped, and whether the generator can use them.

bits
how much info?

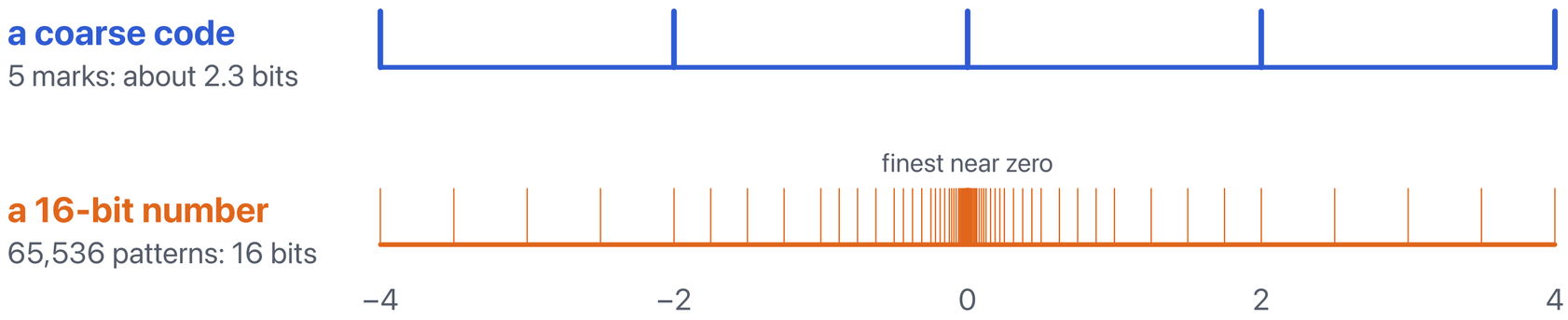
shape
how is it arranged?

generator
can the model use it?

Arithmetic: ours (ruler = bf16, every 32nd value; 5 marks ≈ 2.3 bits; bell-curve entropy: bf16 ≈ 10.5, fp16 ≈ 13.5 bits)

A “continuous” number is also a code

Generators usually read latents in bf16 (training often runs in fp32): 16 bits, one of 65,536 patterns.



Same axis for both: how many bits are really used, how they are shaped, and whether the generator can use them.

bits
how much info?

shape
how is it arranged?

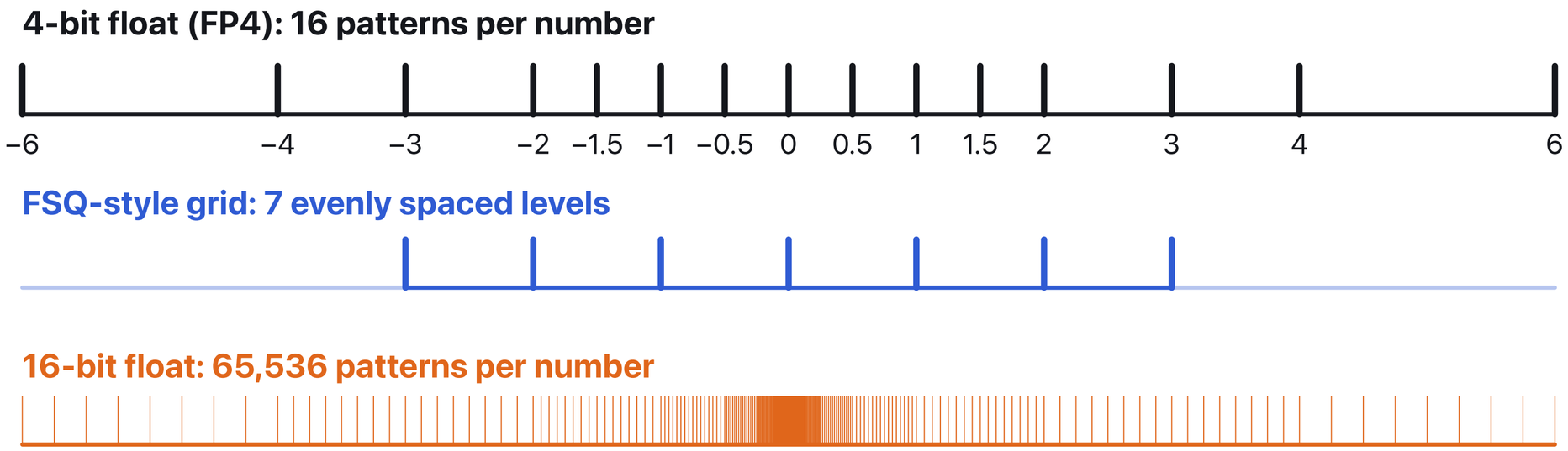
generator
can the model use it?

Information used = the average number of yes/no questions you need, given how the values really spread (entropy).

A number that follows a bell curve, stored in a 16-bit float (bf16), carries about 10.5 bits, not 16. A noisy VAE channel: far fewer (unmeasured).

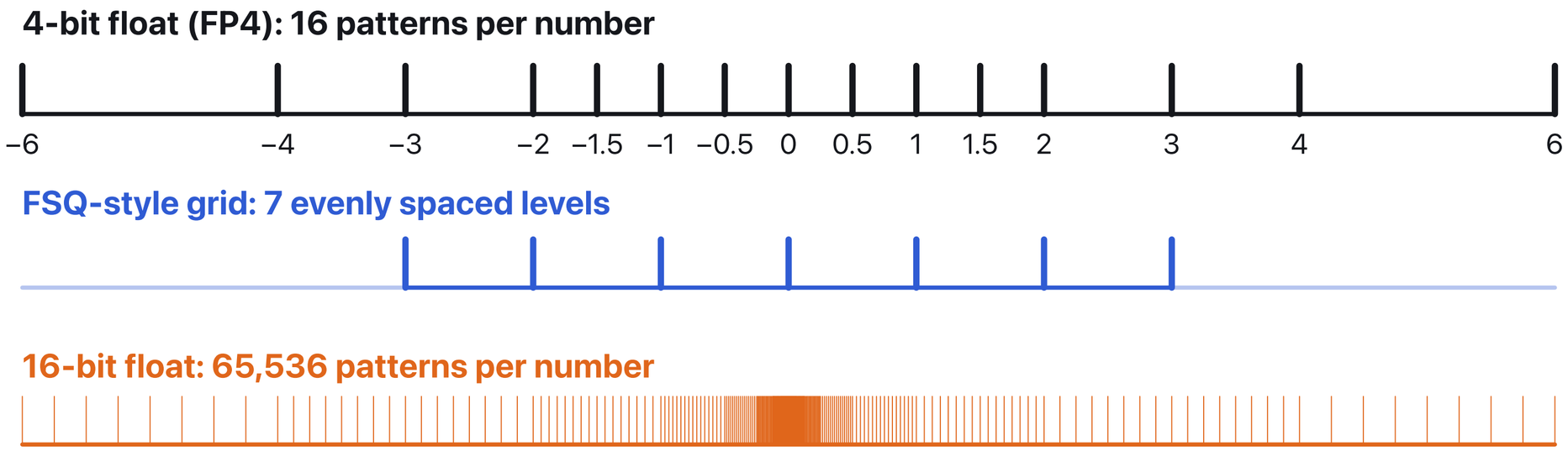
Arithmetic: ours (ruler = bf16, every 32nd value; 5 marks ≈ 2.3 bits; bell-curve entropy: bf16 ≈ 10.5, fp16 ≈ 13.5 bits)

What if we train a VAE in FP4?



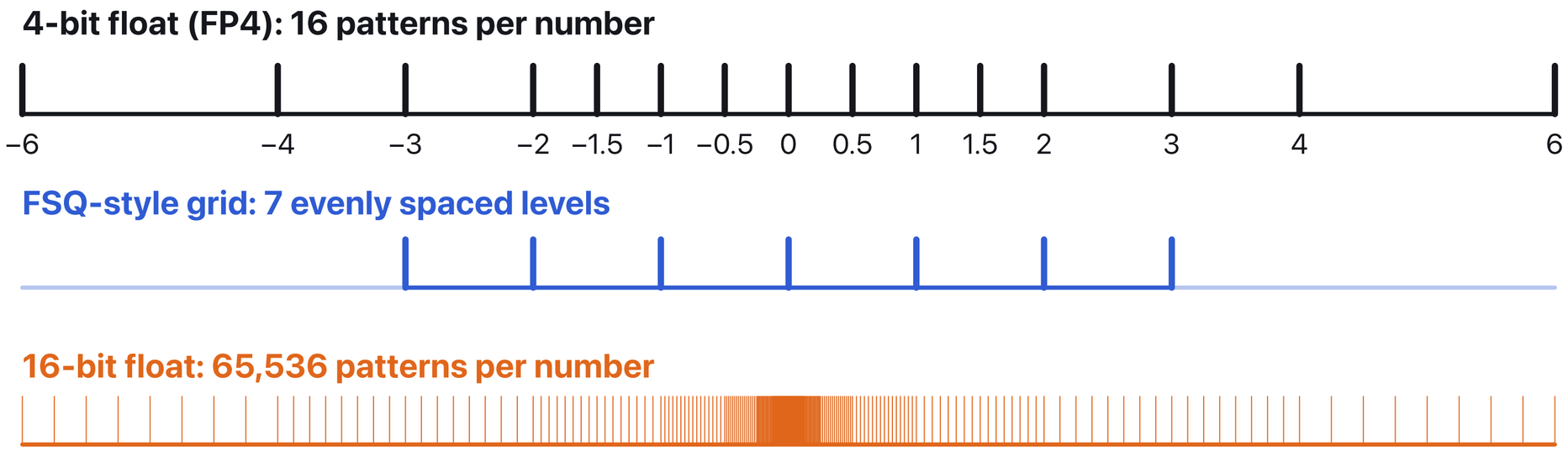
Is this VAE continuous or discrete? **Your guess?**

What if we train a VAE in FP4?



4 channels → $16^4 = 65{,}536 = 2^{16}$ codes per position. It **is** a discrete tokenizer, 16 bits per token: an FSQ-like quantizer on an uneven grid, trained with the same round + straight-through trick.

What if we train a VAE in FP4?



4 channels → $16^4 = 65{,}536 = 2^{16}$ codes per position. It **is** a discrete tokenizer, 16 bits per token: an FSQ-like quantizer on an uneven grid, trained with the same round + straight-through trick.

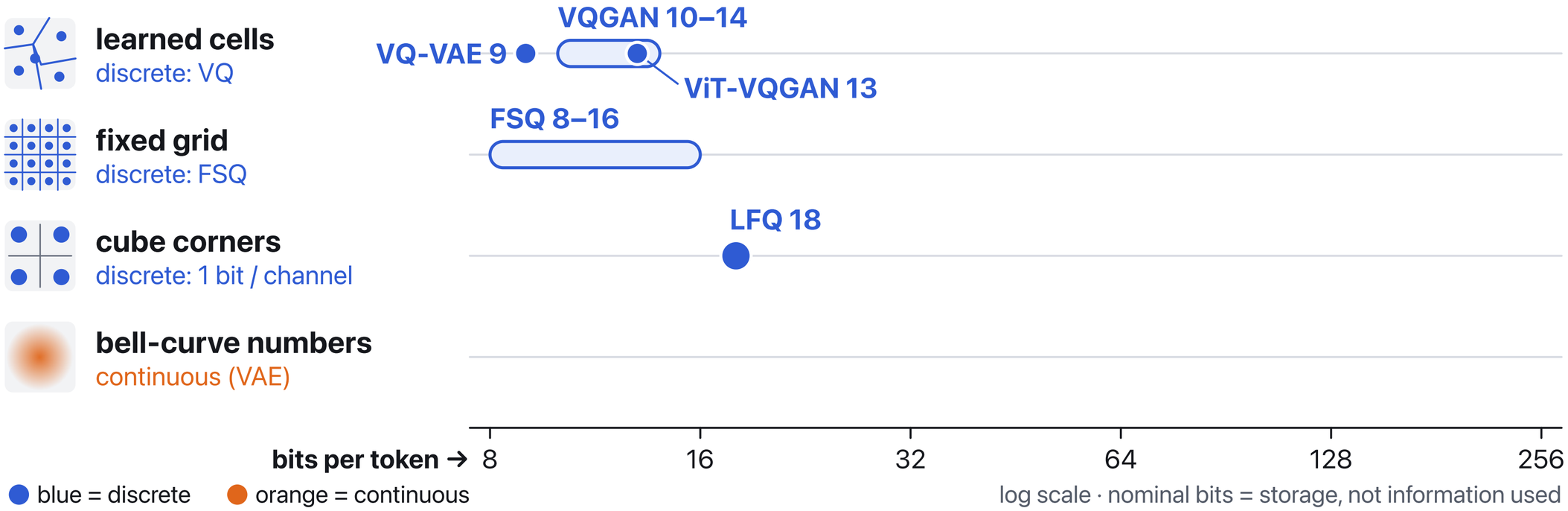
TAKEAWAY

So precision decides how "continuous" a latent really is.

Thought experiment: ours · FP4 = E2M1: OCP Microscaling Formats (MX) Specification v1.0, 2023 · Sec. 5.3.3, Tab. 5

PART 3 · UNIFIED VIEW

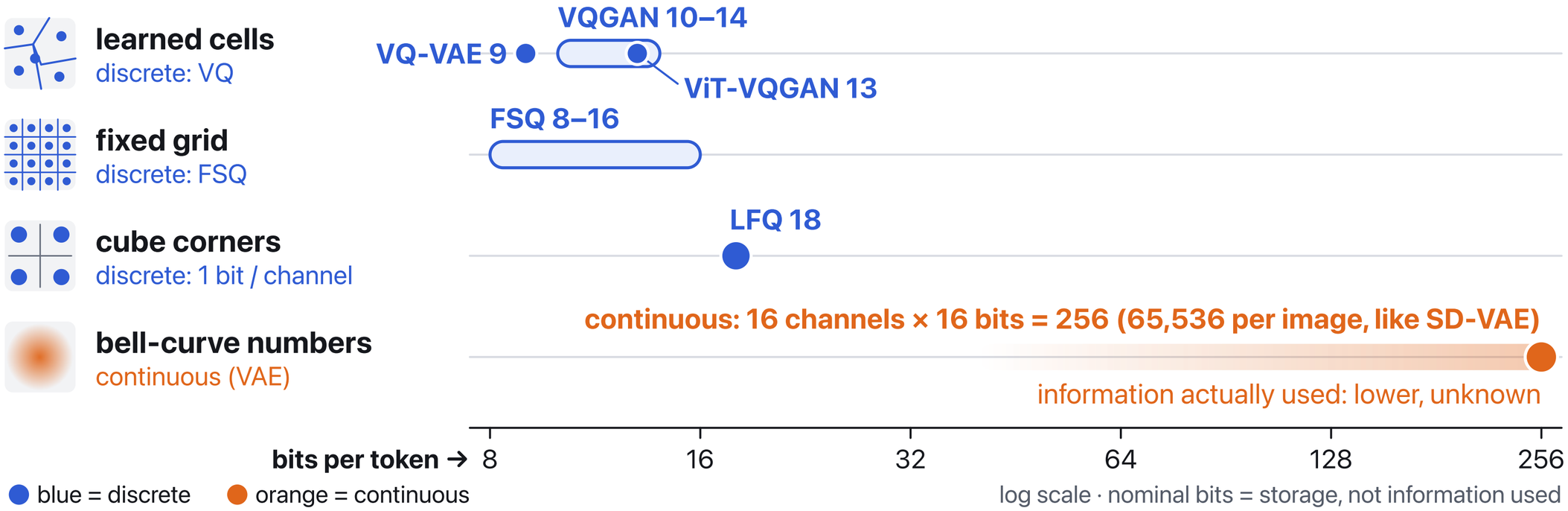
The path so far on one map



Codebook sizes: the five papers above · 16 bits per channel as in Yu et al., BAR, ICML 2026 · Arithmetic: ours

PART 3 · UNIFIED VIEW

The path so far on one map



TAKEAWAY

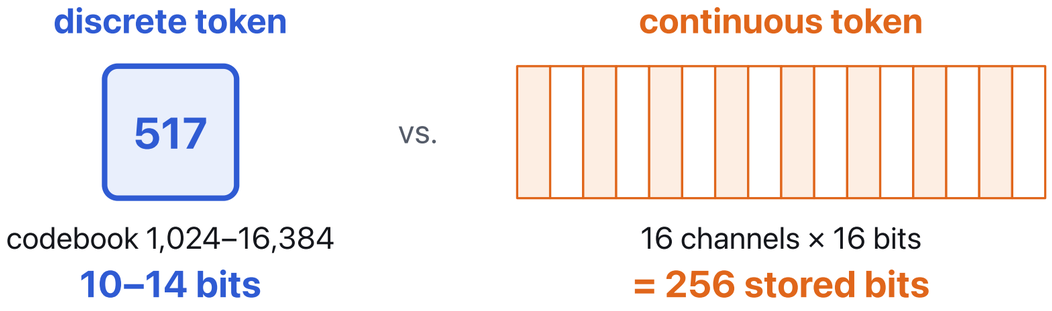
Most fixes changed how the bits are used. The count stayed between 8 and 18.

Codebook sizes: the five papers above · 16 bits per channel as in Yu et al., BAR, ICML 2026 · Arithmetic: ours

Rounding, or just too few bits?

(a) Rounding loses information? The textbook answer.

(b) Far fewer bits? A VQ token stores 14 bits; a 16-channel float token stores 256.



256 yes/no answers = $2^{256} \approx 10^{77}$ stored patterns

storage, not information used; a fixed grid, not a learned codebook

Rombach et al., LDM, CVPR 2022 · Tab. 8 · 16 bits per channel as in Yu et al., BAR, ICML 2026 · Arithmetic: ours

Rounding, or just too few bits?

(a) Rounding loses information? The textbook answer.

(b) Far fewer bits? A VQ token stores 14 bits; a 16-channel float token stores 256.

discrete token

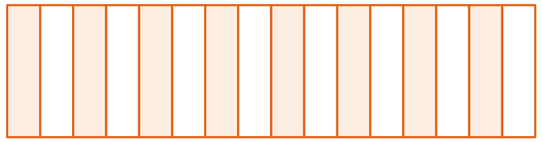
517

codebook 1,024–16,384

10–14 bits

vs.

continuous token

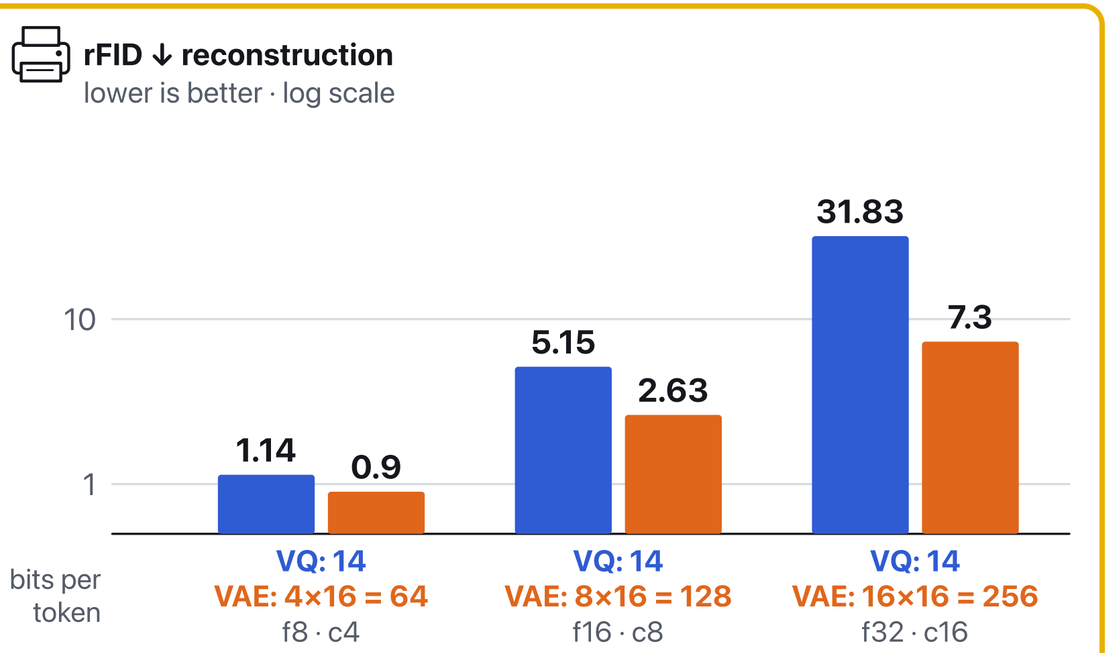


16 channels × 16 bits

= 256 stored bits

256 yes/no answers = $2^{256} \approx 10^{77}$ stored patterns

storage, not information used; a fixed grid, not a learned codebook



Same autoencoder, **discrete VQ** or **continuous VAE** bottleneck (f8·c4: 8×8 pixels per token, 4 numbers)

Source: LDM, Tab. 8 (rFID ↓ reconstruction) · Arithmetic: ours

Rombach et al., LDM, CVPR 2022 · Tab. 8 · 16 bits per channel as in Yu et al., BAR, ICML 2026 · Arithmetic: ours

Rounding, or just too few bits?

(a) Rounding loses information? The textbook answer.

(b) Far fewer bits? A VQ token stores 14 bits; a 16-channel float token stores 256.

discrete token

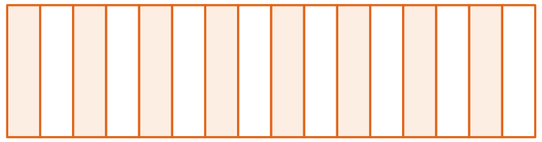
517

codebook 1,024–16,384

10–14 bits

vs.

continuous token



16 channels × 16 bits

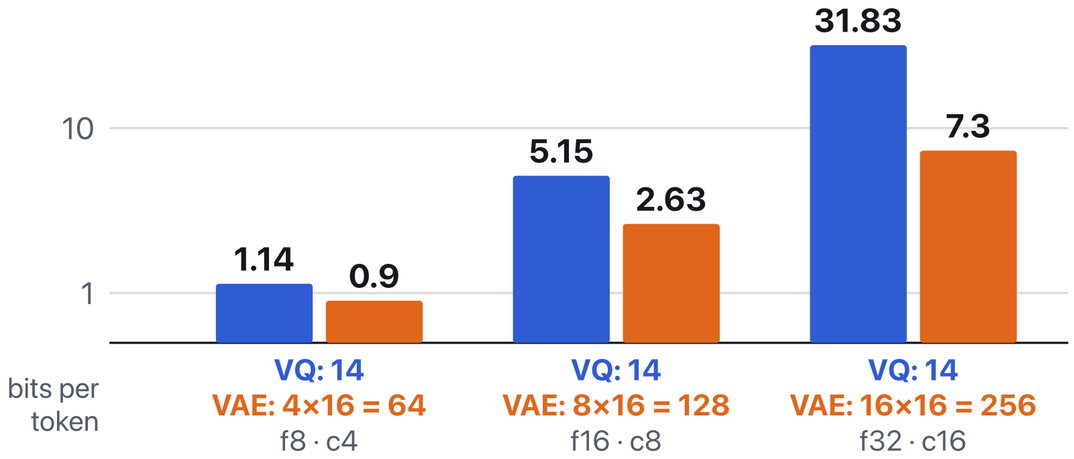
= 256 stored bits

256 yes/no answers = $2^{256} \approx 10^{77}$ stored patterns

storage, not information used; a fixed grid, not a learned codebook

rFID ↓ reconstruction

lower is better · log scale



Same autoencoder, **discrete VQ** or **continuous VAE** bottleneck (f8·c4: 8×8 pixels per token, 4 numbers)

Source: LDM, Tab. 8 (rFID ↓ reconstruction) · Arithmetic: ours

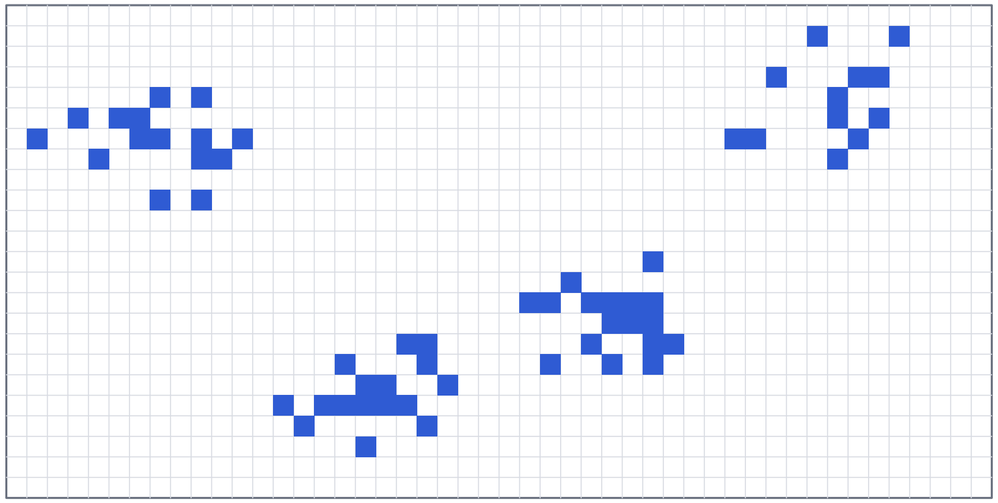
TAKEAWAY

My guess: the gap is mostly the bit budget, not discreteness.

Rombach et al., LDM, CVPR 2022 · Tab. 8 · 16 bits per channel as in Yu et al., BAR, ICML 2026 · Arithmetic: ours

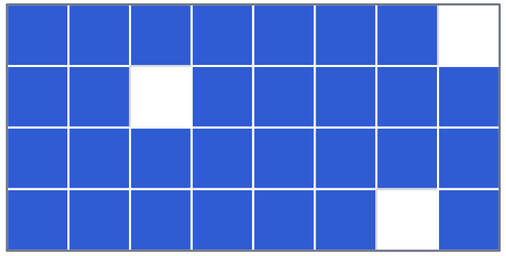
Capacity vs. utilisation

big space, sparse use



high capacity · low utilisation

small space, dense use



low capacity · high utilisation

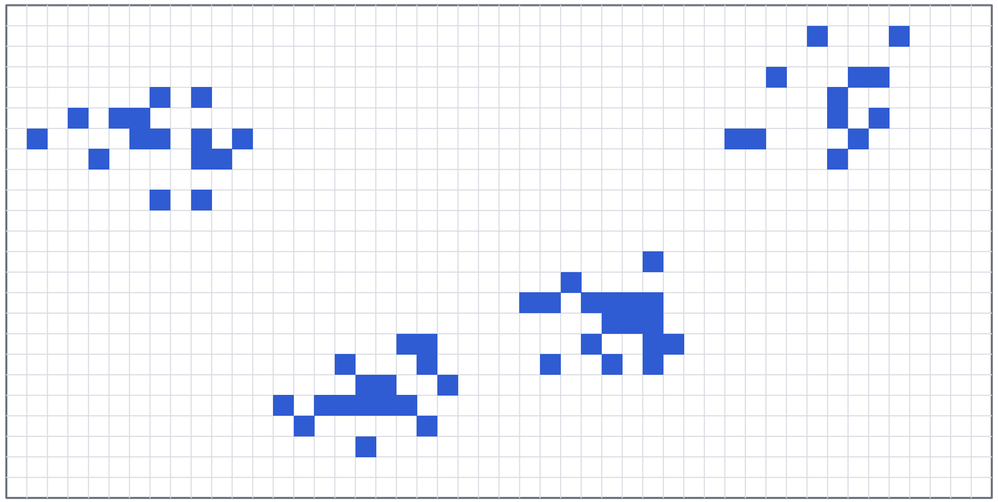
each square = one code

Capacity: how big the space is.

Schematic: ours · rate-distortion-modelability: Dieleman, sander.ai blog, 2025

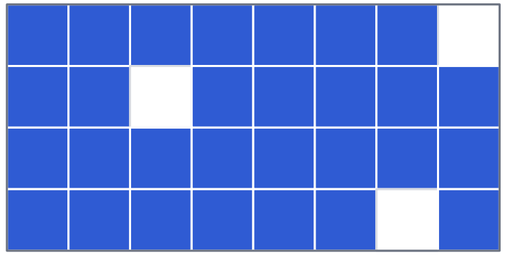
Capacity vs. utilisation

big space, sparse use



high capacity · low utilisation

small space, dense use



low capacity · high utilisation

each square = one code

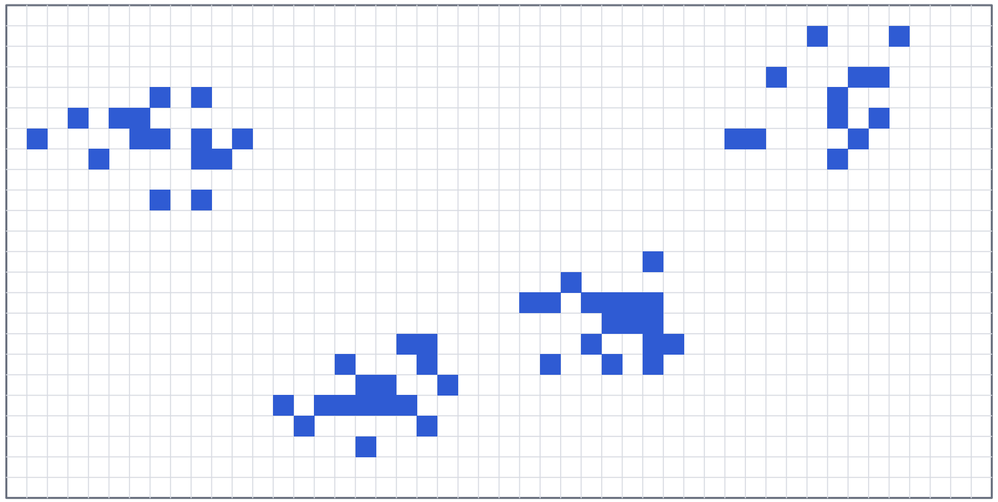
Capacity: how big the space is.

Utilisation: how much of it real images use.

Schematic: ours · rate-distortion-modelability: Dieleman, sander.ai blog, 2025

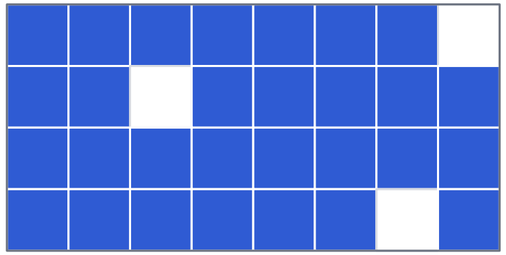
Capacity vs. utilisation

big space, sparse use



high capacity · low utilisation

small space, dense use



low capacity · high utilisation

each square = one code

Capacity: how big the space is.

Utilisation: how much of it real images use.

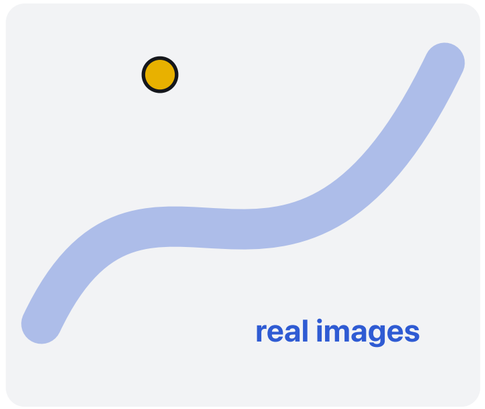
TAKEAWAY

Space real images never use is wasted. Space they do use is what the generator must learn. So more bits help the copy, but they also give the generator more to learn.

Schematic: ours · rate-distortion-modelability: Dieleman, sander.ai blog, 2025

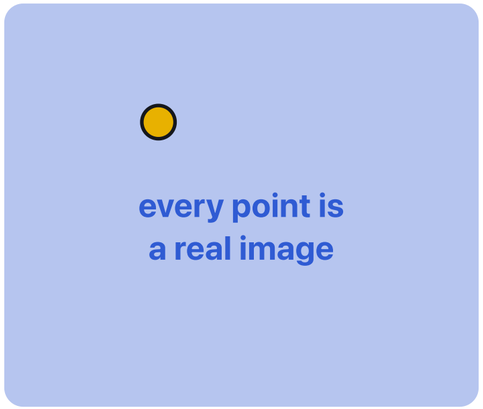
A generator with zero parameters?

most points mean nothing



random point → not an image ✕

every point means something



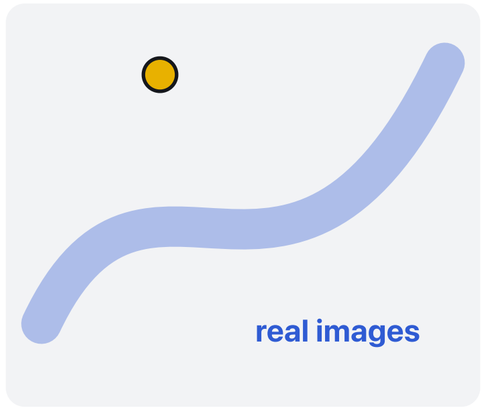
random point → an image ✓

generator =
pick at random

0 parameters
(no learned numbers)

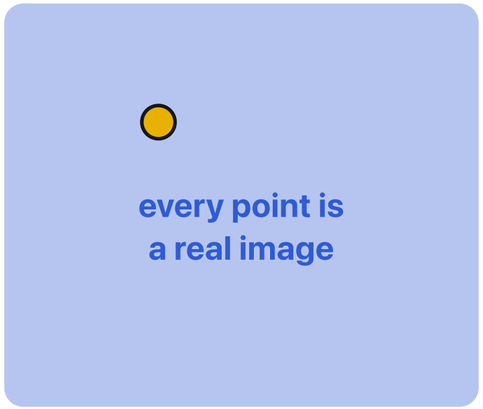
A generator with zero parameters?

most points mean nothing



random point → not an image ✕

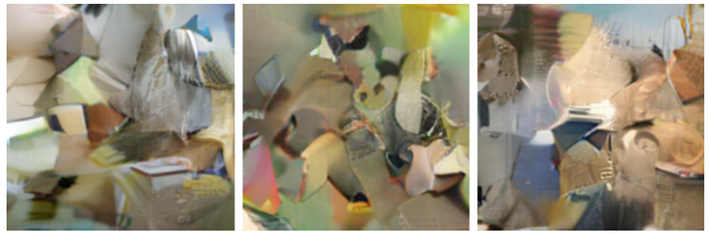
every point means something



random point → an image ✓

generator =
pick at random

0 parameters
(no learned numbers)



Each token drawn on its own, from real code frequencies: a soup of patches

Source: FSQ, App. Fig. 9

Every code gets used, but most combinations of codes still aren't images.

If every random code were an image, the work would move into the decoder.

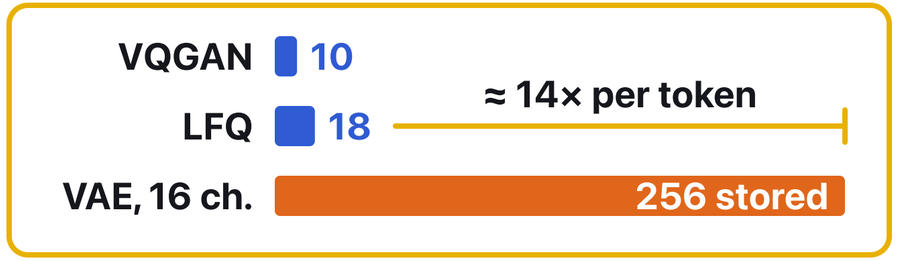
What still limits discrete tokenizers?

VQ-VAE	VQGAN	ViT-VQGAN	FSQ	LFQ (MAGVIT-v2)	Masked bit modeling
2017	2021	2022	2024	2024	
a learned discrete code, trained end to end	short, sharp grids: the decoder paints detail	tidy lookup space: almost every code used	no codebook to learn: bound and round	sign per channel: vocabulary grows cheaply	**enough bits, a head for any vocabulary**

1

Too few bits per token.

8–18 bits, vs. 256 stored bits for a 16-bit float latent



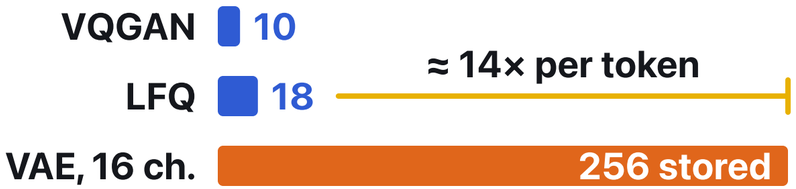
What still limits discrete tokenizers?

VQ-VAE	VQGAN	ViT-VQGAN	FSQ	LFQ (MAGVIT-v2)	Masked bit modeling
2017	2021	2022	2024	2024	
a learned discrete code, trained end to end	short, sharp grids: the decoder paints detail	tidy lookup space: almost every code used	no codebook to learn: bound and round	sign per channel: vocabulary grows cheaply	**enough bits, a head for any vocabulary**

1

Too few bits per token.

8–18 bits, vs. 256 stored bits for a 16-bit float latent



2

Scoring every code at once can't grow.

one softmax score per code: 2^{18} = 262,144 per token

04

Bit representations: bridging discrete and continuous

What if discrete tokens get enough bits?

1 Images are not text ✓ | 2 Discrete vs. continuous ✓ | 3 The discrete path ✓ | **4 Bits bridge the gap**

Inside a computer every latent is a finite code, so discrete and continuous tokenizers sit on one axis: **bits**. Stored bits are an upper bound on what they really carry. What differs is **how those bits are shaped for the generator**. More bits keep helping reconstruction.

Part 4 compares **stored** (nominal) bits, since we can't measure what a VAE really carries.

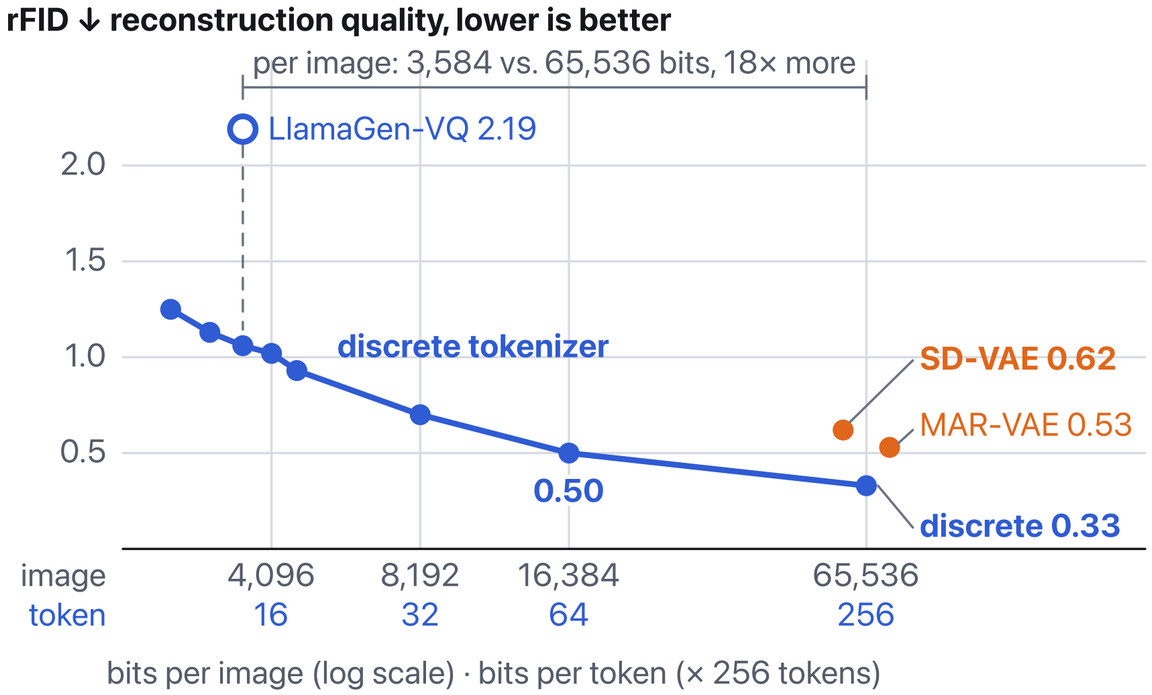
Per image (bits per token × 256): 16,384-code VQ, e.g. LlamaGen's, 14 × 256 = 3,584 · continuous VAE 256 × 256 = 65,536

Based on: Yu et al., BAR, ICML 2026 · LlamaGen: Sun et al., LlamaGen, arXiv 2024 (3,584 bits: BAR Fig. 3) · Arithmetic: ours

At equal bits, discrete copies as well or better

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

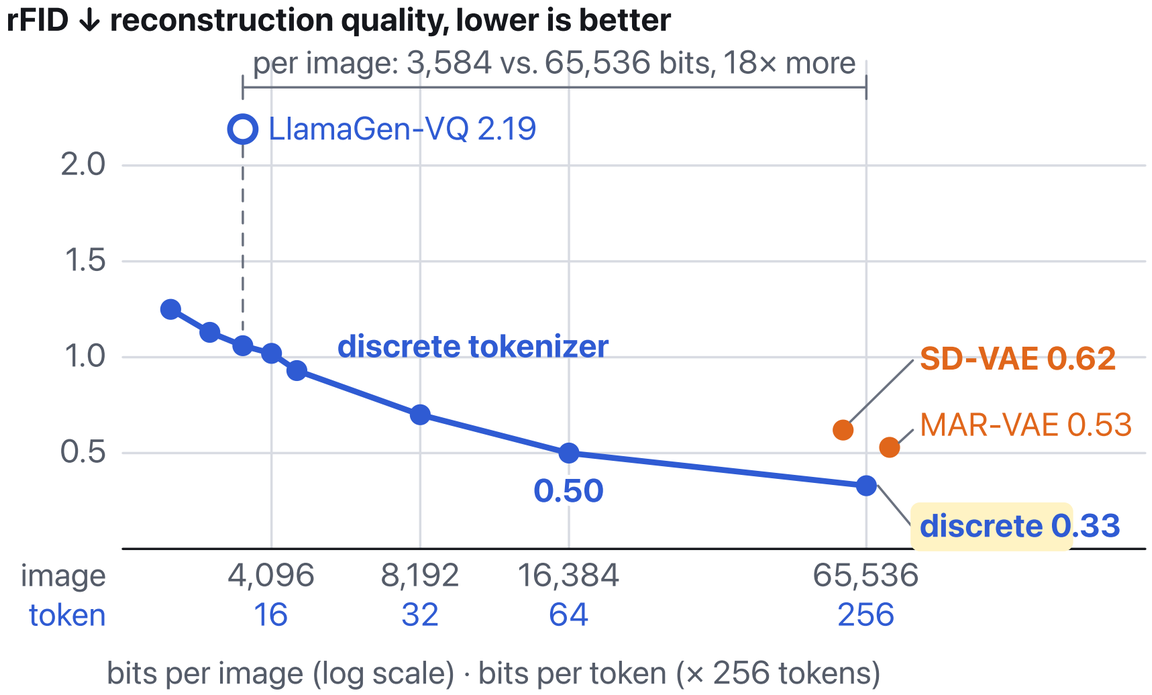
fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



At equal bits, discrete copies as well or better

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



With today's training recipes, discrete tokenizers make better use of a given (nominal) bit budget.

0.33 rFID at 65,536 bits

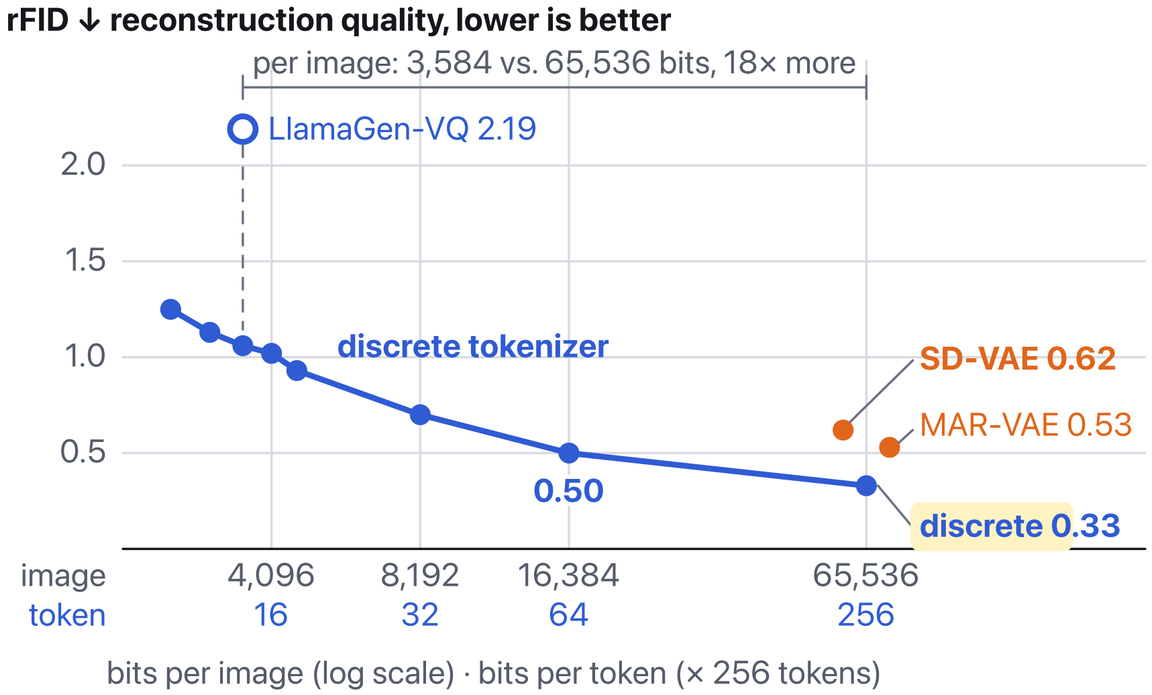
SD-VAE at the same bits: **0.62**

Yu et al., BAR, ICML 2026 · Tab. 1, Fig. 3–4, Sec. 3.3 (re-drawn; 256×256 images) · Arithmetic: ours

At equal bits, discrete copies as well or better

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



With today's training recipes, discrete tokenizers make better use of a given (nominal) bit budget.

0.33 rFID at 65,536 bits

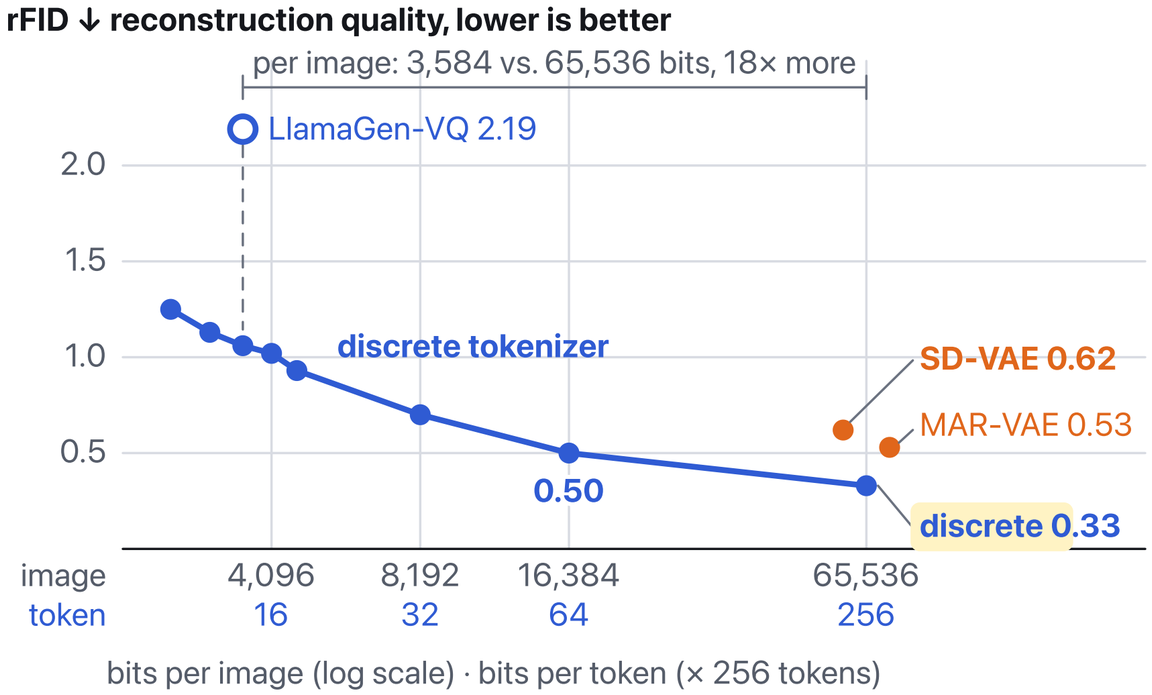
SD-VAE at the same bits: **0.62**

Matches continuous VAEs with **4× fewer bits** (0.50 at 16,384).

At equal bits, discrete copies as well or better

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



With today's training recipes, discrete tokenizers make better use of a given (nominal) bit budget.

0.33 rFID at 65,536 bits

SD-VAE at the same bits: **0.62**

Matches continuous VAEs with **4× fewer bits** (0.50 at 16,384).

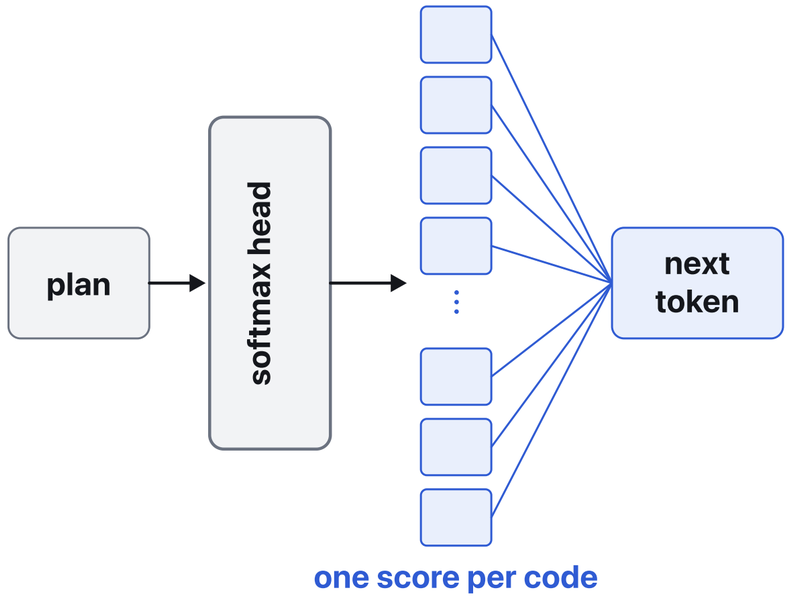
Reconstruction only; tokenizers differ in architecture and pre-training (BAR-FSQ: SigLIP2-initialised encoder, ViT-L decoder), so this is not a controlled comparison.

Yu et al., BAR, ICML 2026 · Tab. 1, Fig. 3–4, Sec. 3.3 (re-drawn; 256×256 images) · Arithmetic: ours

How do you generate from 2^{256} choices?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits** ●

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



The usual last layer: a softmax over the K codes

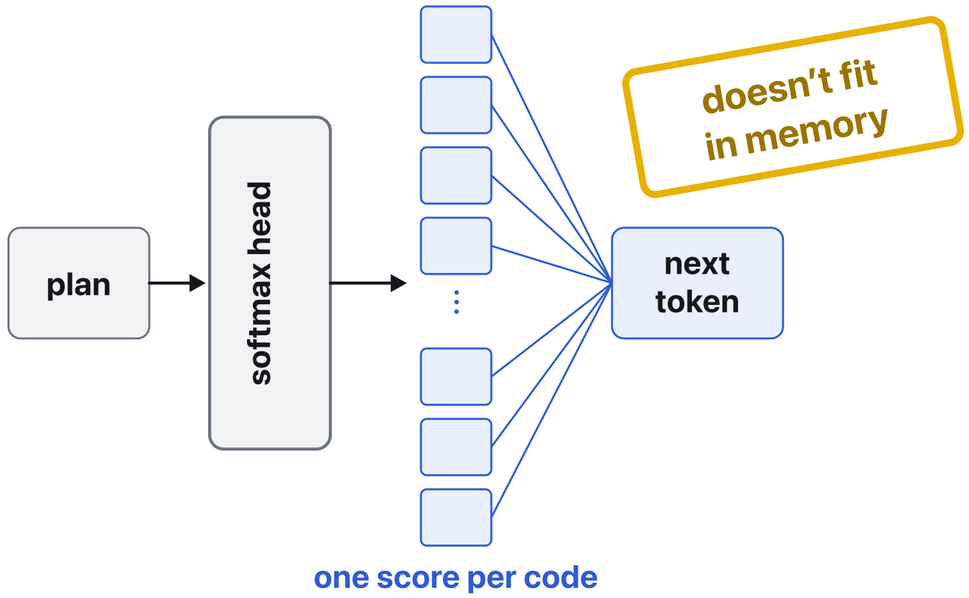
A token generator usually gives **every** code a score, then picks one.

Head = the small last part of a network that turns its output into an answer.

How do you generate from 2^{256} choices?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



The usual last layer: a softmax over the K codes

A token generator usually gives **every** code a score, then picks one.

Head = the small last part of a network that turns its output into an answer.

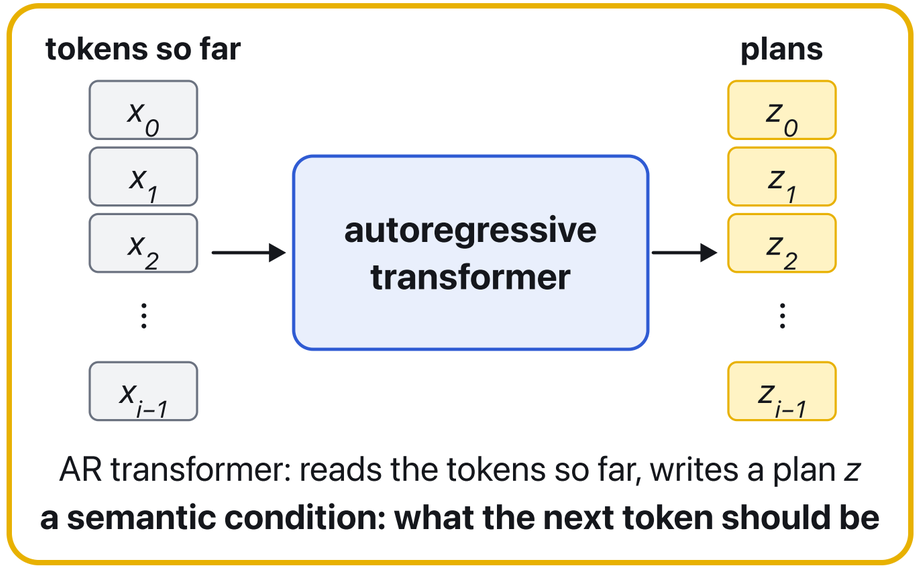
$2^{256} \approx 10^{77}$ scores per token

In practice the limit is about 2^{18} codes.

Plan the next token, then fill in its bits

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



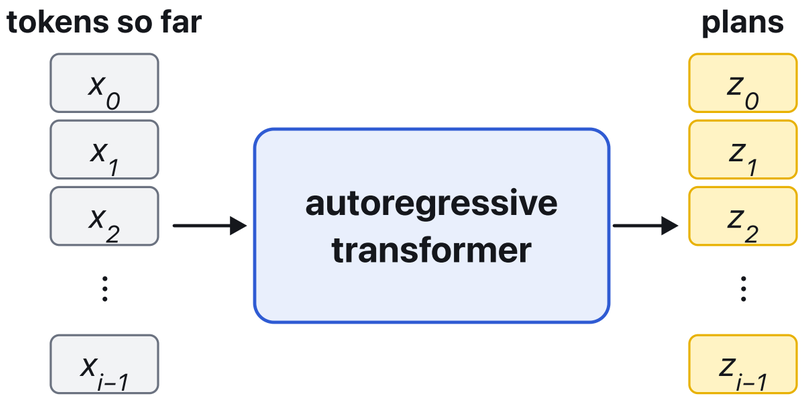
AR transformer: reads the tokens so far, writes a plan z

a semantic condition: what the next token should be

Plan the next token, then fill in its bits

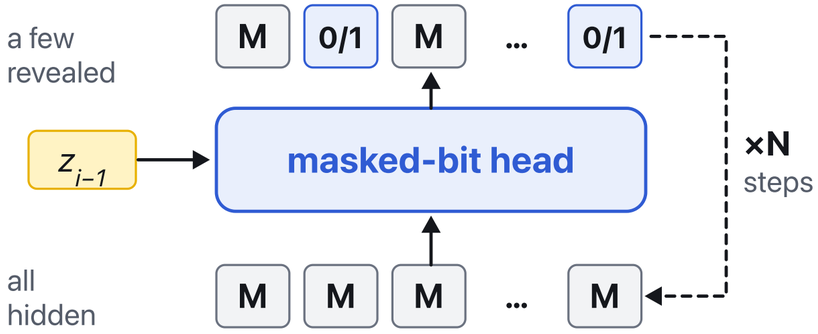
VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



AR transformer: reads the tokens so far, writes a plan *z*

a semantic condition: what the next token should be



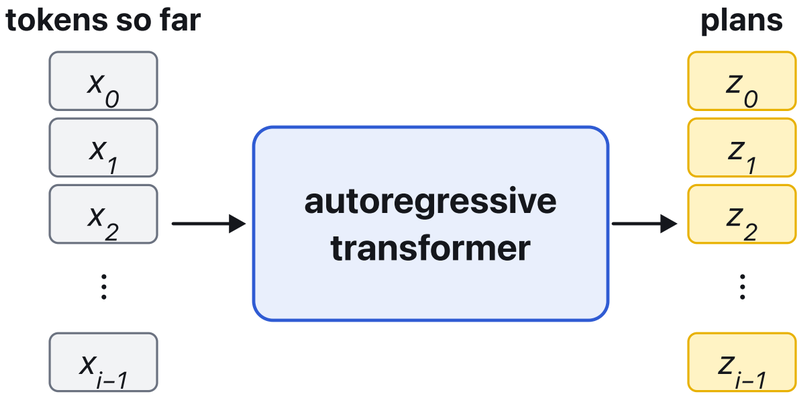
Masked-bit head: fills in its bits, a few per step

a rendering job: the low-level bits

Plan the next token, then fill in its bits

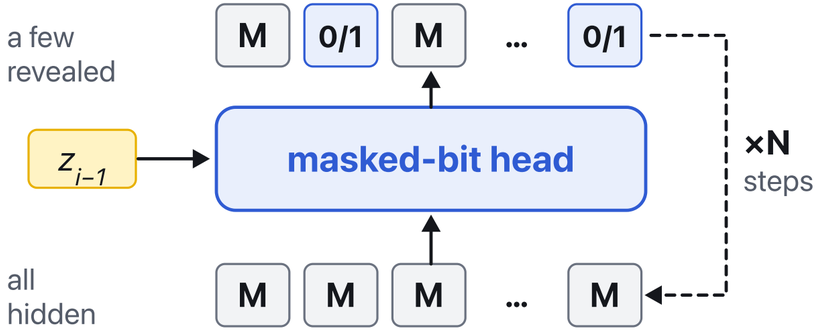
VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



AR transformer: reads the tokens so far, writes a plan *z*

a semantic condition: what the next token should be



Masked-bit head: fills in its bits, a few per step

a rendering job: the low-level bits

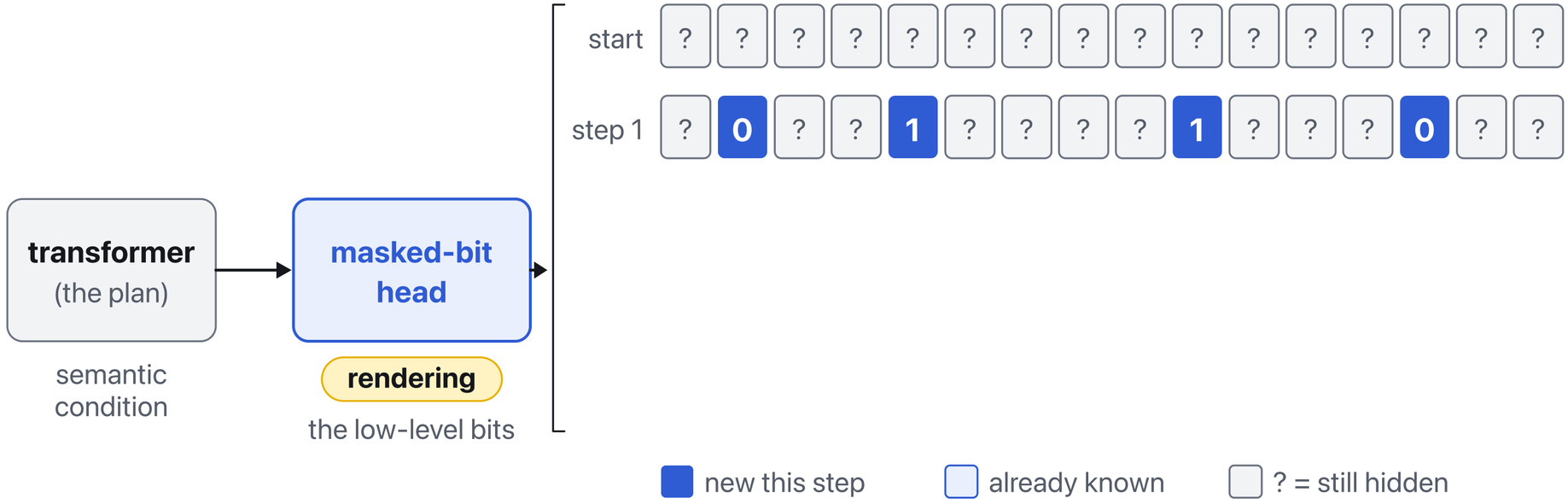
TAKEAWAY

Design intuition (our view): the transformer doesn't have to spell out the bits or score every code. Only the small head grows with the bits. Same idea as VQGAN's decoder painting the texture.

Filling in bits, step by step

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?

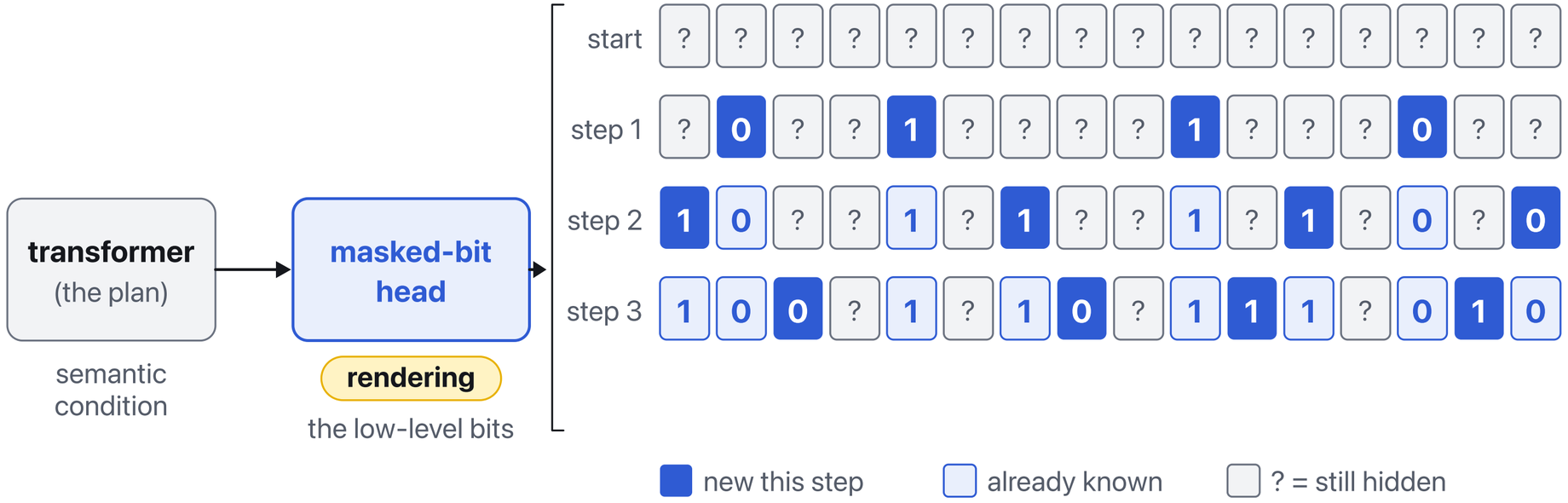


Each step reveals bits, using the plan and the bits so far

Filling in bits, step by step

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?

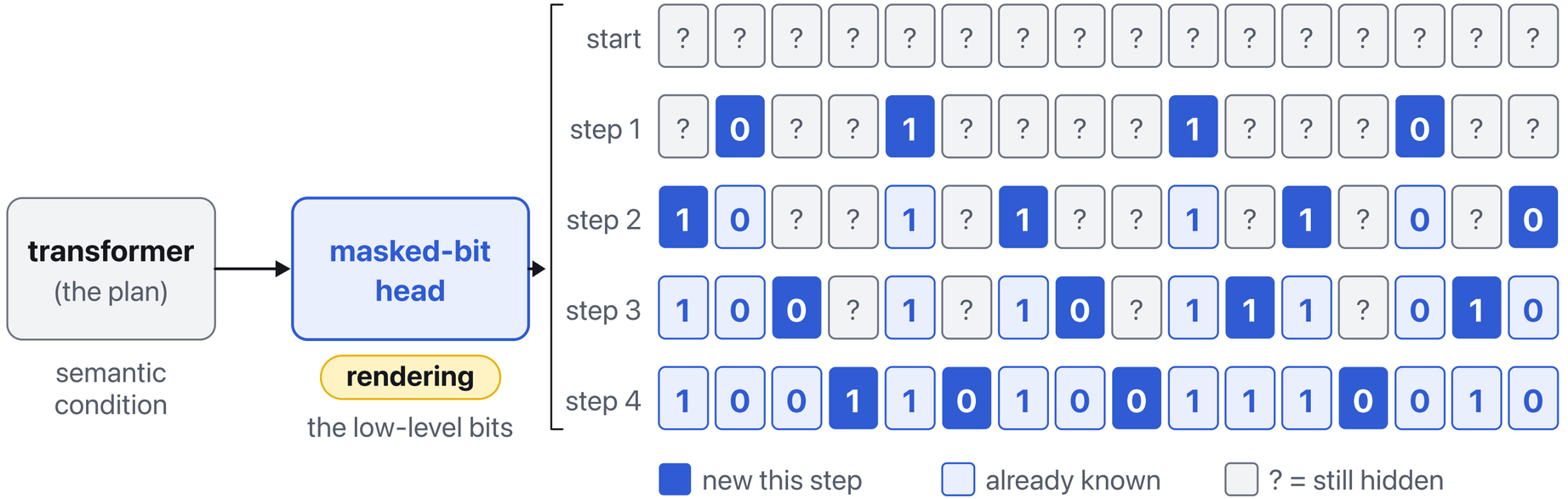


Each step reveals bits, using the plan and the bits so far

Filling in bits, step by step

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?

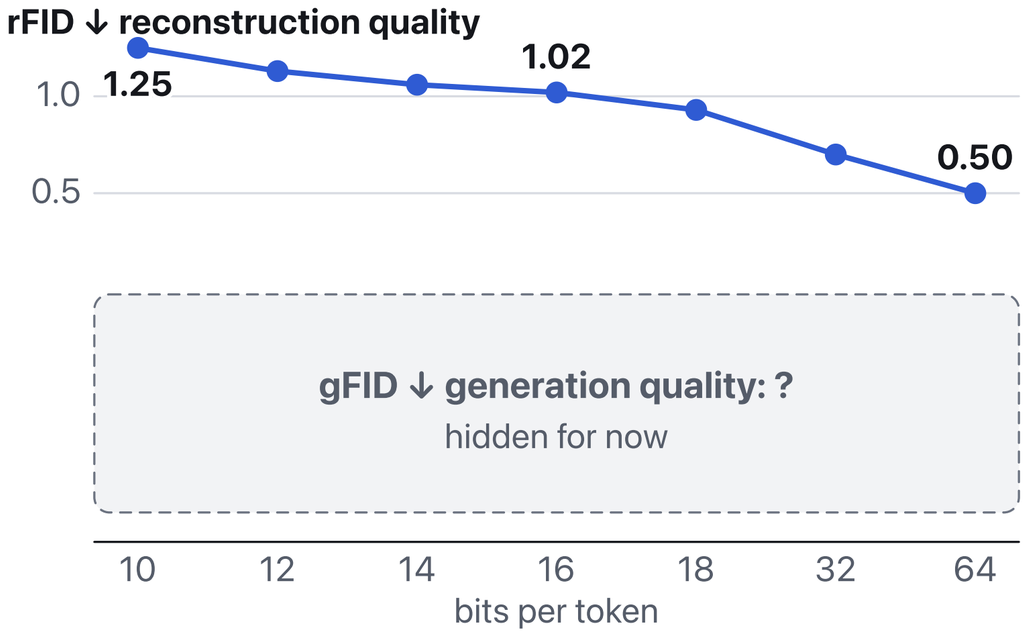


Each step reveals bits, using the plan and the bits so far

Do more bits keep helping generation?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



Your guess?

Reconstruction got better every time.

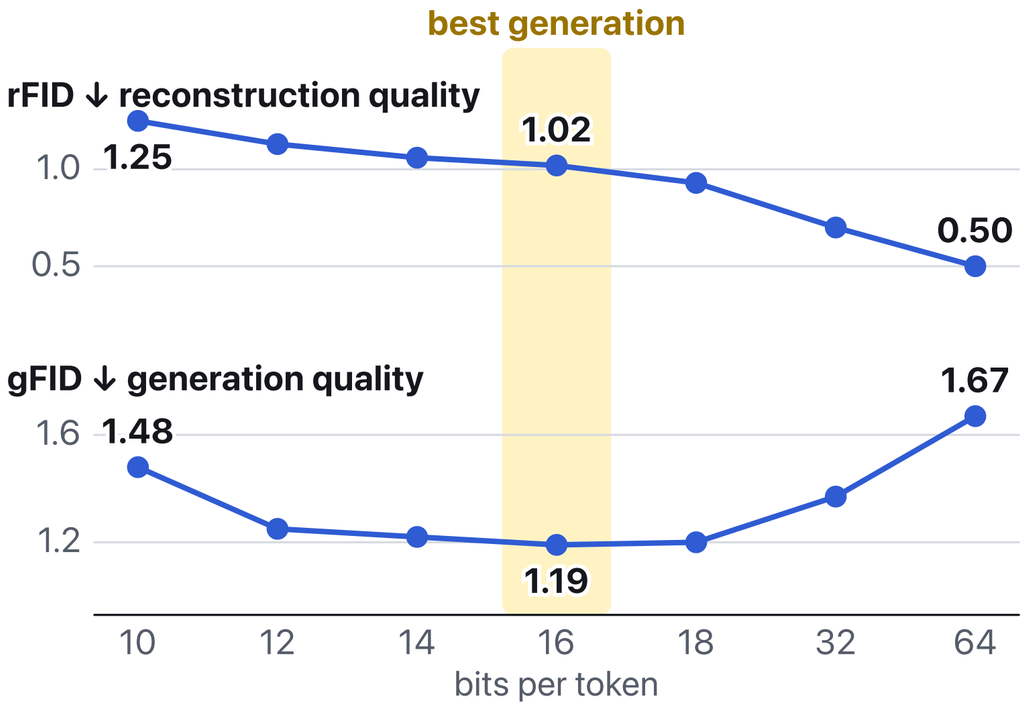
Does generation keep getting better too, all the way to 64 bits?

Yu et al., BAR, ICML 2026 · Tab. 1 · same trend: Esser et al., SD3, ICML 2024; Yao et al., VA-VAE, CVPR 2025

Do more bits keep helping generation?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



A bigger latent asks more of the generator.

Best around 16 bits: 1.19 (14 and 18 bits are within 0.03). 1.48 at 10, 1.67 at 64.

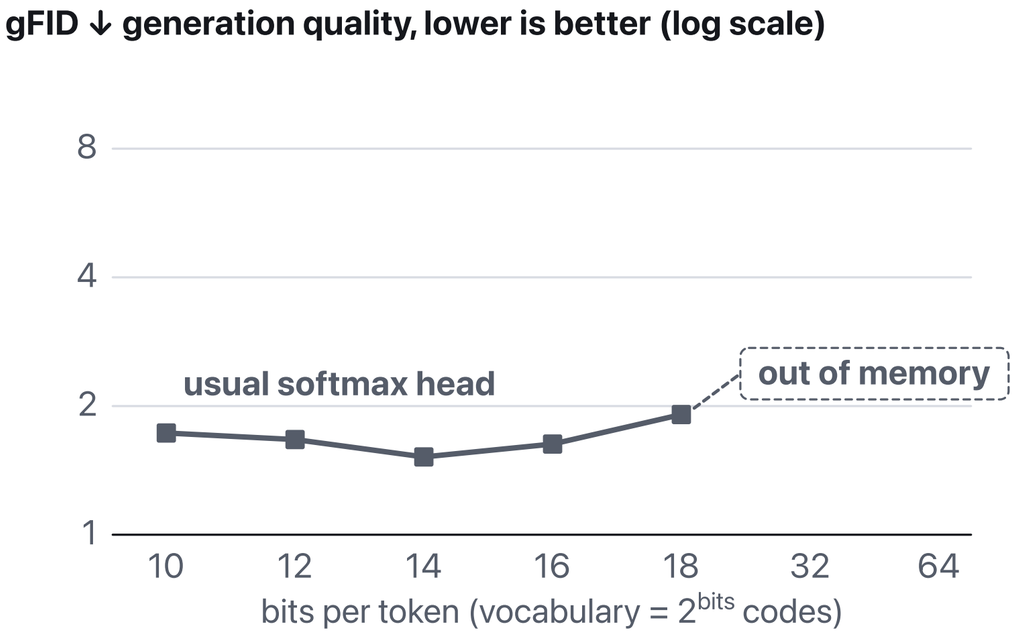
At a fixed recipe it needs a bigger generator or longer training. Continuous latents show the same pattern.

Yu et al., BAR, ICML 2026 · Tab. 1 · same trend: Esser et al., SD3, ICML 2024; Yao et al., VA-VAE, CVPR 2025

Which head keeps working as bits grow?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits** ●

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



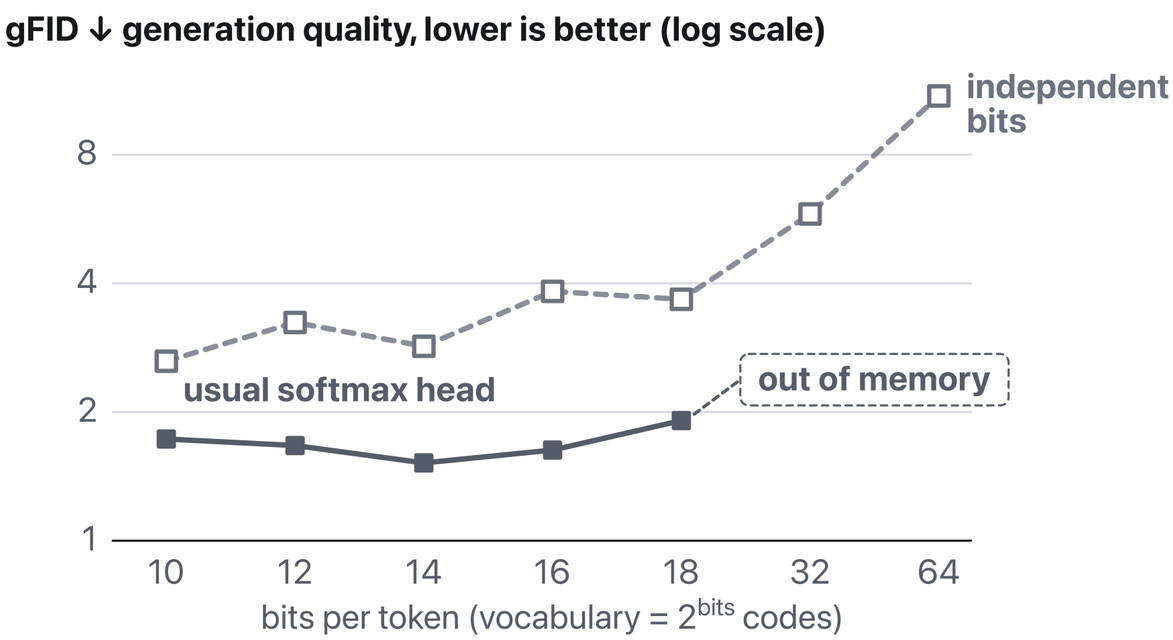
Generation quality by head, same tokenizers, ImageNet 256

Usual softmax: out of memory beyond 18 bits.

Which head keeps working as bits grow?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



Generation quality by head, same tokenizers, ImageNet 256

Usual softmax: out of memory beyond 18 bits.

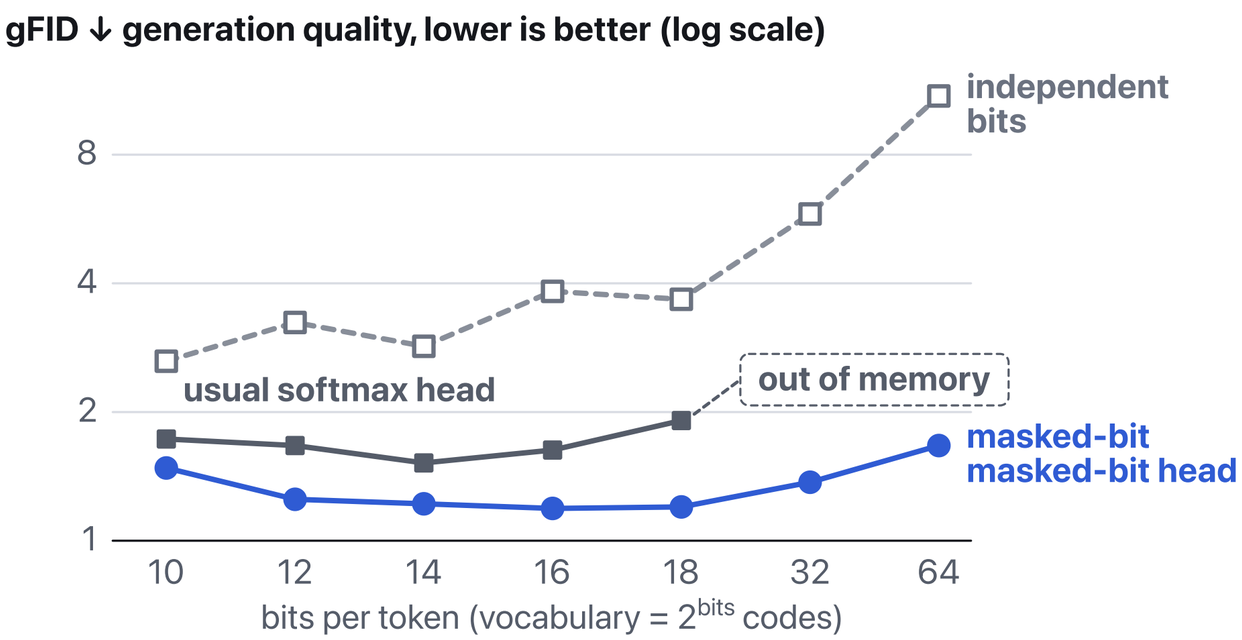
Independent bits: fits, but gets worse.

Yu et al., BAR, ICML 2026 · Tab. 1 (same data as Fig. 6), with guidance · re-drawn

Which head keeps working as bits grow?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



Generation quality by head, same tokenizers, ImageNet 256

Usual softmax: out of memory beyond 18 bits.

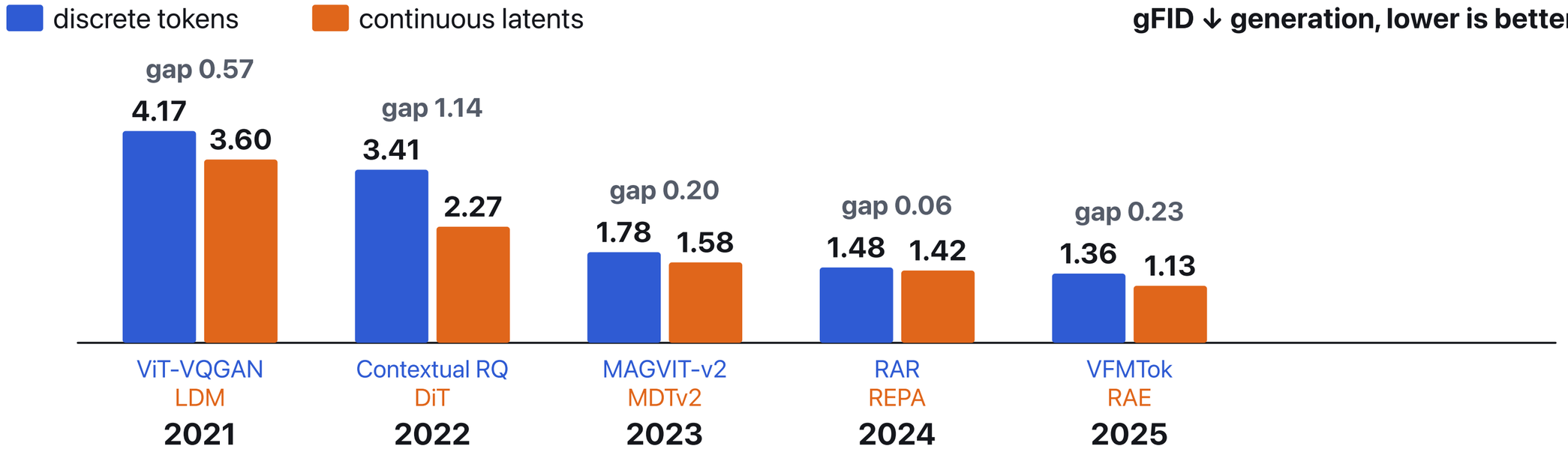
Independent bits: fits, but gets worse.

Masked-bit head: trains at every size, and is best wherever the softmax fits.

Back to the scoreboard

VQ-VAE 2017 → VQGAN 2021 → ViT-VQGAN 2022 → FSQ 2024 → LFQ (MAGVIT-v2) 2024 → **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?

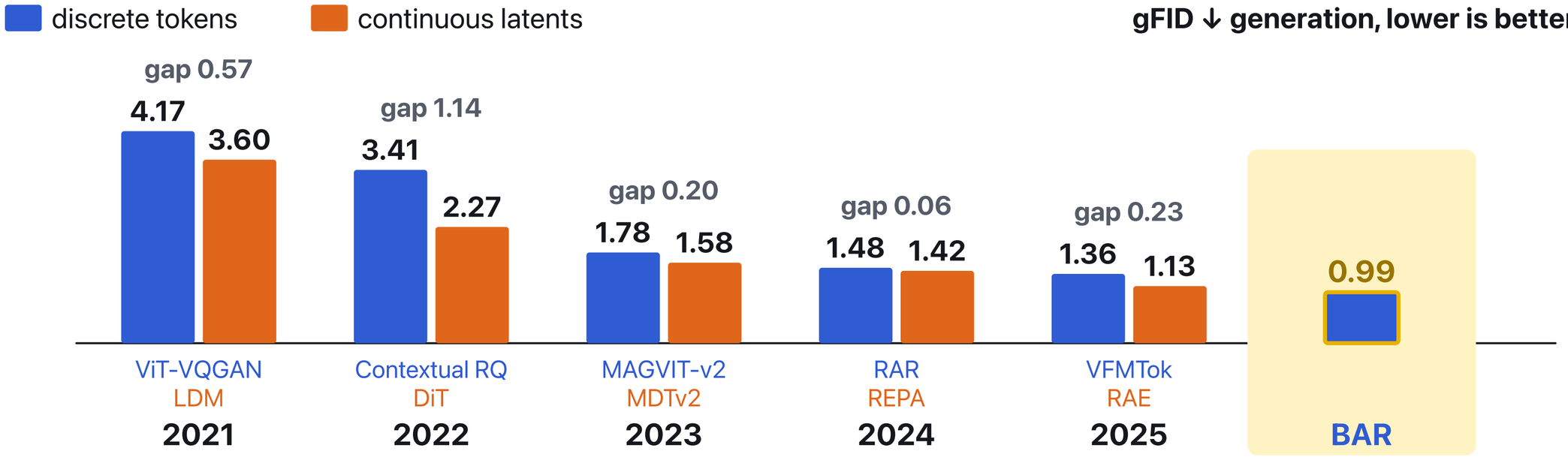


Continuous is leading, but the gap is shrinking: from over 1 point in 2022 to about 0.2 or less since 2023.

Back to the scoreboard

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?

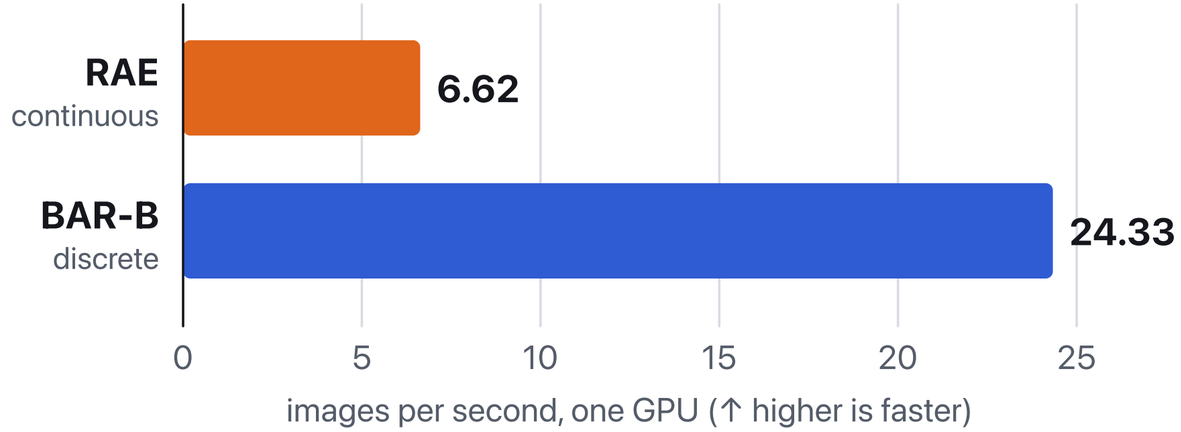


Best reported at the time of the paper. One result doesn't settle which kind is easier to generate.

Same quality, about 3.7× faster

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits**

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?



Sampling speed at the same gFID (↓ generation), 1.13

3.7×

faster at equal quality

Standard diffusion samplers run the full network 50-250 times per image.

Yu et al., BAR, ICML 2026 · Tab. 4 (one H200 GPU; re-drawn) · steps: Peebles & Xie, DiT, ICCV 2023; Yao et al., VA-VAE, CVPR 2025 (250); Zheng et al., RAE, ICLR 2026 (50)

What does it generate?

VQ-VAE 2017 ✓ — VQGAN 2021 ✓ — ViT-VQGAN 2022 ✓ — FSQ 2024 ✓ — LFQ (MAGVIT-v2) 2024 ✓ — **Masked bits** ●

fixed: enough bits, a head for any vocabulary · **still missing:** does generation keep improving with more bits?

Our BAR-L samples, ImageNet 256, class label only: husky, lorikeet, volcano

The whole path in one picture

VQ-VAE 2017
a learned discrete code, trained end to end
blurry at high compression

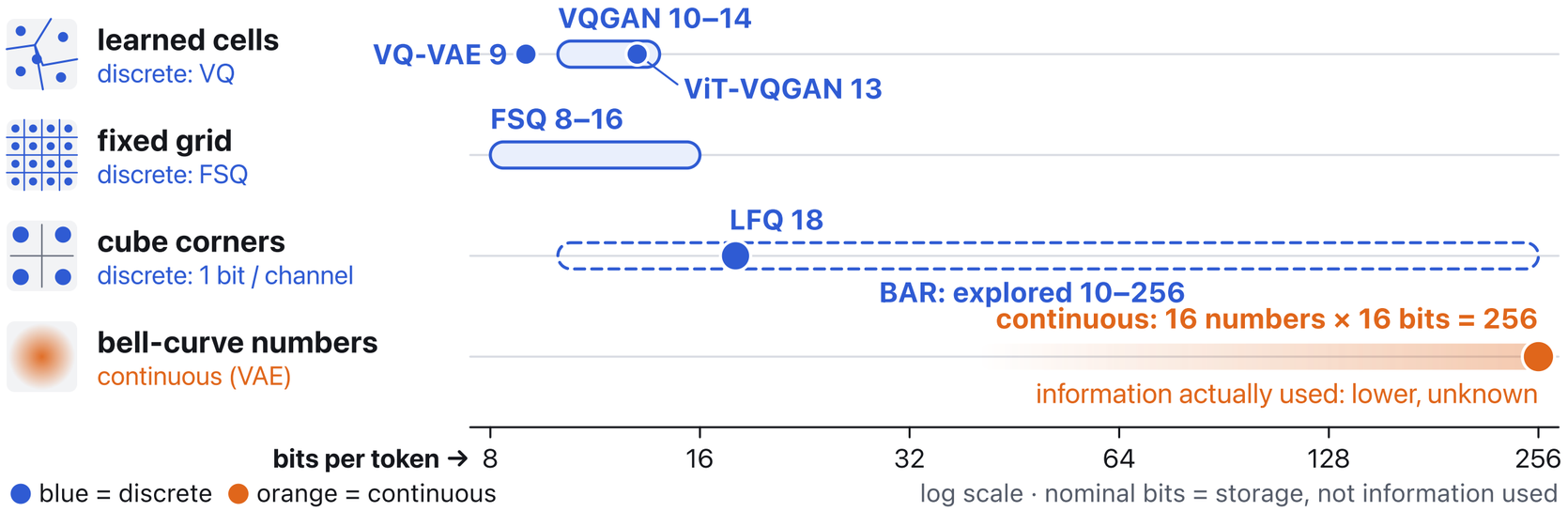
VQGAN 2021
short, sharp grids: decoder paints detail
most codes never used

ViT-VQGAN 2022
tidy lookup space: almost every code used
still a learned codebook, extra losses

FSQ 2024
no codebook to learn: bound and round
small vocabulary; does bigger help?

LFQ (MAGVIT-v2) 2024
sign per channel: vocabulary grows cheaply
a softmax over codes can't grow further

Masked bits
enough bits, a head for any vocabulary
generation has a sweet spot



Path: the six papers above · Yu et al., BAR, ICML 2026

The whole path in one picture

VQ-VAE 2017
a learned discrete code, trained end to end
blurry at high compression

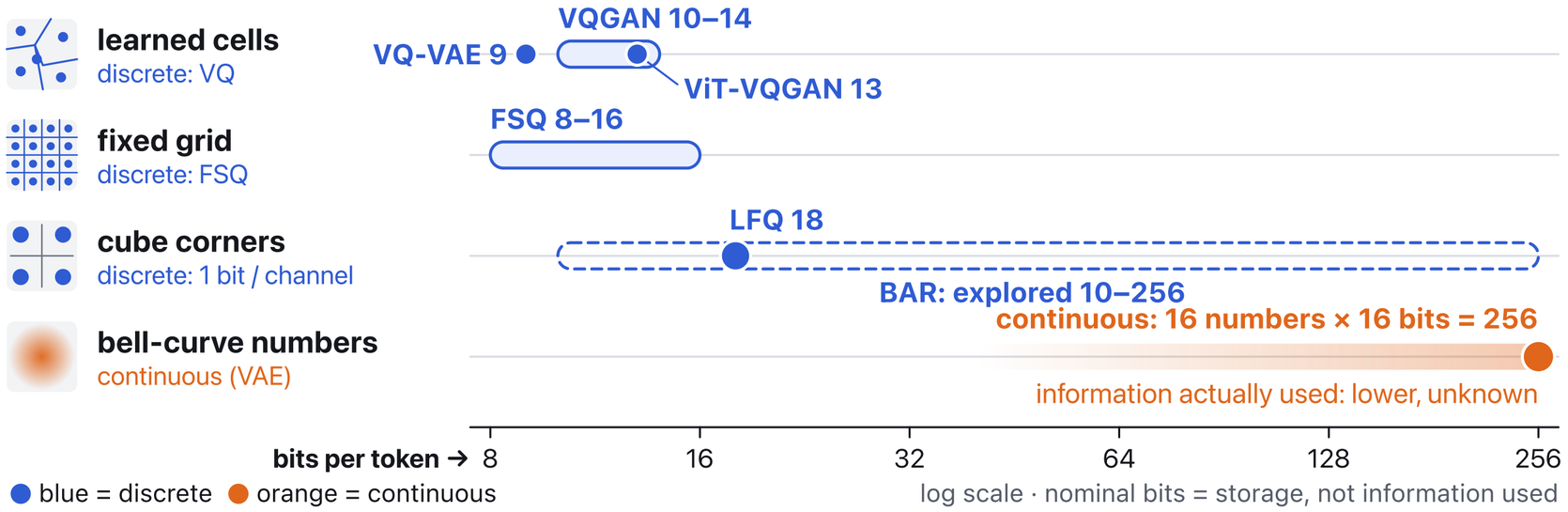
VQGAN 2021
short, sharp grids: decoder paints detail
most codes never used

ViT-VQGAN 2022
tidy lookup space: almost every code used
still a learned codebook, extra losses

FSQ 2024
no codebook to learn: bound and round
small vocabulary; does bigger help?

LFQ (MAGVIT-v2) 2024
sign per channel: vocabulary grows cheaply
a softmax over codes can't grow further

Masked bits
enough bits, a head for any vocabulary
generation has a sweet spot



Take-home

Give discrete tokens enough bits and a head that can use them.

Path: the six papers above · Yu et al., BAR, ICML 2026

The whole path in one picture

VQ-VAE 2017
a learned discrete code, trained end to end
blurry at high compression

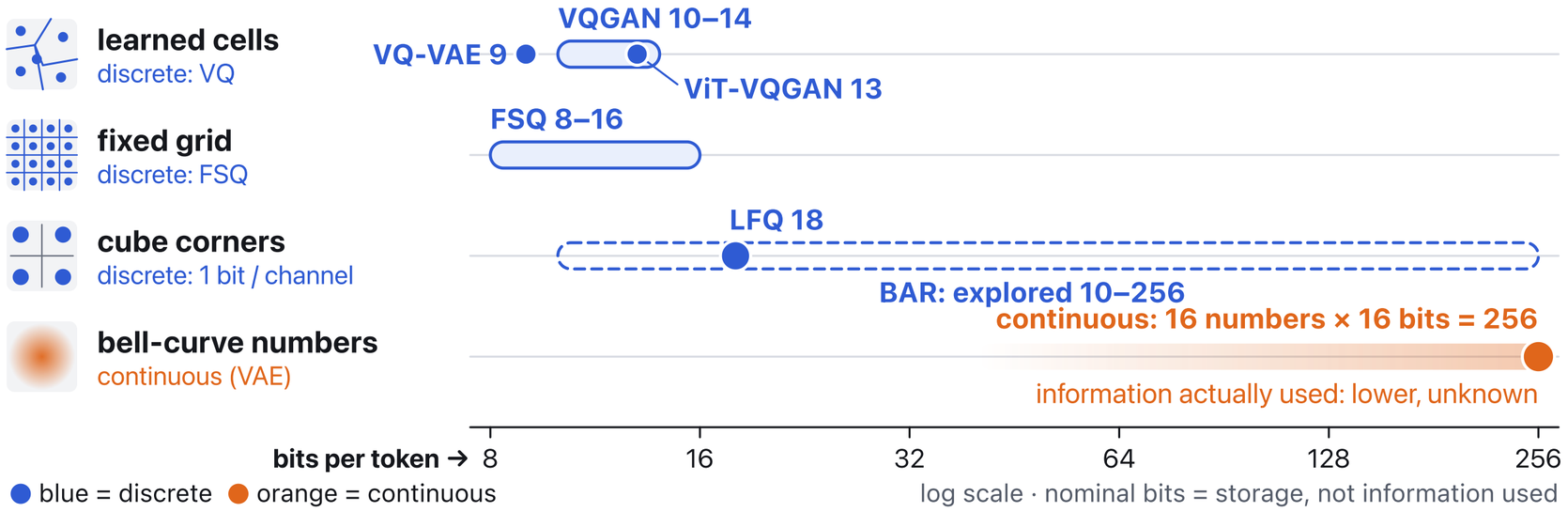
VQGAN 2021
short, sharp grids: decoder paints detail
most codes never used

ViT-VQGAN 2022
tidy lookup space: almost every code used
still a learned codebook, extra losses

FSQ 2024
no codebook to learn: bound and round
small vocabulary; does bigger help?

LFQ (MAGVIT-v2) 2024
sign per channel: vocabulary grows cheaply
a softmax over codes can't grow further

Masked bits
enough bits, a head for any vocabulary
generation has a sweet spot



Take-home

Give discrete tokens enough bits and a head that can use them.

What it changed

Discrete or continuous becomes a question of how many bits, and whether the generator can use them.

Path: the six papers above · Yu et al., BAR, ICML 2026

BEYOND IMAGES

The same choice shows up beyond images

One model that reads and writes text and images

Chameleon	Janus	Transfusion
images as codes	reads vectors, writes codes	images as vectors

Chameleon Team, Chameleon, arXiv 2024 · Wu et al., Janus, CVPR 2025 · Zhou et al., Transfusion, ICLR 2025 · Brohan et al., RT-2, CoRL 2023 ·
Fig. 1 · Pertsch et al., FAST, RSS 2025 · Black et al., π0, RSS 2025 · Fig. 3 · Spectrum: ours

BEYOND IMAGES

The same choice shows up beyond images

All discrete: symbols — **Hybrid** — **All continuous: vectors**

One model that reads and writes text and images

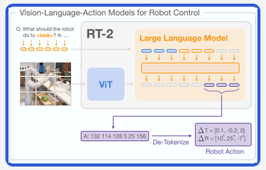
Chameleon
images as codes

Janus
reads vectors, writes codes

Transfusion
images as vectors

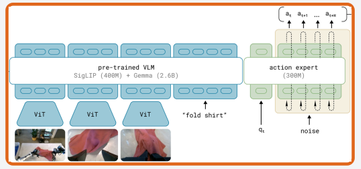
Robot actions: the motor commands a robot outputs

RT-2
each motor number → 1 of 256 levels



FAST
compressed action tokens

π0
vectors, made by flow matching



TAKEAWAY

Same question everywhere: symbols or vectors, and how many bits?

Chameleon Team, Chameleon, arXiv 2024 · Wu et al., Janus, CVPR 2025 · Zhou et al., Transfusion, ICLR 2025 · Brohan et al., RT-2, CoRL 2023 · Fig. 1 · Pertsch et al., FAST, RSS 2025 · Black et al., π0, RSS 2025 · Fig. 3 · Spectrum: ours

Some other related, interesting works

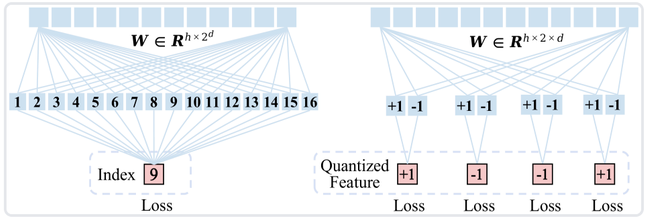
Discrete side

BSQ: LFQ's ±1 codes, rescaled to the same length.

MaskBit: the generator reads the bits directly, no lookup table.

Infinity: one yes/no prediction per bit, so the vocabulary can get huge.

Open-MAGVIT2: splits a 2^{18}-code vocabulary into two smaller ones.



Some other related, interesting works

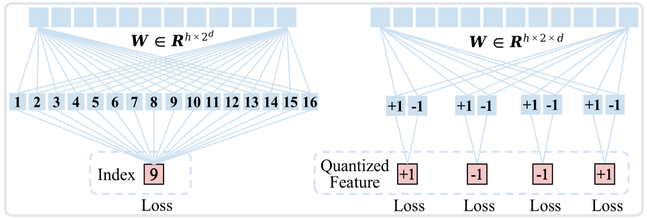
Discrete side

BSQ: LFQ's ±1 codes, rescaled to the same length.

MaskBit: the generator reads the bits directly, no lookup table.

Infinity: one yes/no prediction per bit, so the vocabulary can get huge.

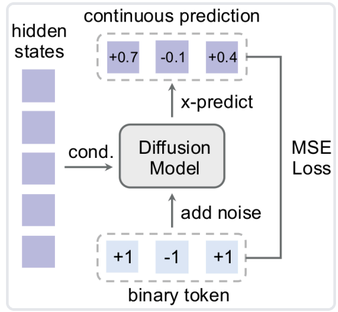
Open-MAGVIT2: splits a 2^{18}-code vocabulary into two smaller ones.



Crossing the divide

iFSQ: change one line of FSQ, and one discrete latent serves both AR and diffusion.

BitDance: binary tokens, but a small diffusion head makes each token's bits.



Some other related, interesting works

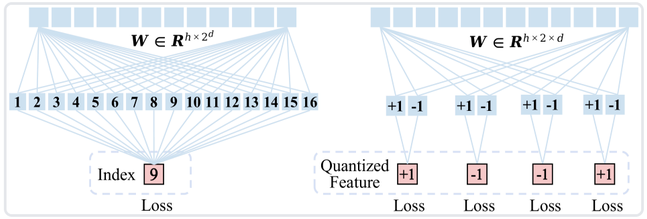
Discrete side

BSQ: LFQ's ±1 codes, rescaled to the same length.

MaskBit: the generator reads the bits directly, no lookup table.

Infinity: one yes/no prediction per bit, so the vocabulary can get huge.

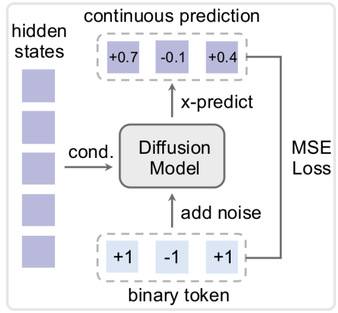
Open-MAGVIT2: splits a 2^{18}-code vocabulary into two smaller ones.



Crossing the divide

iFSQ: change one line of FSQ, and one discrete latent serves both AR and diffusion.

BitDance: binary tokens, but a small diffusion head makes each token's bits.



Continuous side

TarFlow, STARFlow: flows (reversible noise-to-image networks) with Transformers.

JetFormer: a flow turns pixels into vector tokens for the transformer.

GIVT, MAR: vectors instead of codes, still made one token at a time.

VA-VAE, RAE *(shaping the latent space with semantics)*: align the latent to a pretrained encoder, or use its features.

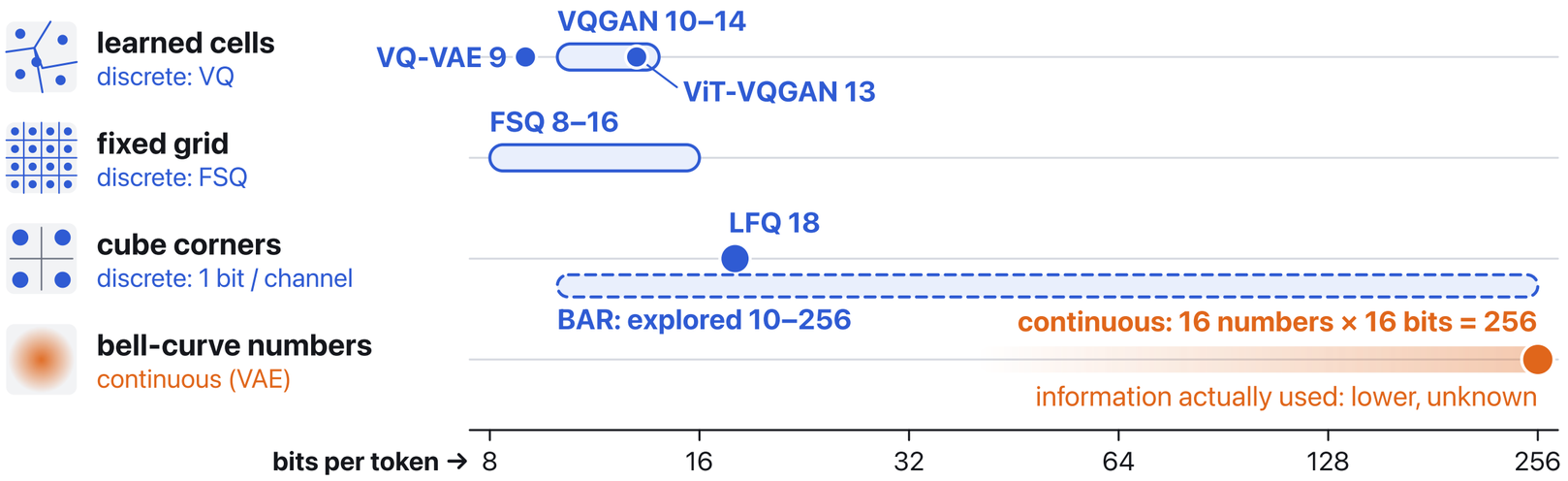
JiT, pMF *(no tokenizer at all)*: denoise pixels directly, with large patches; pMF in one step.

15 papers, arXiv v1 2023–2026 (venues in the notes) · Thumbnails: Infinity, CVPR 2025, Fig. 2 · BitDance, arXiv, head fig. (c)

TAKEAWAYS

What I hope you take home

Inside a computer every latent is a finite code, so discrete and continuous tokenizers sit on one axis: **bits**. Stored bits are an **upper bound** on what they really carry. What differs is **how those bits are shaped for the generator**. More bits keep helping reconstruction, but generation has a sweet spot.



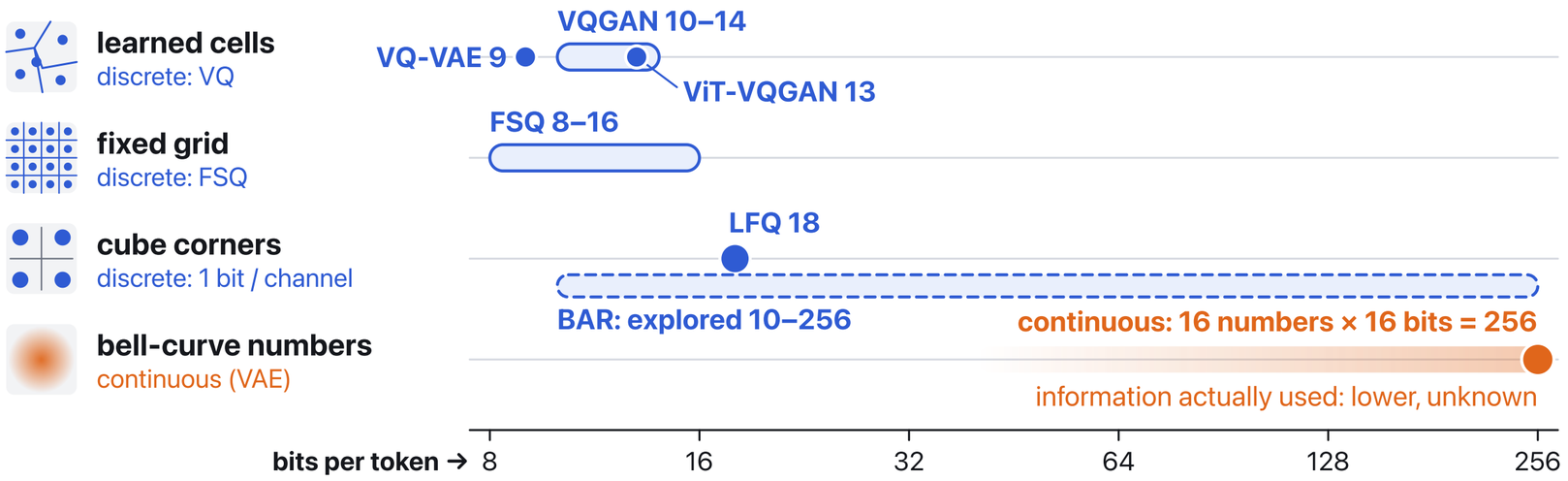
Bits per token, log scale. Stored bits, not information used.

Sweet spot: Yu et al., BAR, ICML 2026 · Tab. 1 · Teaching illustration: ours · Arithmetic: ours

TAKEAWAYS

What I hope you take home

Inside a computer every latent is a finite code, so discrete and continuous tokenizers sit on one axis: **bits**. Stored bits are an **upper bound** on what they really carry. What differs is **how those bits are shaped for the generator**. More bits keep helping reconstruction, but generation has a sweet spot.



Bits per token, log scale. Stored bits, not information used.

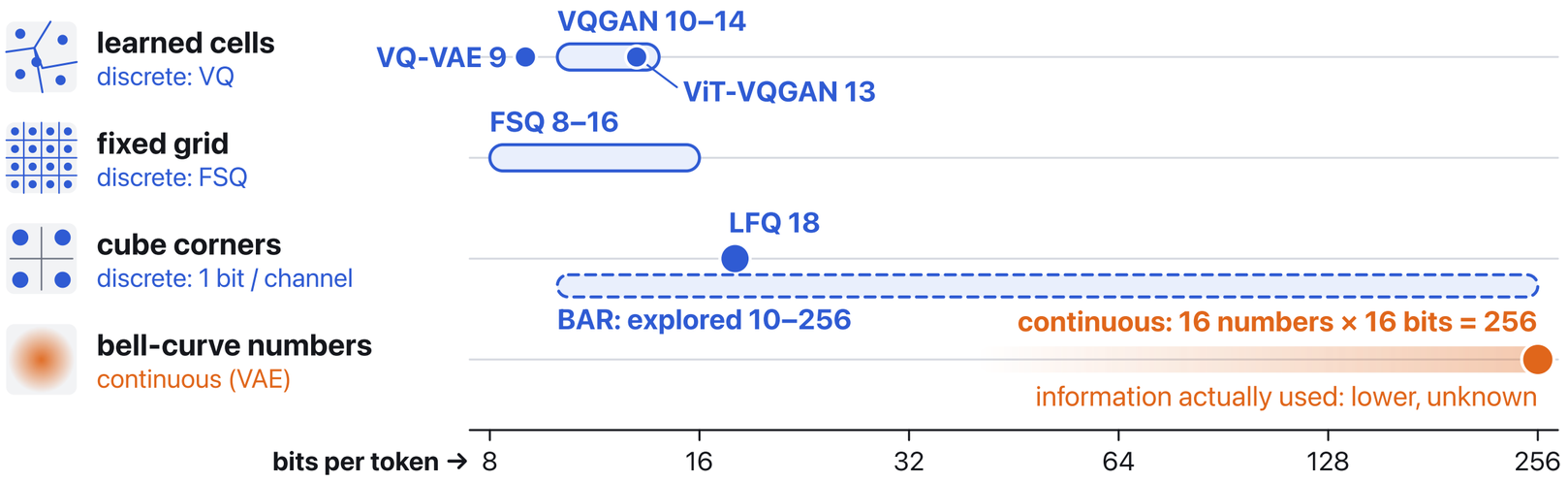
1 · A FLOAT IS A CODE TOO

Compare tokenizers by their bits, whatever the type.

TAKEAWAYS

What I hope you take home

Inside a computer every latent is a finite code, so discrete and continuous tokenizers sit on one axis: **bits**. Stored bits are an **upper bound** on what they really carry. What differs is **how those bits are shaped for the generator**. More bits keep helping reconstruction, but generation has a sweet spot.



Bits per token, log scale. Stored bits, not information used.

1 · A FLOAT IS A CODE TOO

Compare tokenizers by their bits, whatever the type.

2 · SHAPE MATTERS

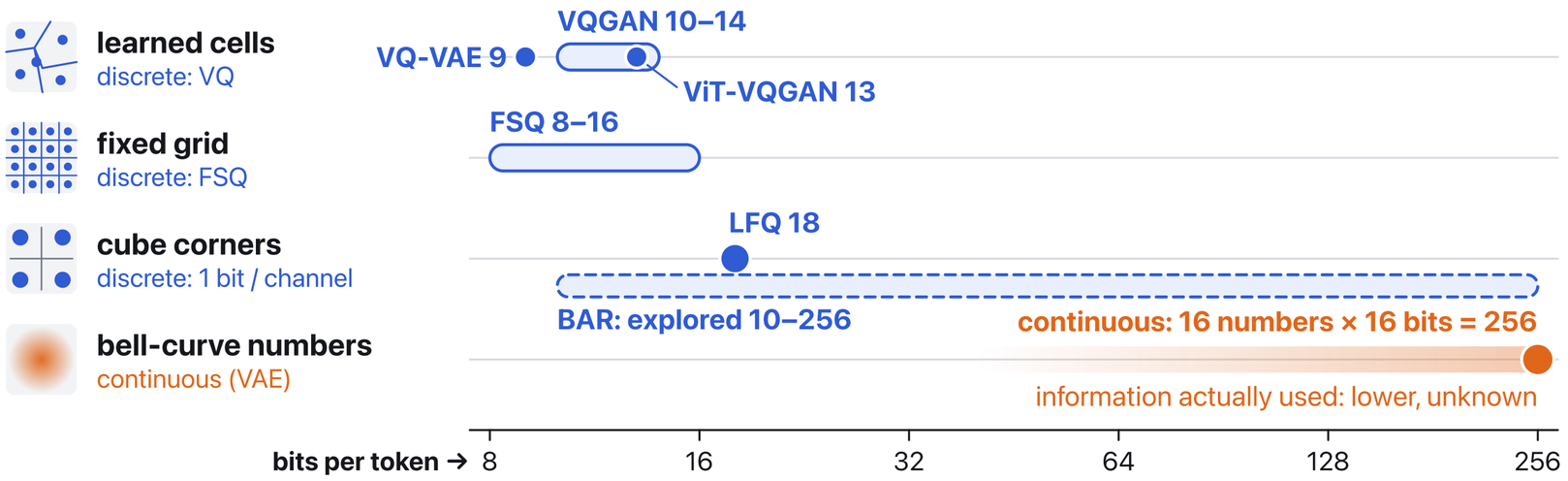
A tidy, bounded space actually gets used.

Sweet spot: Yu et al., BAR, ICML 2026 · Tab. 1 · Teaching illustration: ours · Arithmetic: ours

TAKEAWAYS

What I hope you take home

Inside a computer every latent is a finite code, so discrete and continuous tokenizers sit on one axis: **bits**. Stored bits are an **upper bound** on what they really carry. What differs is **how those bits are shaped for the generator**. More bits keep helping reconstruction, but generation has a sweet spot.



Bits per token, log scale. Stored bits, not information used.

1 · A FLOAT IS A CODE TOO

Compare tokenizers by their bits, whatever the type.

2 · SHAPE MATTERS

A tidy, bounded space actually gets used.

3 · THE GENERATOR HAS TO KEEP UP

More bits help reconstruction, but generation has a sweet spot. It also helps to keep low-level detail away from the main generator.

Sweet spot: Yu et al., BAR, ICML 2026 · Tab. 1 · Teaching illustration: ours · Arithmetic: ours

Where I think this is going

1. **Measure the real bits.** A VAE likely uses far fewer than it stores, but nobody has measured how many.

Where I think this is going

1. **Measure the real bits.** A VAE likely uses far fewer than it stores, but nobody has measured how many.

2. **Understanding and generation.** Today was all generation; understanding models (VLMs) may prefer different latents. How to align or unify the two is open.

Row 3: Dieleman, sander.ai blog, 2025 · Row 4: CVPR 2026 tutorial on unified models, p. 40 · Rows 1-2: ours · Icons: ours

Where I think this is going

1. **Measure the real bits.** A VAE likely uses far fewer than it stores, but nobody has measured how many.

2. **Understanding and generation.** Today was all generation; understanding models (VLMs) may prefer different latents. How to align or unify the two is open.

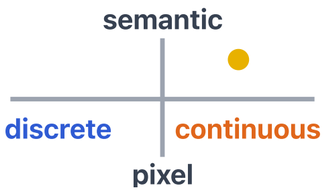
3. **A cheap test.** How easy is a latent to generate, without training a big generator on it?

Where I think this is going

1. **Measure the real bits.** A VAE likely uses far fewer than it stores, but nobody has measured how many.

2. **Understanding and generation.** Today was all generation; understanding models (VLMs) may prefer different latents. How to align or unify the two is open.

3. **A cheap test.** How easy is a latent to generate, without training a big generator on it?



4. **Semantic vs. pixel latents.** Semantic latents capture what is shown, pixel latents how it looks. A second axis, separate from discrete vs. continuous.

Row 3: Dieleman, sander.ai blog, 2025 · Row 4: CVPR 2026 tutorial on unified models, p. 40 · Rows 1-2: ours · Icons: ours

Thank you — questions welcome

Qihang Yu

Amazon FAR